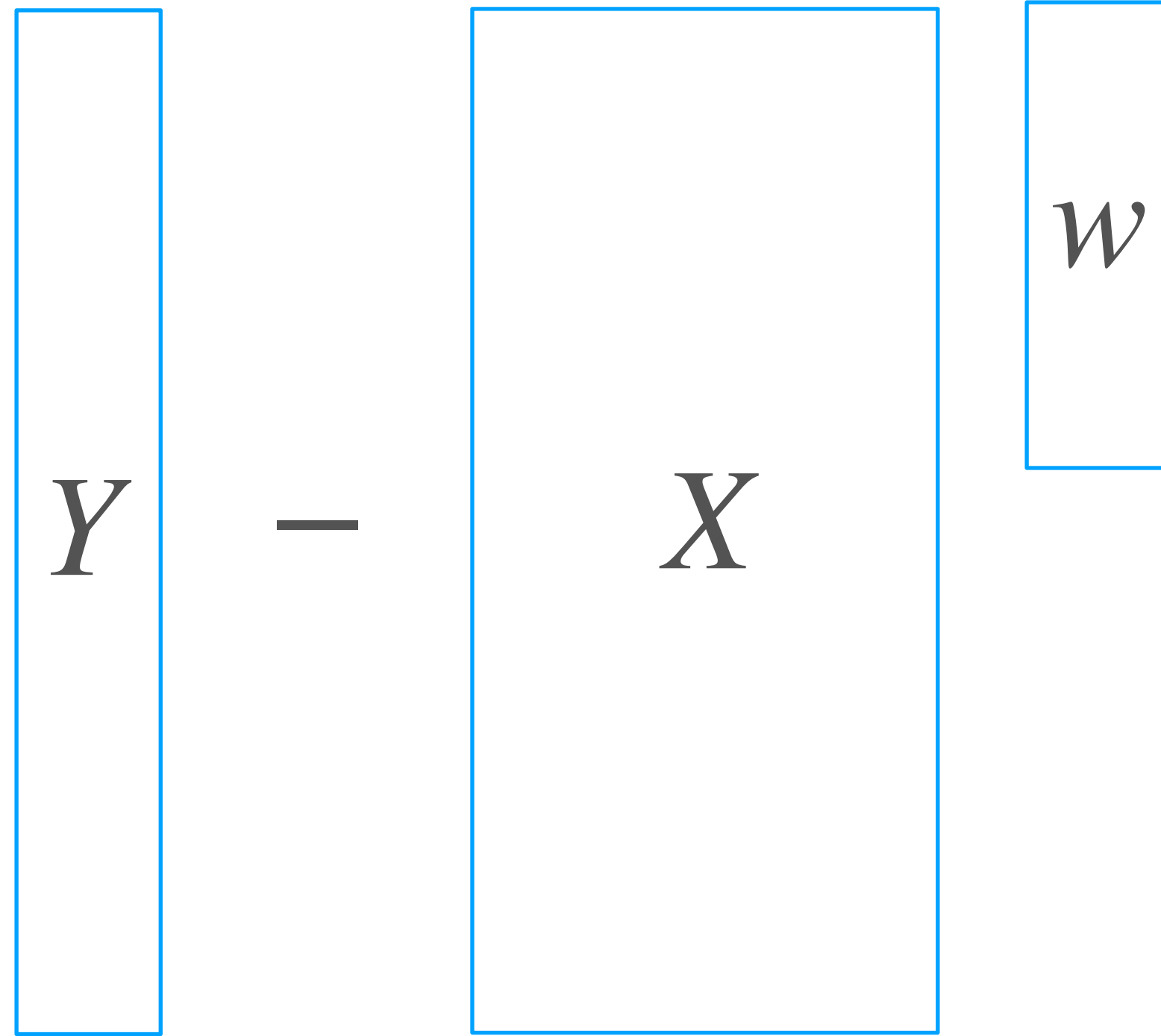


Linear regression



10-701 Introduction to Machine Learning
Geoff Gordon

Gradients



- For any of the solution methods, need gradients

$$\begin{aligned} \blacktriangleright L(w) &= \frac{\lambda}{2} \|w\|^2 + \frac{1}{2} \sum_{t=1}^N (Y^{(t)} - w \cdot X^{(t)})^2 \\ &= \frac{\lambda}{2} \|w\|^2 + \frac{1}{2} \|Y - Xw\|^2 \end{aligned}$$

$$\blacktriangleright \nabla L(w) = \lambda w + \sum_{t=1}^N (Y^{(t)} - w \cdot X^{(t)}) (-X^{(t)})$$

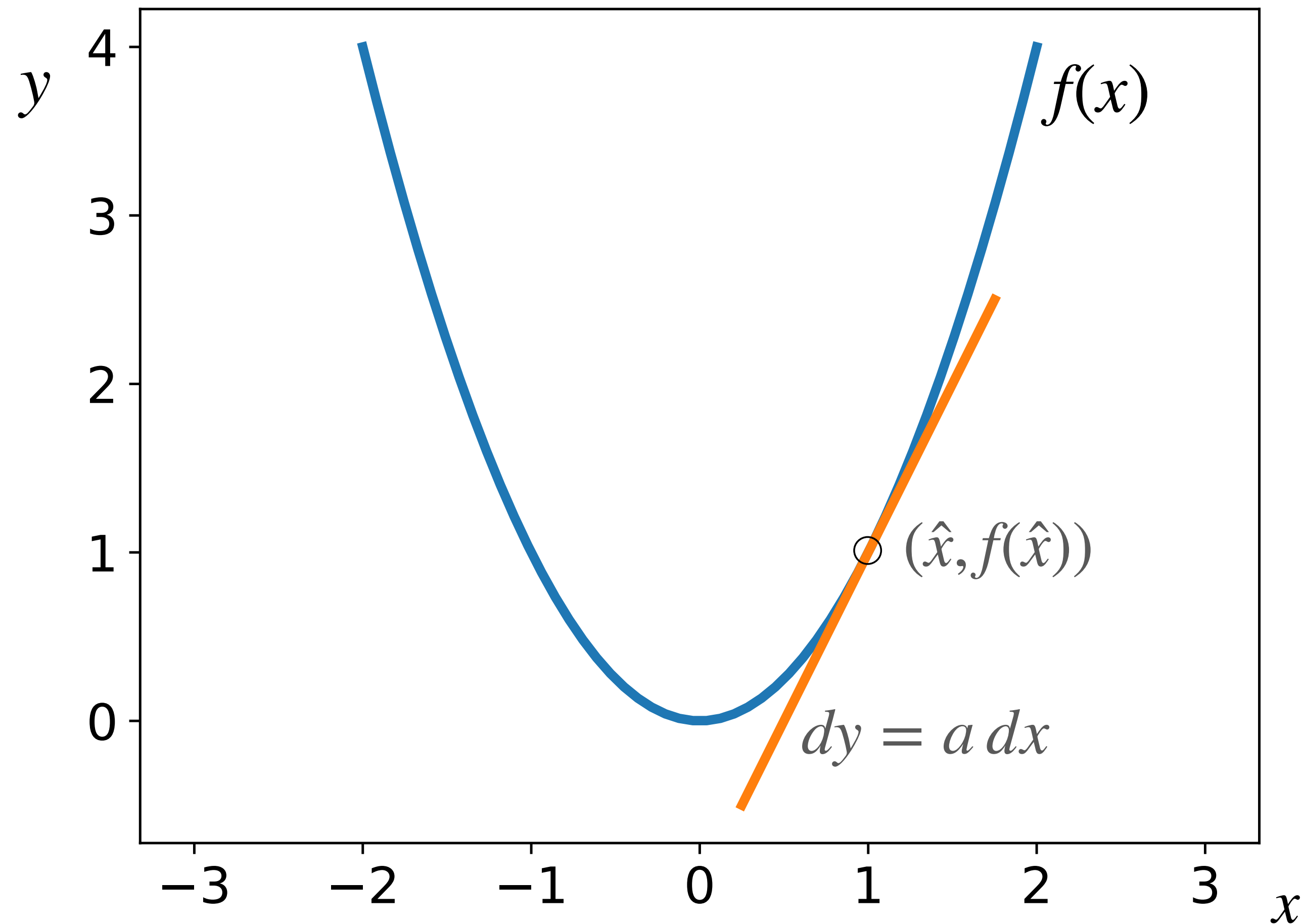
Derivatives everywhere

- The way we derive many ML methods:
 - ▶ start from objective to minimize (here, sum of squared errors plus optional regularizer)
 - ▶ take a few derivatives ($0 = d \text{ objective} / dw$)
 - ▶ find optimum (e.g., by setting to 0 and solving, or SGD)
 - ▶ done!
- Will continue to be a common pattern
- Review of intuition and tools for high-d derivatives, so we can do it fast

Derivatives notation

dx, dy are called
differentials

$$\frac{dy}{dx} \approx a$$

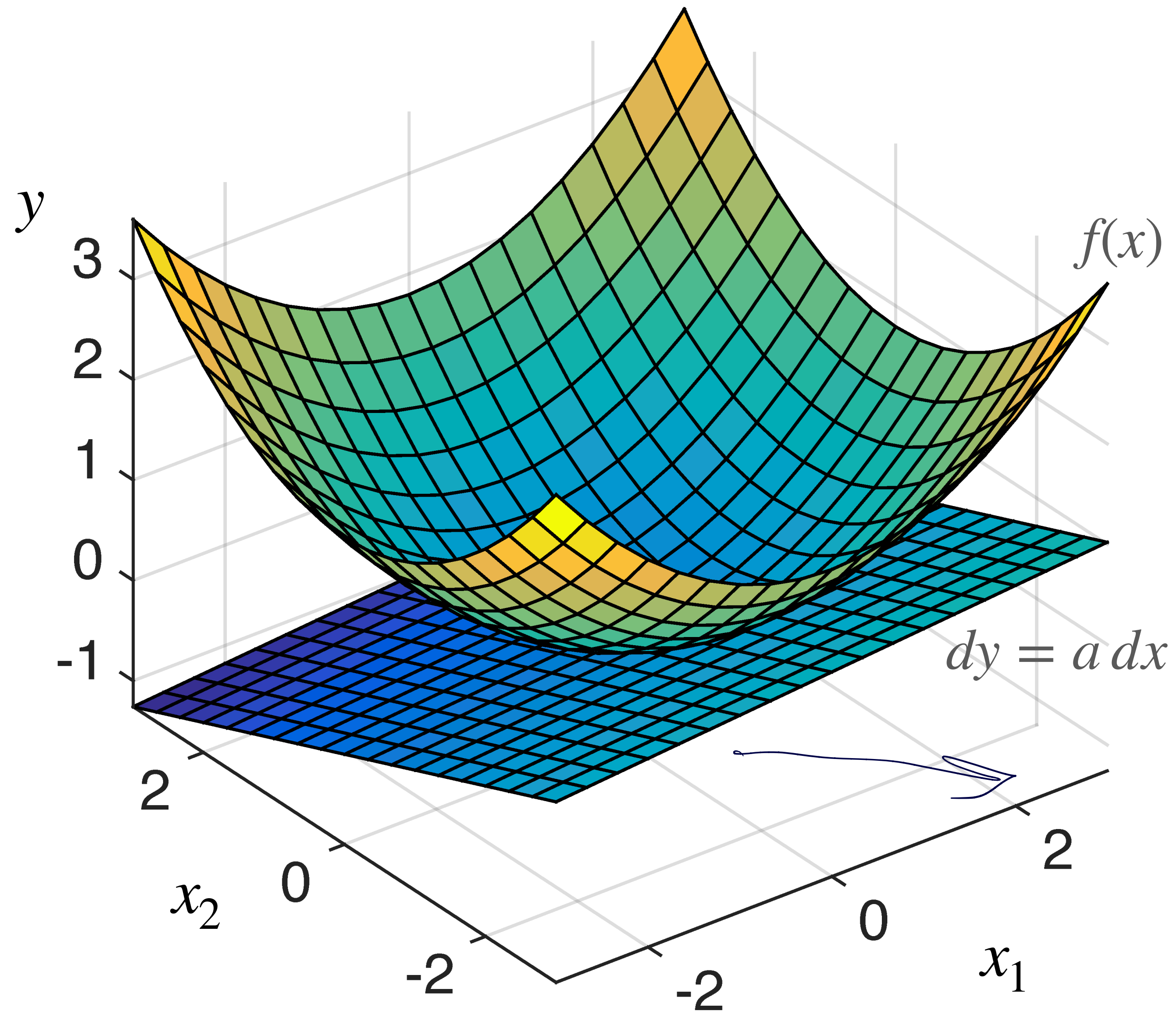


- Function $y = f(x)$
 - ▶ $x \in \mathbb{R}^n, y \in \mathbb{R}^m, f \in \mathbb{R}^n \rightarrow \mathbb{R}^m$
- Derivative is a *linear fn* that *locally* approximates f near $\hat{x}$
 - ▶ $dy = a dx$, where $a \in \mathbb{R}$ is the derivative

Compare older vs. newer notation

- Older version:
 - ▶ $\frac{dy}{dx}$ is atomic: derivative of y wrt x
 - ▶ dx and dy are meaningless on their own
 - ▶ focused on infinitesimals: change in x or $y \rightarrow 0$
- Newer version:
 - ▶ dx and dy have separate meaning: terms in a linear function (the Taylor expansion)
 - ▶ dx and dy can be any size: linear function is well defined even for large changes in x or y (though accuracy of Taylor approximation can be bad if dx , dy are big)
- Heuristic to convert notation: “divide through” or “multiply through” by dx (only actually makes sense in scalar case, but often succeeds anyway)

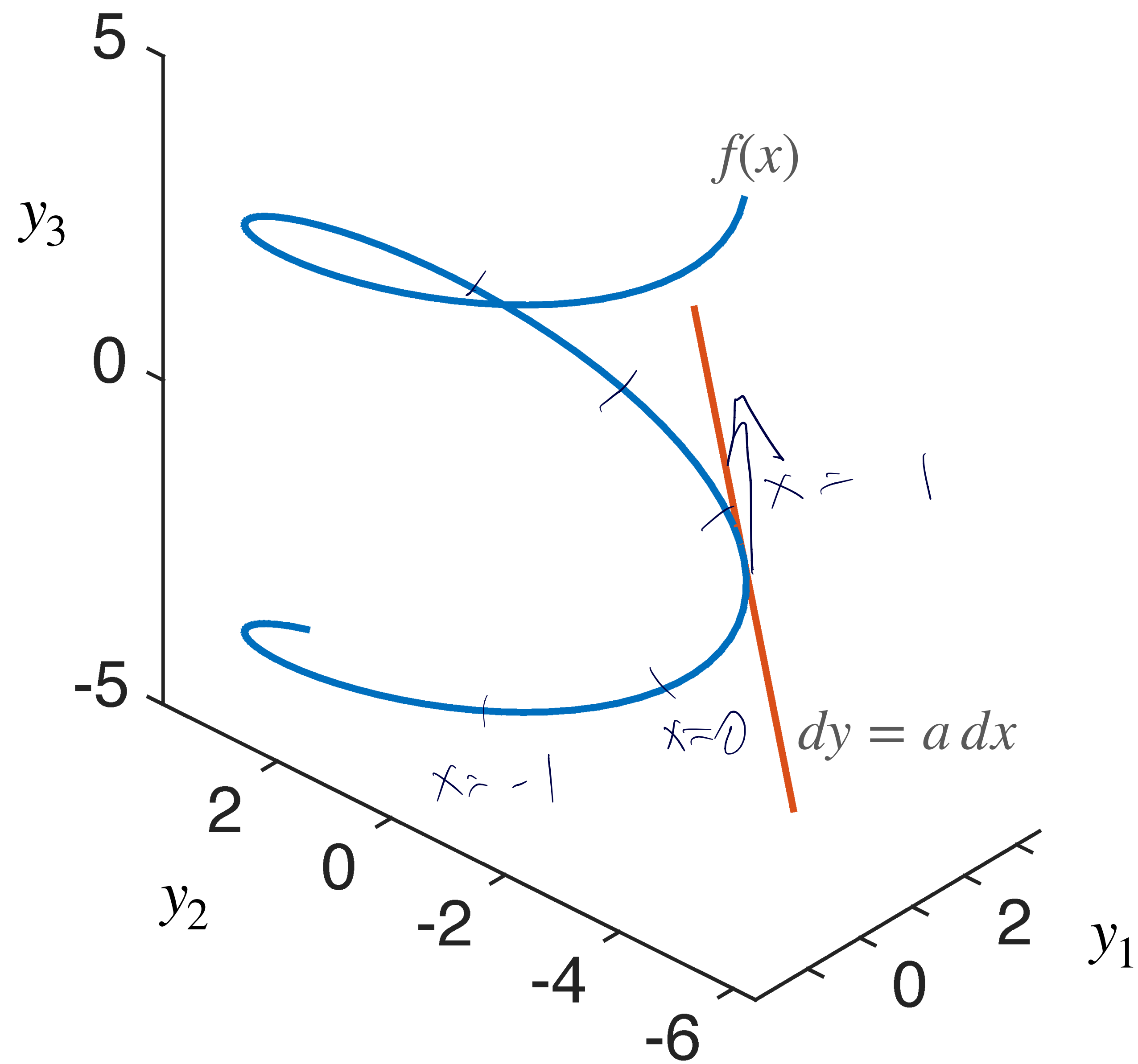
Higher dimensions



$$\mathbb{R}^n \rightarrow \mathbb{R}^m$$
$$m = 1, n = 2$$

Function $y = f(x)$ $dy = a dx$ $a \in \mathbb{R}^{1 \times 2}$

Higher dimensions



$$\mathbb{R}^n \rightarrow \mathbb{R}^m$$

$$m = 3, n = 1$$

Function $y = f(x)$ $dy = a \, dx$ $a \in \mathbb{R}^{3 \times 1}$

Notation for linear functions

output shape (derivative of)

input shape (with respect to)			
	Scalar	Vector	Matrix
Scalar	$ds = a dt$	$ds = u^T dv$	$ds = \text{tr}(M^T dN)$
Vector	$du = v ds$	$du = M dv$	×
Matrix	$dM = N ds$	×	×

- a, s, t : scalars u, v : column vectors M, N : matrices
- note: $ds = a dt$ is a synonym for $a = \frac{ds}{dt}$, etc.
- × means no common notation; see torch.einsum

Notation for linear functions

output shape (derivative of)

input shape (with respect to)			
	Scalar	Vector	Matrix
Scalar	$d \text{ slope } dt$	$(\text{ gradient },$	$ds = \text{tr}(M^T dN)$
Vector	$(\text{ velocity },$	$\text{ Jacobian },$	$\times$
Matrix	$dM = N ds$	$\times$	$\times$

- a, s, t : scalars u, v : column vectors M, N : matrices
- note: $ds = a dt$ is a synonym for $a = \frac{ds}{dt}$, etc.
- $\times$ means no common notation; see torch.einsum

Inner product

- With scalar output s , for any shape of input X , can write $ds = \langle A, dX \rangle$ where A, X, dX all have same shape
 - ▶ $\langle \cdot, \cdot \rangle$ is ***inner product***, in our case $\sum_{\text{index lists } I} A_I dX_I$
 - ▶ e.g., if A, dX are $m \times n$ matrices, then
$$\langle A, dX \rangle = \sum_{i=1}^m \sum_{j=1}^n A_{ij} dX_{ij}$$
- Expressions on previous slide are equivalent to above
 - ▶ most complex is $\text{tr}(M^\top dN)$
 - ▶ tr is sum of diagonal elements of a square matrix
$$[M^\top dN]_{jk} =$$
 - ▶ sum of diagonal elements means set $j = k$, sum over j :

Shape conventions

- Derivative = coefficient of ds, dt, dv, dN on RHS
- Ambiguity: can write $\langle M, dN \rangle$ or $\text{tr}(M^\top dN)$
 - ▶ is the derivative M or $M^\top$?
- In general, x, y , and derivative $\frac{dy}{dx}$ are **tensors** (multidimensional arrays of numbers)
- Convention: in derivative $\frac{dy}{dx}$
 - ▶ the dimensions of y come first, then x
 - ▶ in same order as dimensions of y and x
- For example: if y is 3×2 and x is $4 \times 6 \times 5$ then $\frac{dy}{dx}$ is $3 \times 2 \times 4 \times 6 \times 5$

Special cases

- If y is a scalar (dimensions $()$), then $\frac{dy}{dx}$ is the same shape as x
 - ▶ derivative of a scalar wrt a 5×3 matrix is a 5×3 matrix
- If x and y are vectors, $\frac{dy}{dx}$ (the Jacobian) is the same shape as a linear function from x to y
 - ▶ derivative of a 3-vector wrt a 6-vector is a 3×6 matrix

Useful identities

- $\text{scalar} = \text{scalar}^\top = \text{tr}(\text{scalar})$

- Trace rotation

- ▶ $\text{tr}(ABC) = \text{tr}(CAB)$ when A, B, C have compatible dimensions

compatible dimensions:

- 2nd dimension of A = 1st dimension of B
- 2nd dimension of B = 1st dimension of C
- 2nd dimension of C = 1st dimension of A

- Transpose-Hadamard

- ▶ $A \circ B = \text{Hadamard (componentwise) product,}$

$$[A \circ B]_{ij} = A_{ij}B_{ij}$$

- ▶ $A^\top(B \circ C) = (A \circ B)^\top C$

Tools for derivatives: linearity, chain, product

- Linearity:
 - ▶ $d(ax + b) = d(ax) + \cancel{d(b)} = a dx$, $d(AX + B) = A dX$, ...
- Chain rule for $f(g(x))$
 - ▶ old notation: $\frac{df}{dx} = \frac{df}{dg} \frac{dg}{dx}$ (“cancel the dg ”)
 - ▶ new: if $df = A dg$, $dg = B dx$ then $df = A B dx$
- Product rule for $f(x) g(x)$
 - ▶ $d(fg) = df g + f dg$
 - ▶ works for any kind of product (matrix product, vector cross product, elementwise (Hadamard) product, vector convolution, Khatri-Rao product ...)
 - ▶ unlike scalar case, order can matter (products are not necessarily commutative)

$$Y \in \mathbb{R}^{N \times 1}$$

$$X \in \mathbb{R}^{N \times K}$$

Gradient practice: Linear regression in one slide

$$w \in \mathbb{R}^{K \times 1}$$

$$L = \|Y - Xw\|^2 = (Y - Xw)^T(Y - Xw)$$

$$L = Y^T Y - 2Y^T Xw + w^T X^T Xw$$

$$\begin{aligned} dL &= \cancel{dY^T Y} - 2 dY^T Xw + d w^T X^T Xw \\ &= 0 - 2 Y^T X dw + w^T X^T X dw + (dw)^T X^T Xw \\ &= -2 Y^T X dw + 2 w^T X^T X dw \\ &= \langle -2 X^T Y + 2 X^T Xw, dw \rangle \end{aligned}$$

Learning objectives— linear regression, derivatives

- What is the linear regression model? How can we represent sum of squared errors with matrix notation?
- How should we pick and pre-process features for linear regression?
- What are the effects of common regularizers? How do they affect model complexity and overfitting?
- How can we find gradients of vector, matrix, and tensor functions?
- What is the gradient of the linear regression loss?
- What are the normal equations, and how can we solve them exactly?

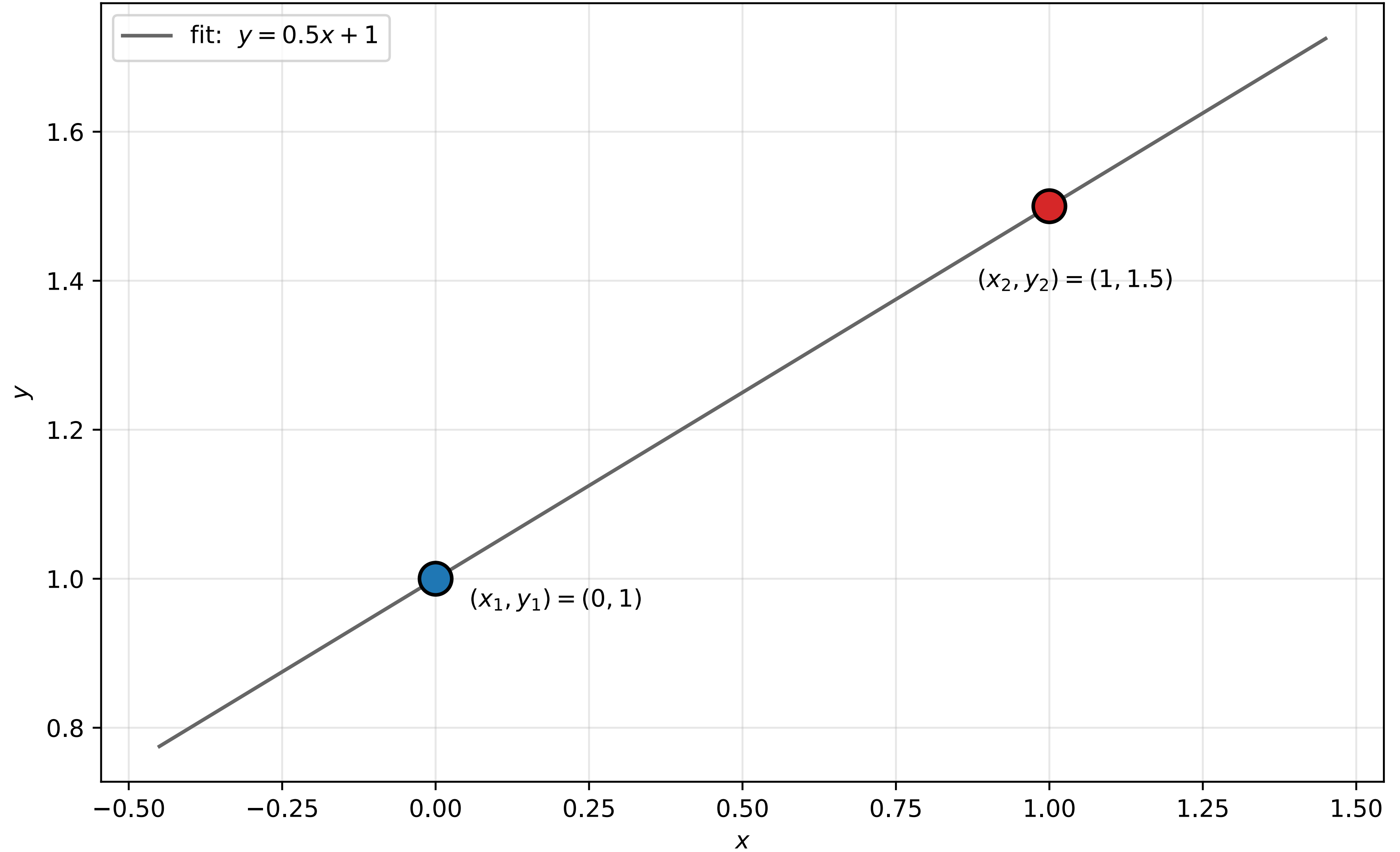
Gradient descent



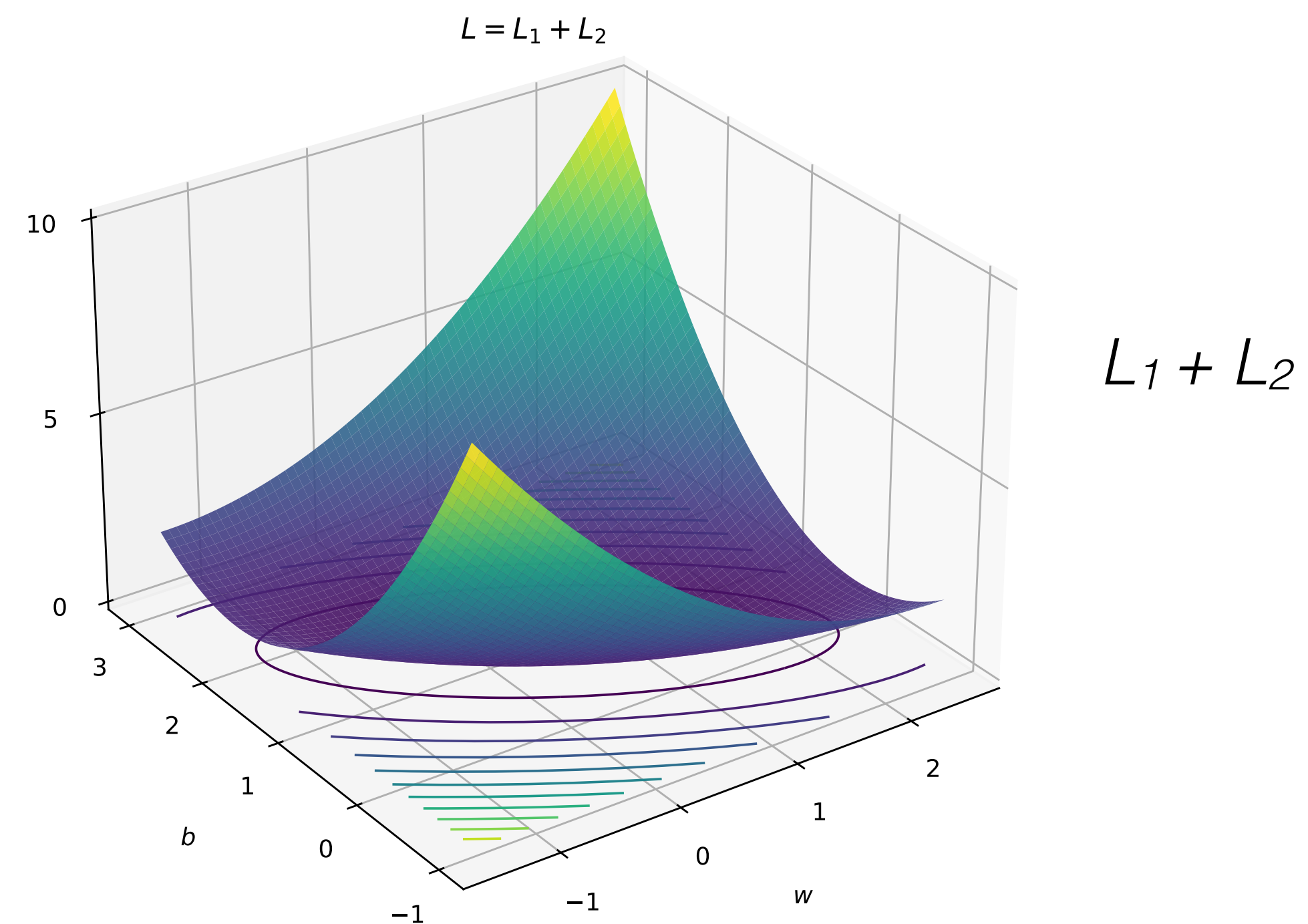
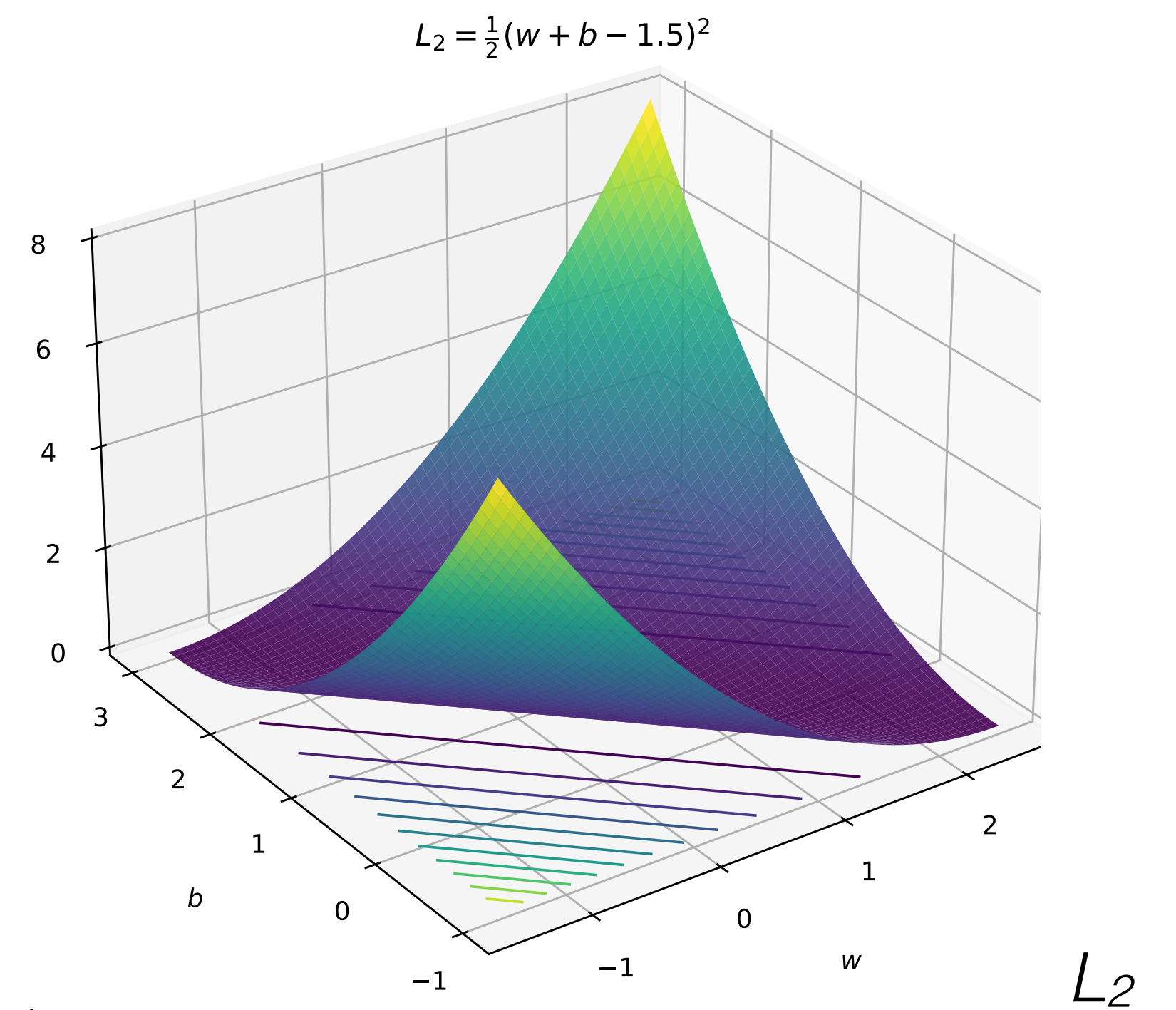
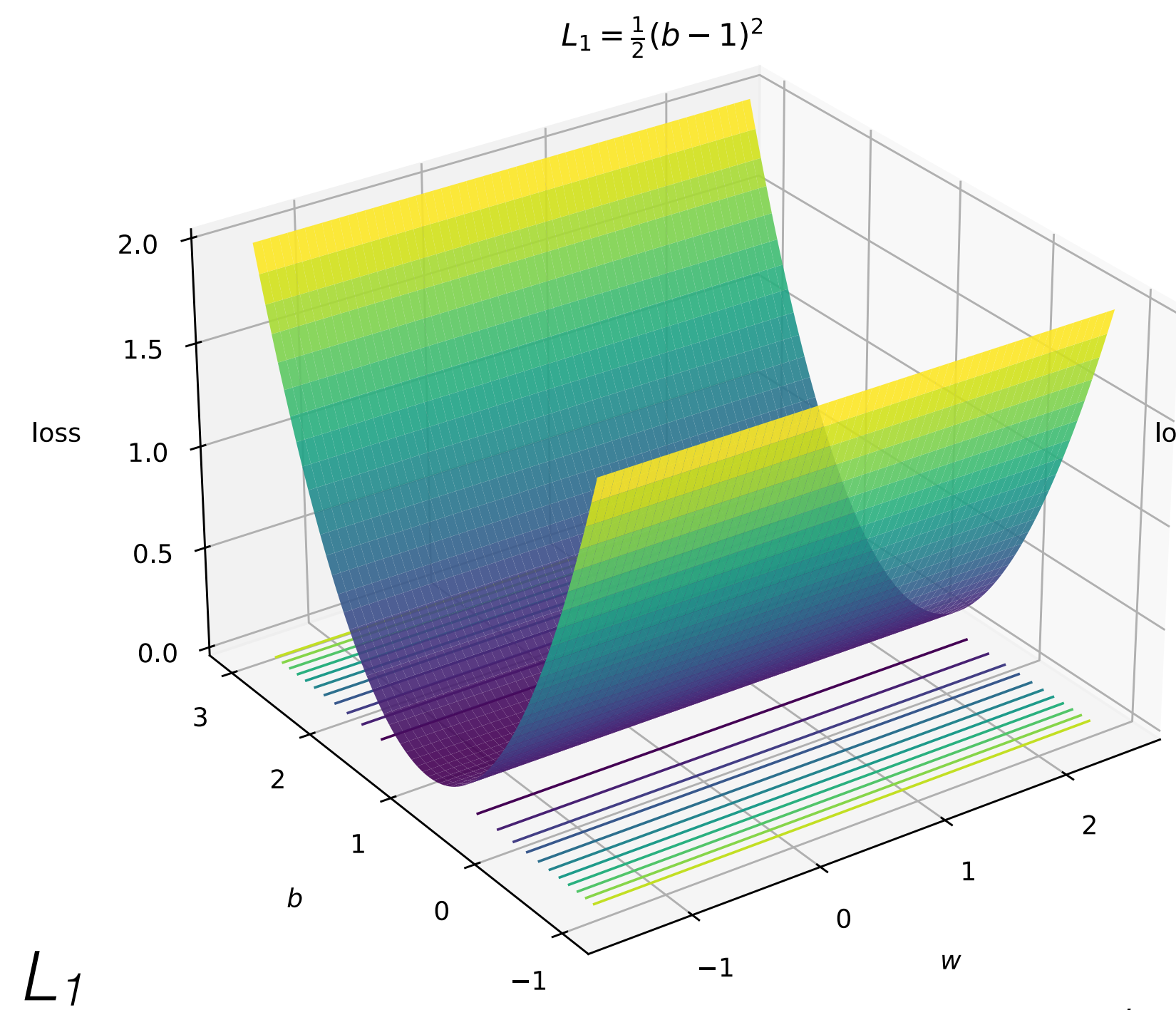
10-701 Introduction to Machine Learning
Geoff Gordon

thanks to Matt Gormley and Henry Chai for some content

A very simple regression problem

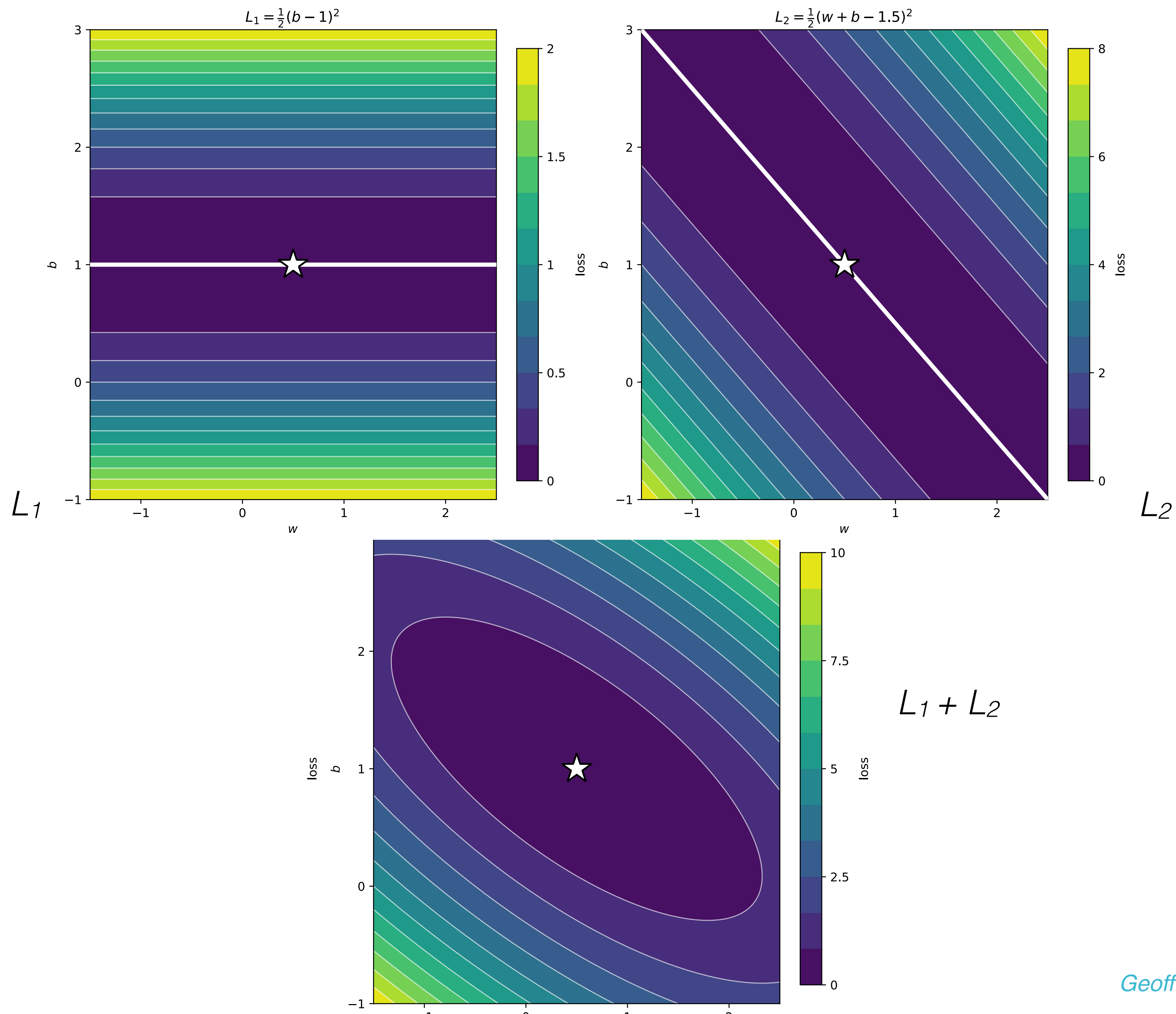


Loss function: sum of squared errors



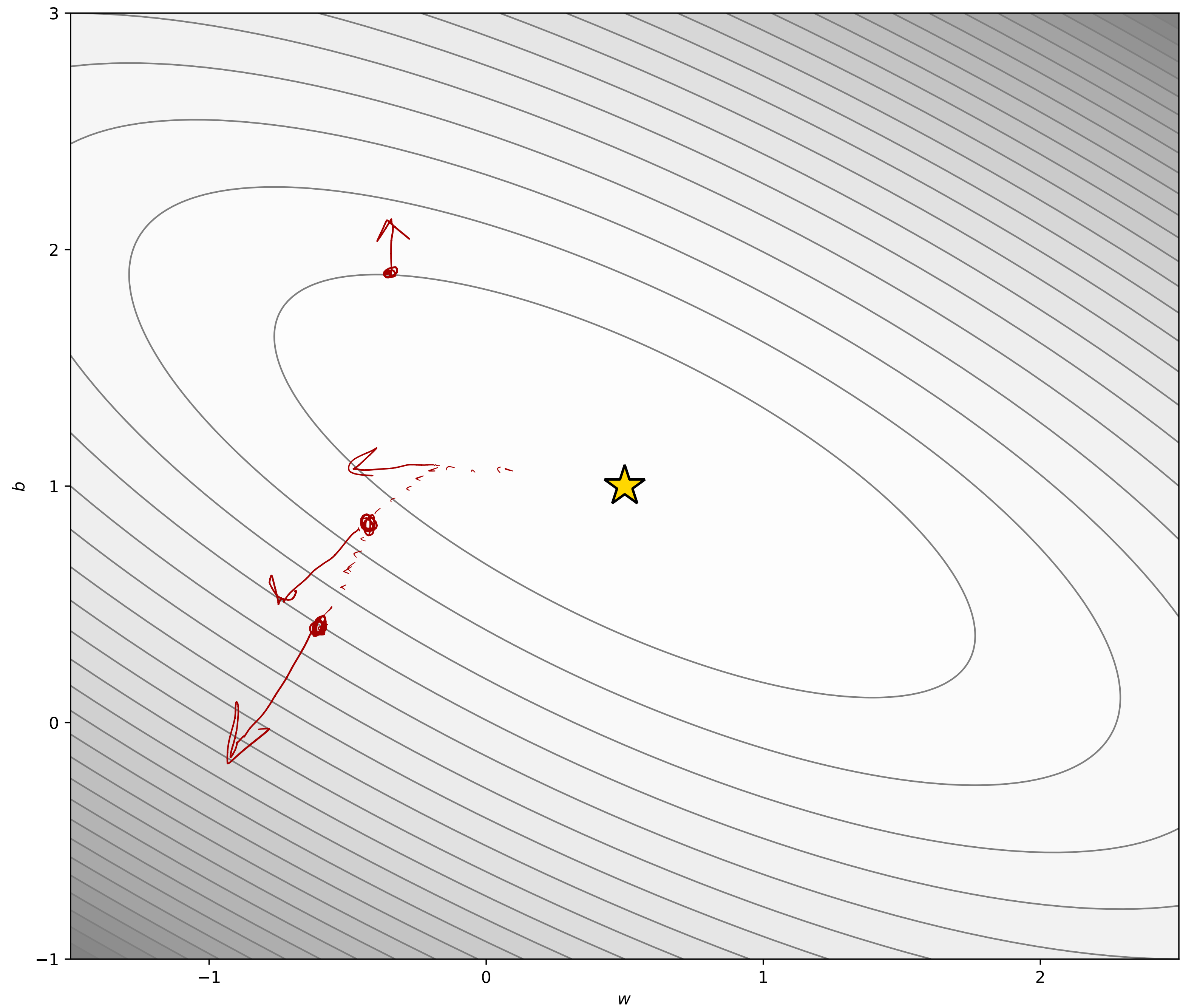
As contour plots

Colors show height
Each contour is equal height



***Find the
optimal w, b***

Gradients $\perp$ contours



Gradient descent

Algorithm: gradient descent

Inputs: loss function $L(\theta)$, initial value $\theta^{(1)}$, step size η

For $i \leftarrow 1, 2, \dots$:

if **converged** break

$g \leftarrow \nabla L(\theta^{(i)})$ $\leftarrow$ for linear/ridge regression, $\lambda w + X^T(Xw - Y)$

$\theta^{(i+1)} \leftarrow \theta^{(i)} - \eta g$

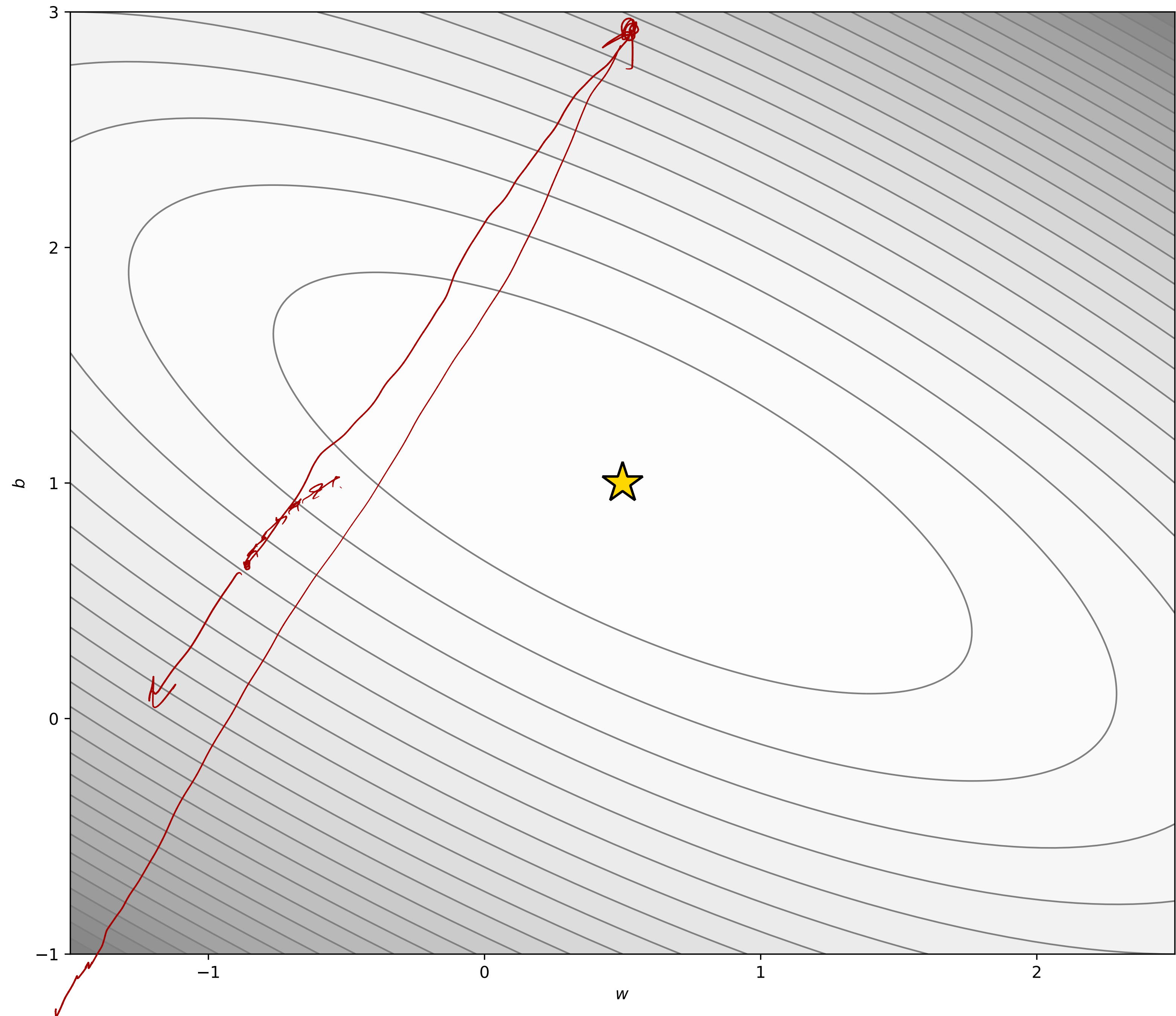
return $\theta^{(i)}$

- Converged: e.g., validation error stops decreasing
- Problem: have to set hyperparameter η (also $\theta^{(1)}$)
- Variant: $\eta^{(i)}$ (can work better but more hyperparameters)

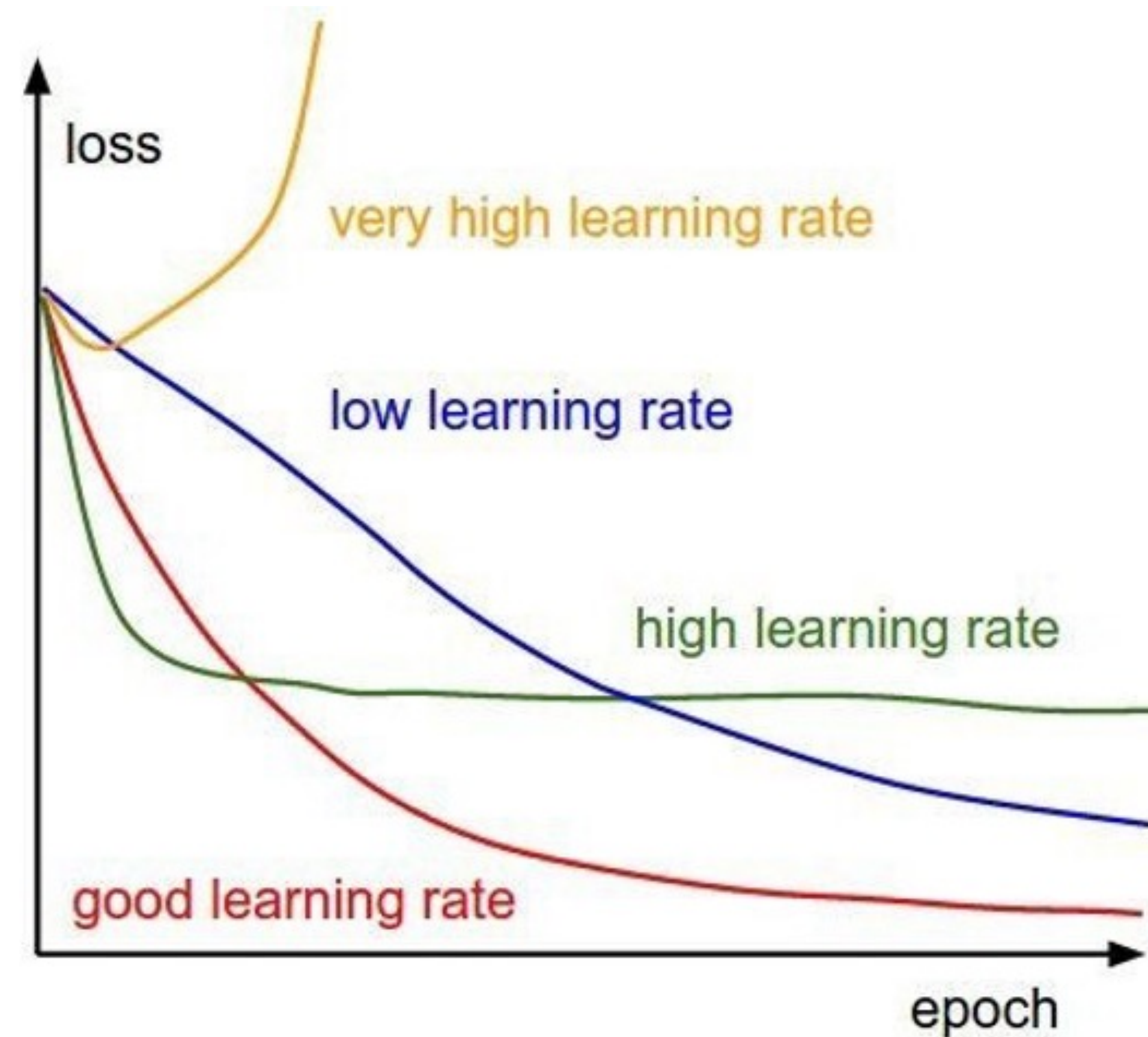
Problem with having to pick learning rate

*Limit on η : **curvature!***

*As long as loss is locally
near linear, we keep
going down; when that
approximation fails we
slow or reverse*



Effect of learning rate



- Learning rate too big: divergence or noisy convergence
- Too small: slow progress
- Can adapt learning rate over time (warmup or anneal) to try to get best of both worlds

Why (not) gradient descent?

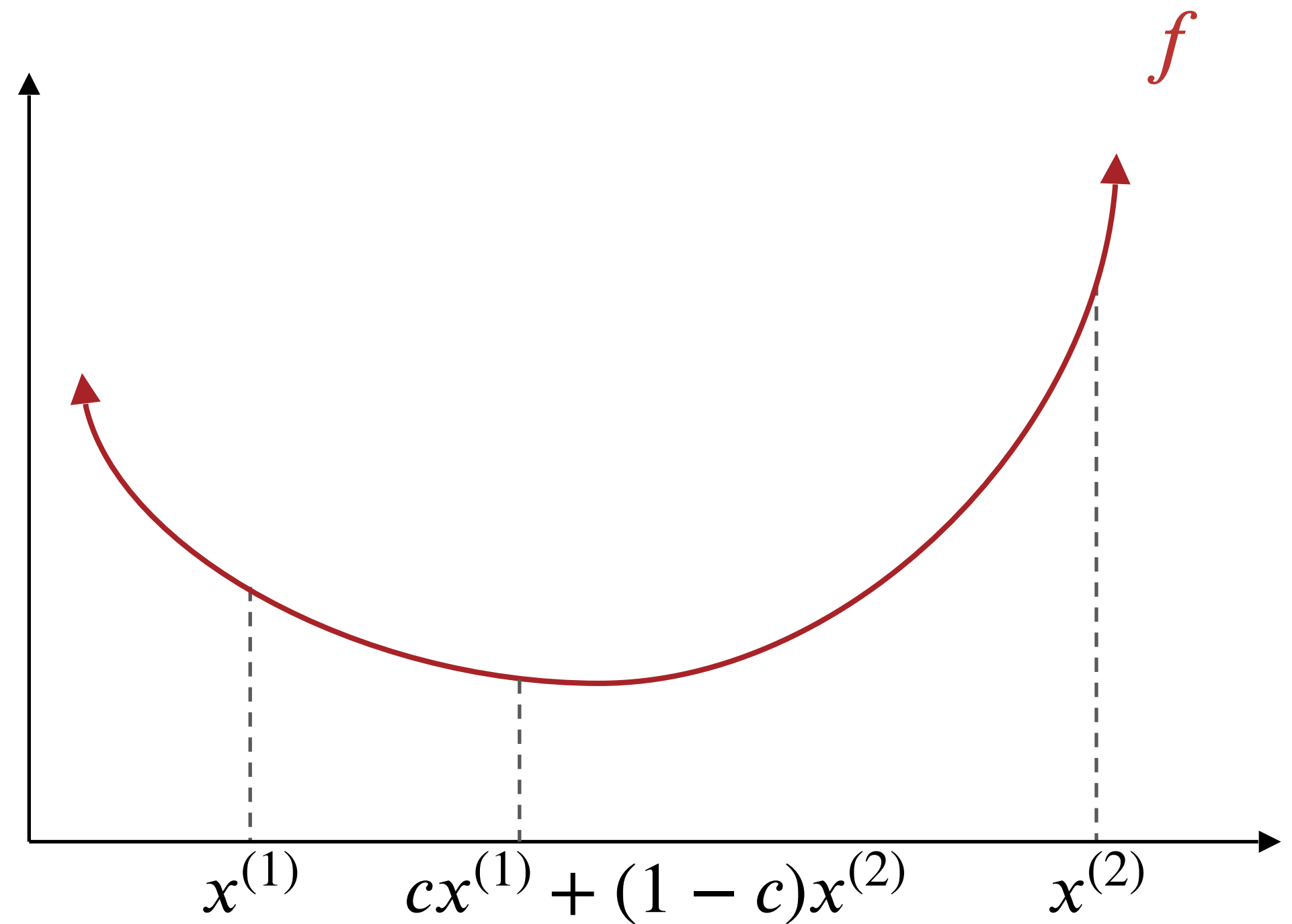
- Globally convergent! *→ on linear regression*
- Cost: $O(Nk)$ per iteration (# examples $\times$ # features)
 - ▶ “good enough” answer = not too many iterations (treat as independent of N, k), as long as we can pick η
 - ▶ vs. $O(Nk^2)$ for `linalg.lstsq`
 - ▶ fast for small problems
 - ▶ not fast for large problems, but lower order in k
- Parallelizable: distribute gradient calculation across multiple GPUs or a cluster
- Extensions are available (momentum, preconditioning, parameter server)
- Works for a lot of models beyond just linear regression!

Convexity

- A function $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is convex if

$$\forall \mathbf{x}^{(1)} \in \mathbb{R}^D, \mathbf{x}^{(2)} \in \mathbb{R}^D \text{ and } 0 \leq c \leq 1$$

$$f(c\mathbf{x}^{(1)} + (1 - c)\mathbf{x}^{(2)}) \leq cf(\mathbf{x}^{(1)}) + (1 - c)f(\mathbf{x}^{(2)})$$

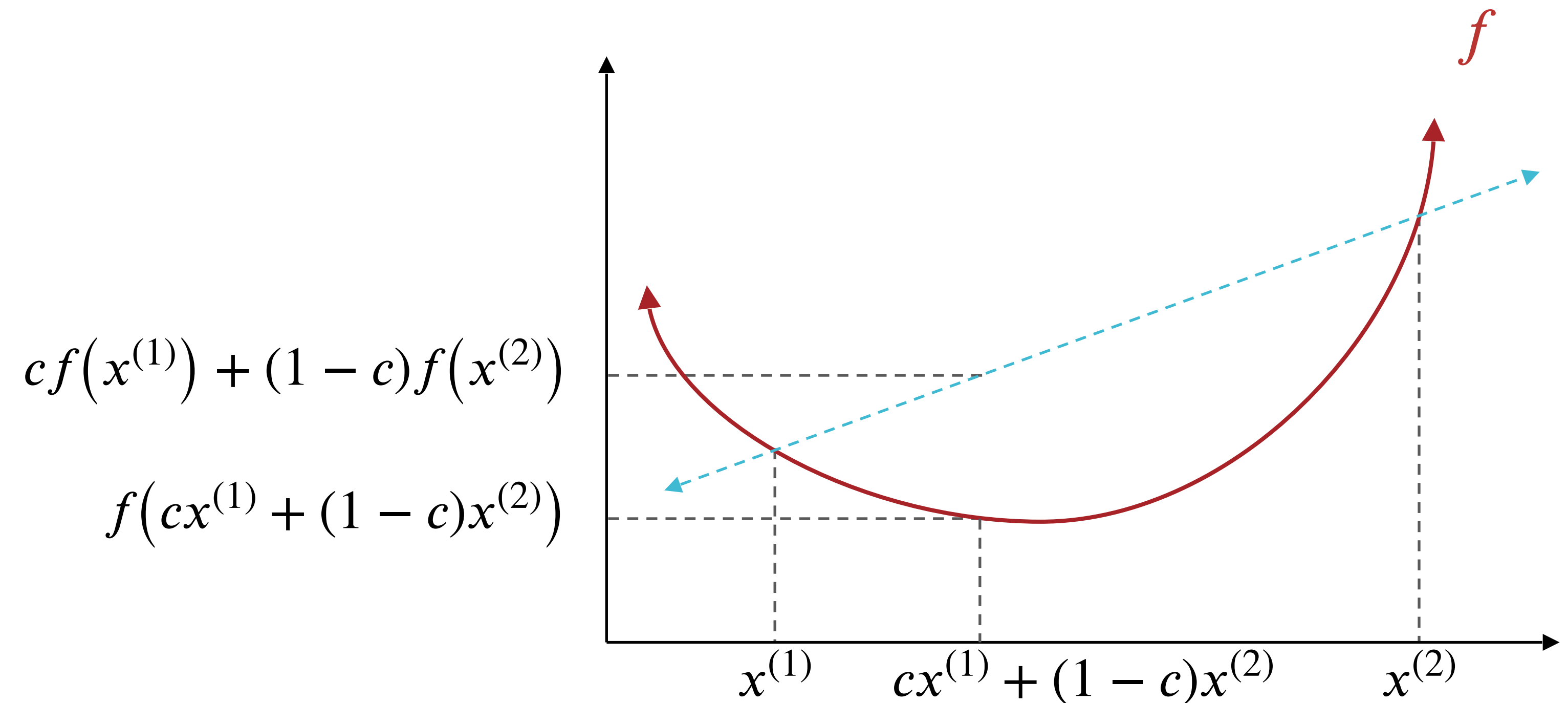


Convexity

- A function $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is convex if

$$\forall \mathbf{x}^{(1)} \in \mathbb{R}^D, \mathbf{x}^{(2)} \in \mathbb{R}^D \text{ and } 0 \leq c \leq 1$$

$$f(c\mathbf{x}^{(1)} + (1 - c)\mathbf{x}^{(2)}) \leq cf(\mathbf{x}^{(1)}) + (1 - c)f(\mathbf{x}^{(2)})$$

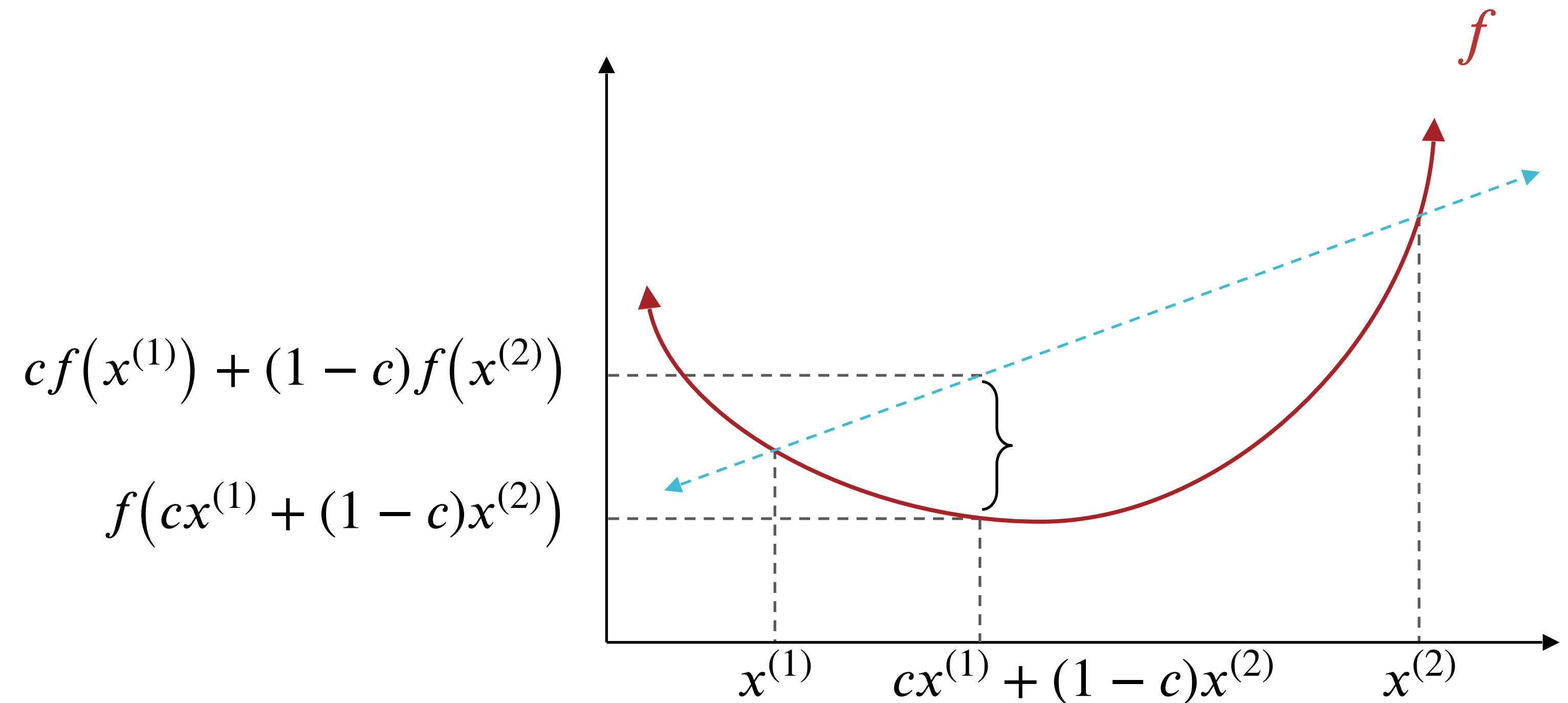


Convexity

- A function $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is convex if

$$\forall \mathbf{x}^{(1)} \in \mathbb{R}^D, \mathbf{x}^{(2)} \in \mathbb{R}^D \text{ and } 0 \leq c \leq 1$$

$$f(c\mathbf{x}^{(1)} + (1-c)\mathbf{x}^{(2)}) \leq cf(\mathbf{x}^{(1)}) + (1-c)f(\mathbf{x}^{(2)})$$

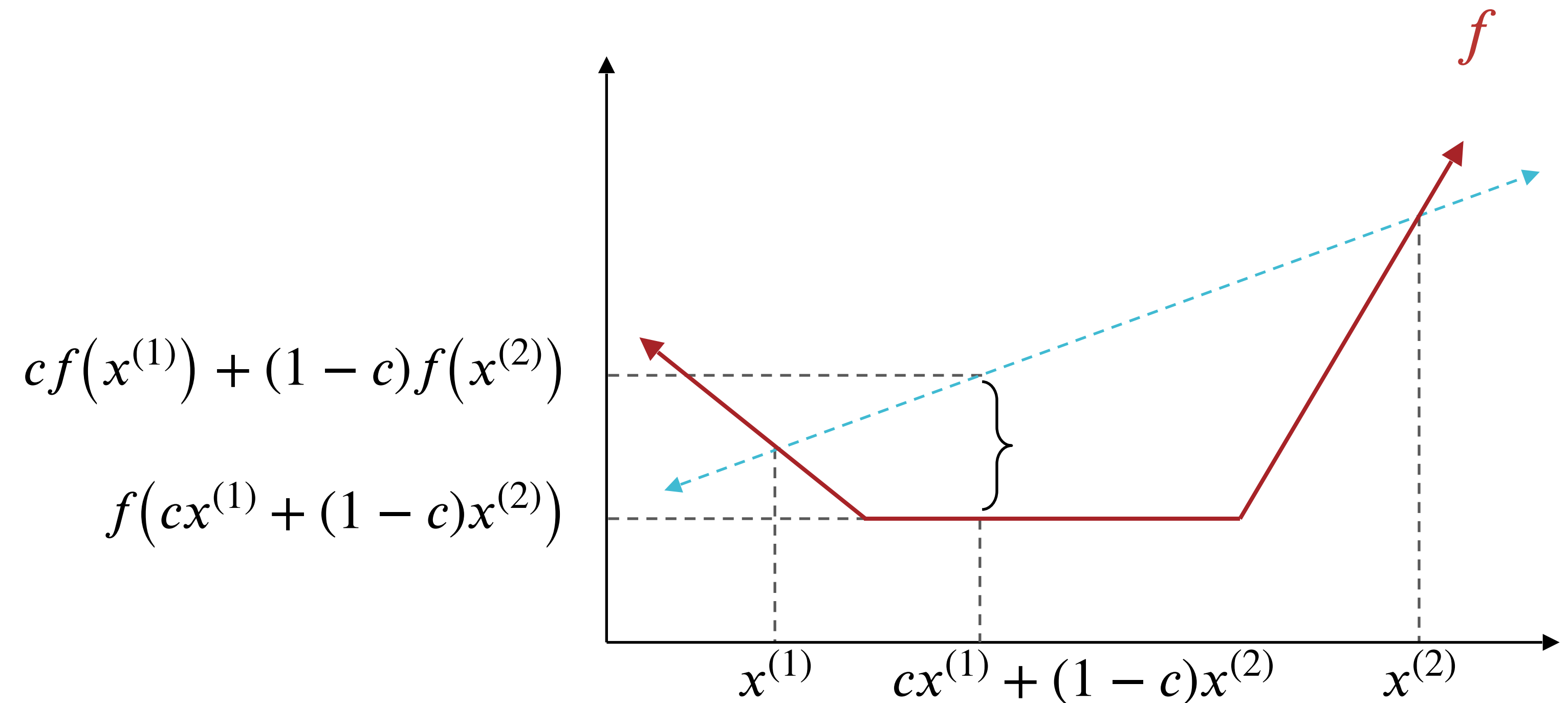


Convexity

- A function $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is convex if

$$\forall \mathbf{x}^{(1)} \in \mathbb{R}^D, \mathbf{x}^{(2)} \in \mathbb{R}^D \text{ and } 0 \leq c \leq 1$$

$$f(c\mathbf{x}^{(1)} + (1 - c)\mathbf{x}^{(2)}) \leq cf(\mathbf{x}^{(1)}) + (1 - c)f(\mathbf{x}^{(2)})$$



Convexity

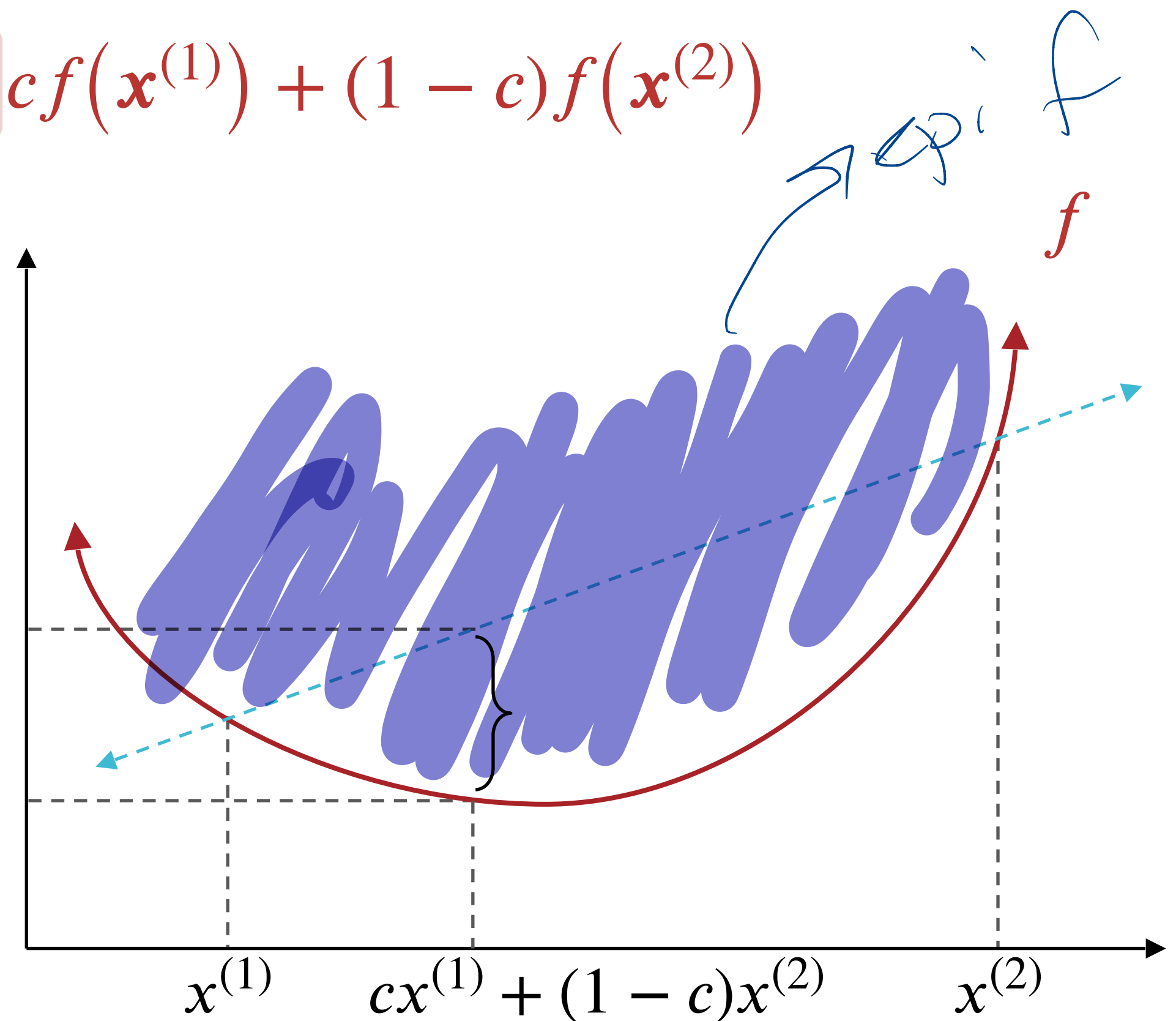
- A function $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is *strictly convex* if

$$\forall \mathbf{x}^{(1)} \in \mathbb{R}^D, \mathbf{x}^{(2)} \in \mathbb{R}^D \text{ and } 0 < c < 1$$

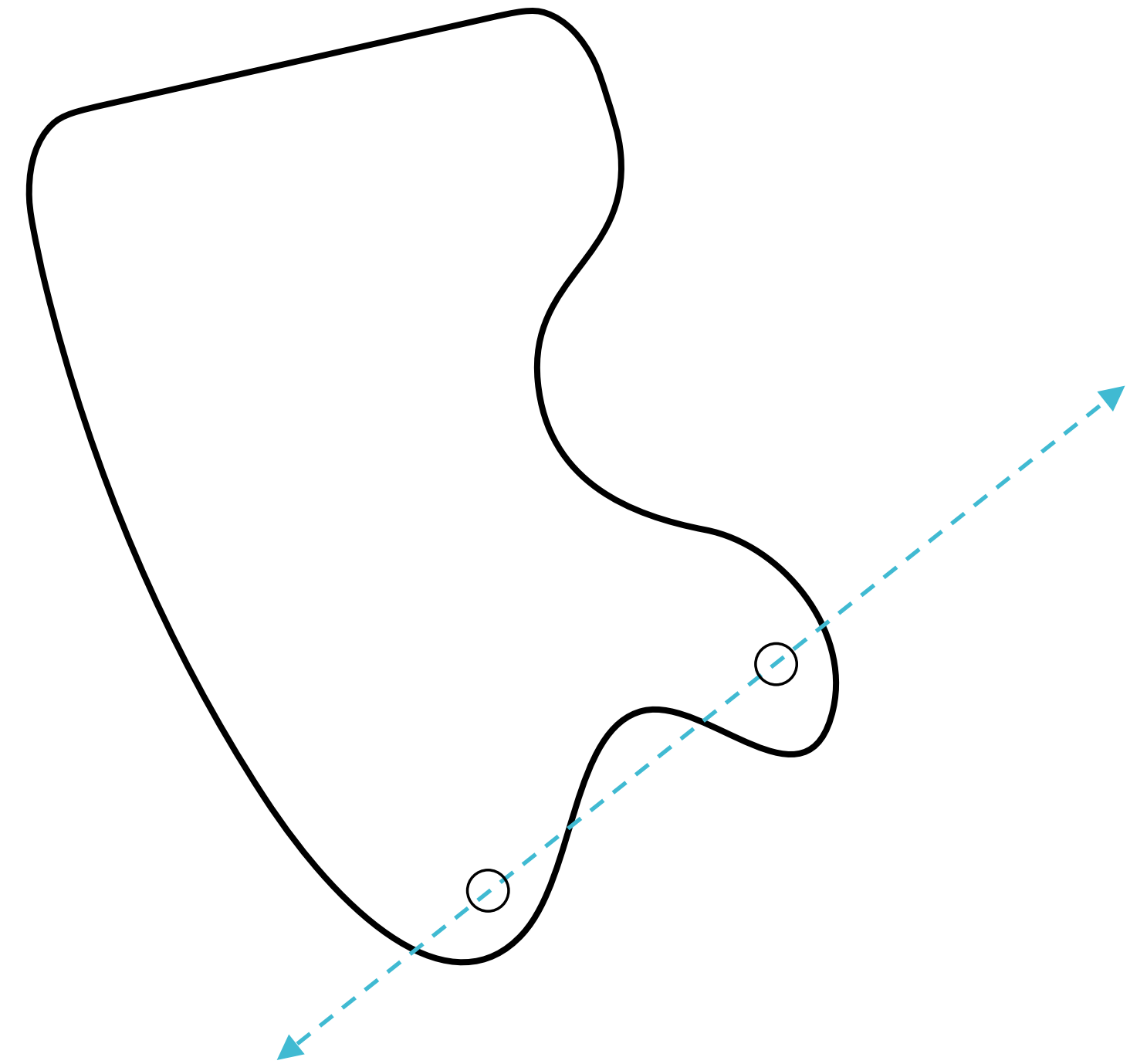
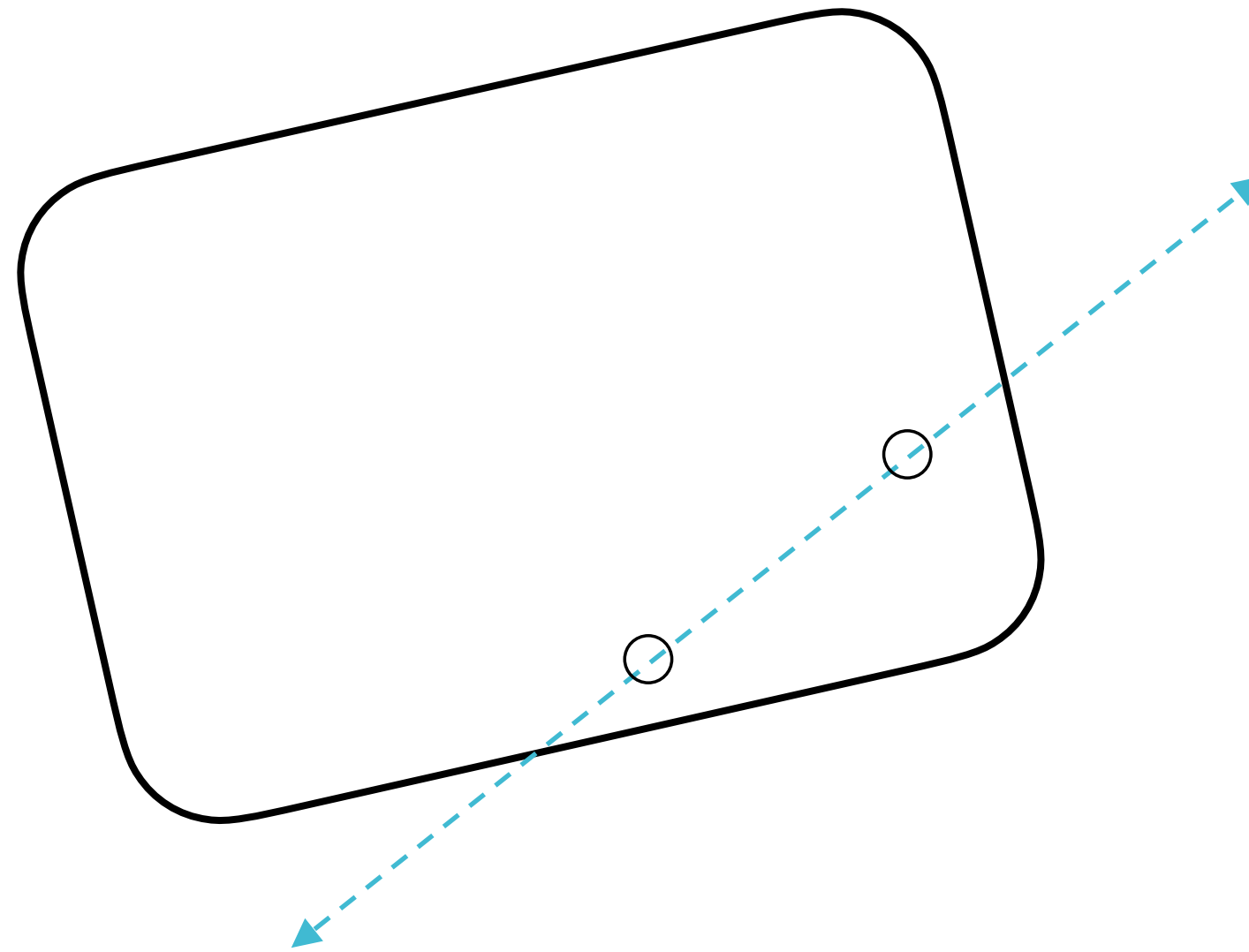
$$f(c\mathbf{x}^{(1)} + (1 - c)\mathbf{x}^{(2)}) < cf(\mathbf{x}^{(1)}) + (1 - c)f(\mathbf{x}^{(2)})$$

$$cf(\mathbf{x}^{(1)}) + (1 - c)f(\mathbf{x}^{(2)})$$

$$f(c\mathbf{x}^{(1)} + (1 - c)\mathbf{x}^{(2)})$$

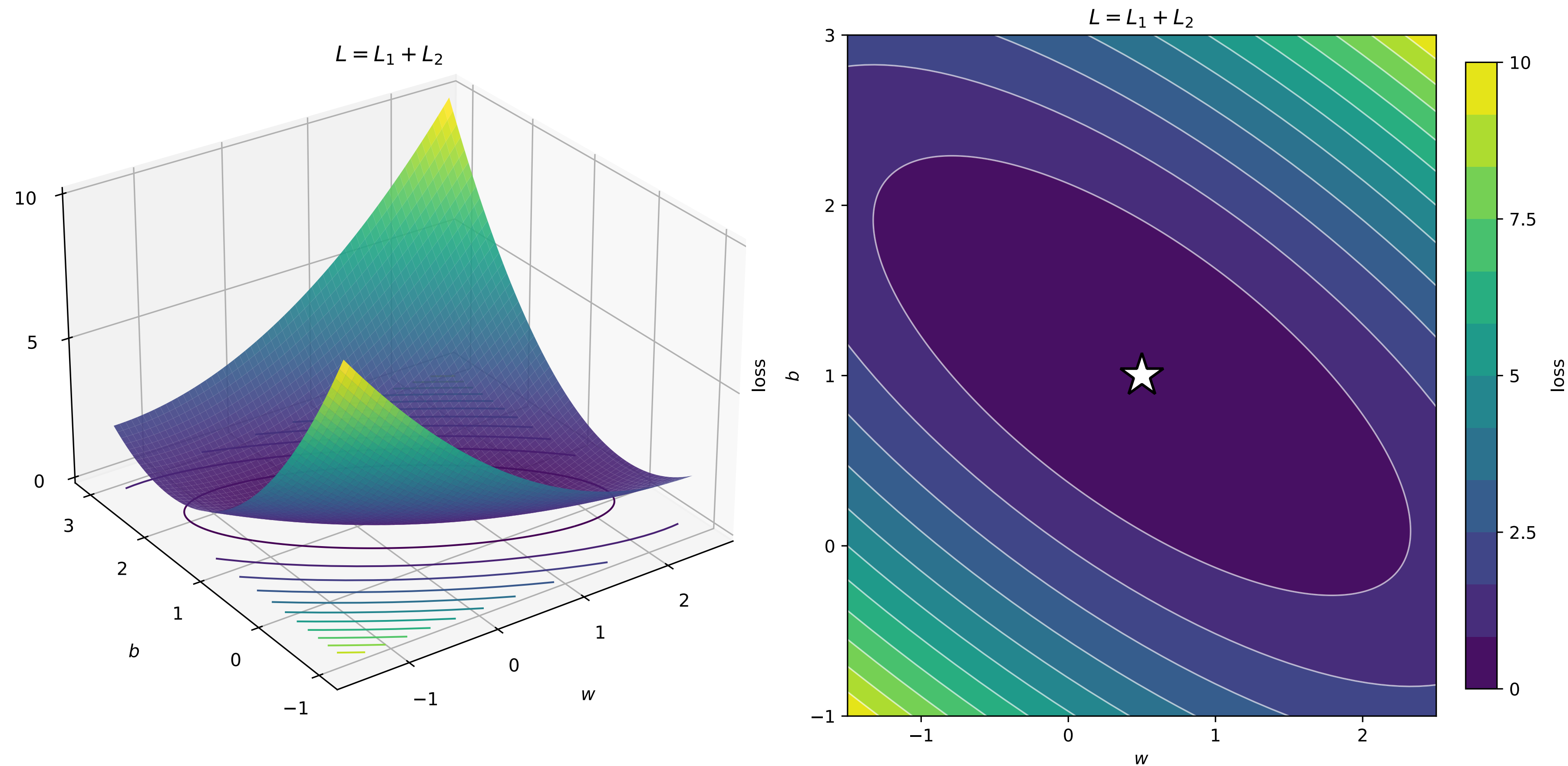


Convex sets



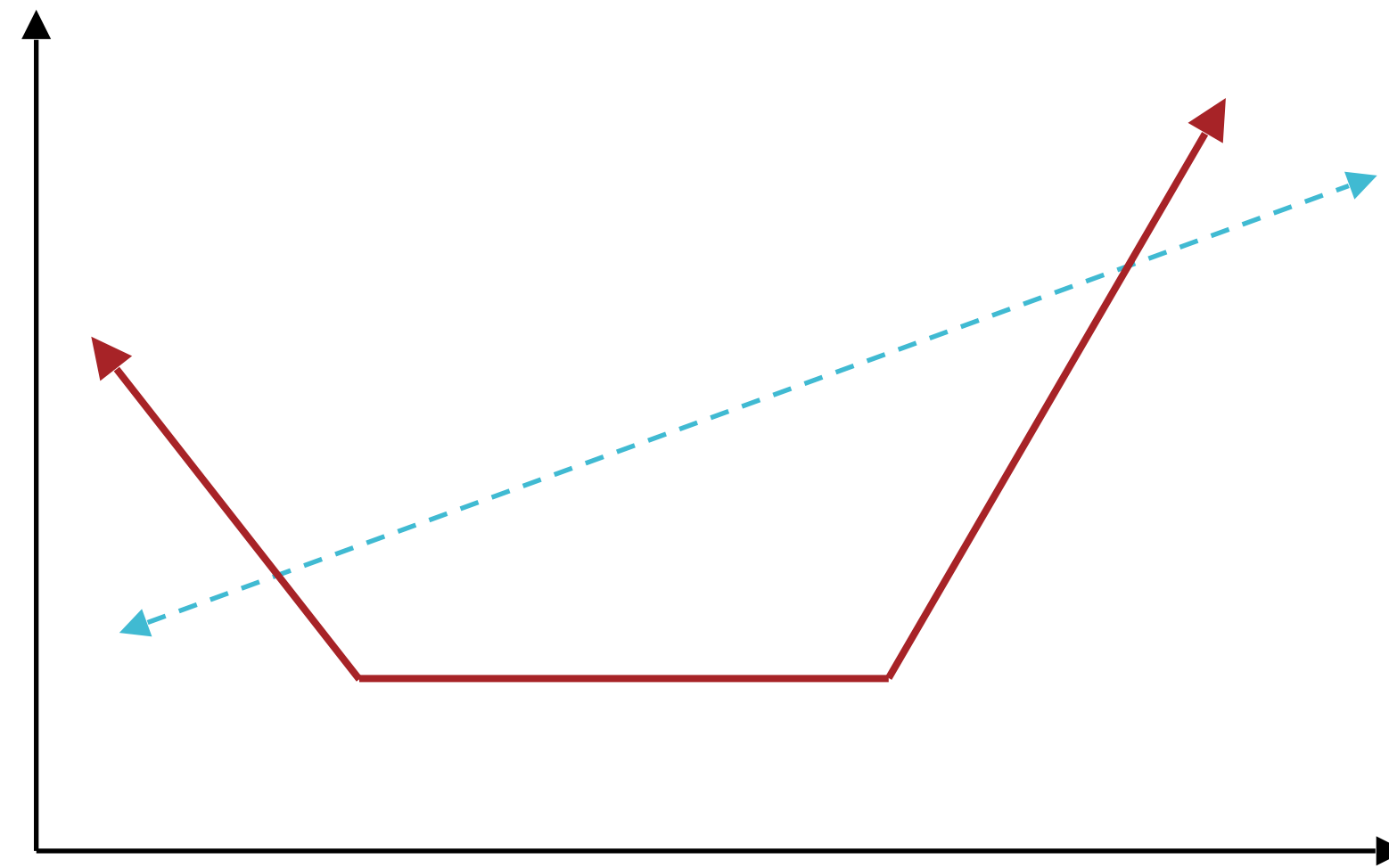
- Similar idea for sets: convex if any line segment between two points in the set stays in the set

Convexity and contour plots

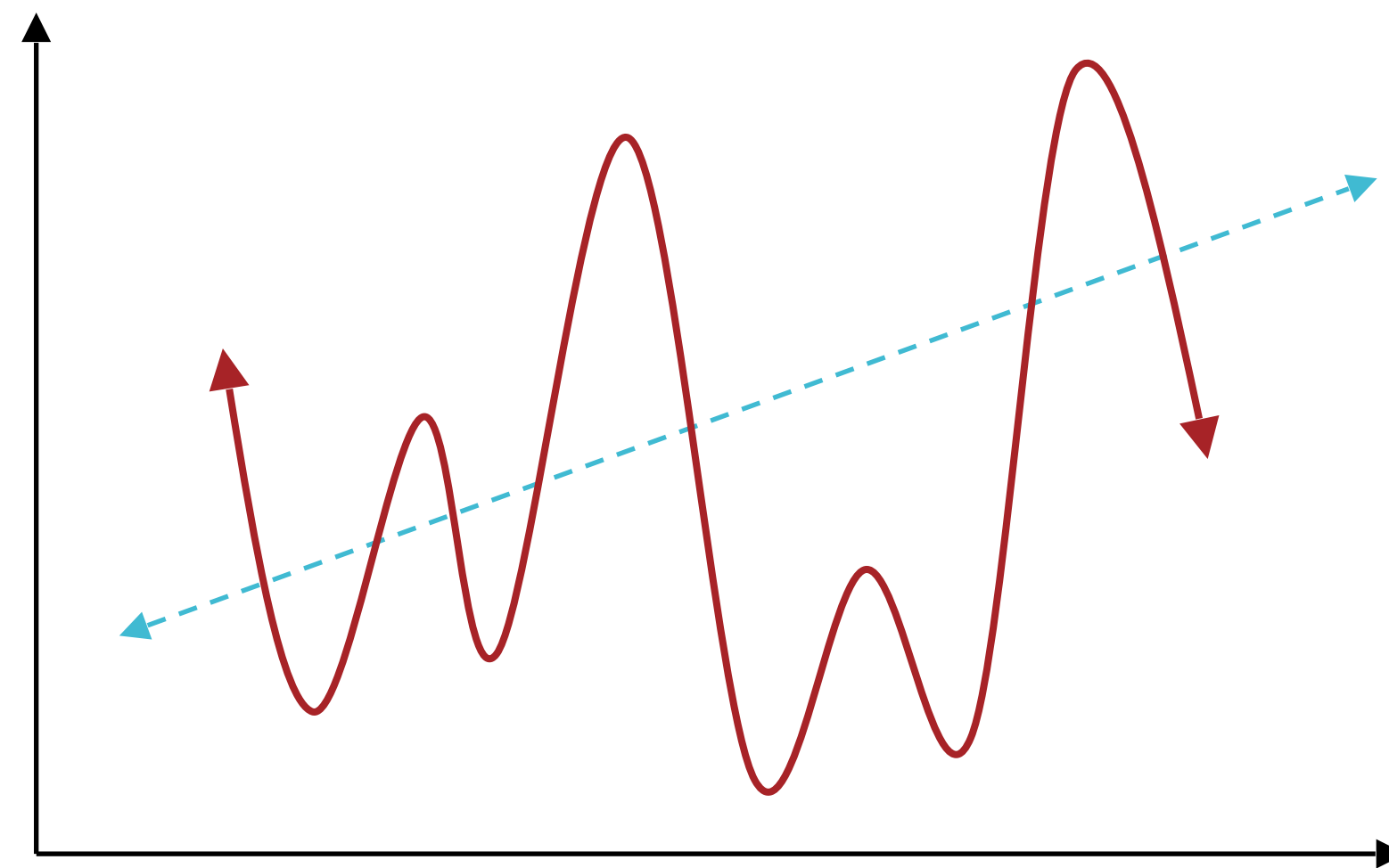


- If f convex, its sublevel sets $(f(x) \leq k)$ are convex
 - ▶ draw line segment between any two points on a contour
 - ▶ by function convexity, function stays below endpoints (all points on segment are inside sublevel set)

Convexity: local and global minima

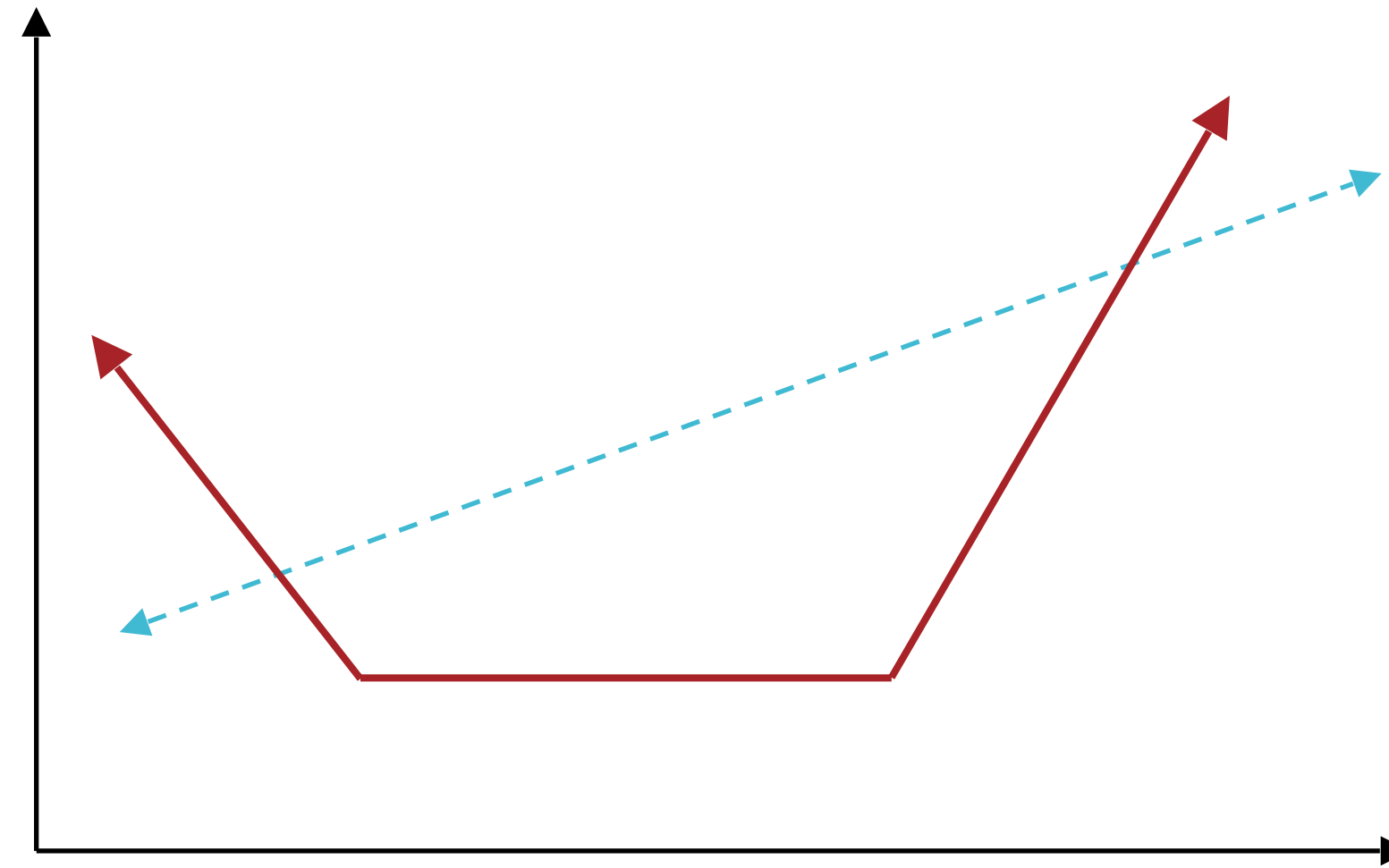


Convex functions



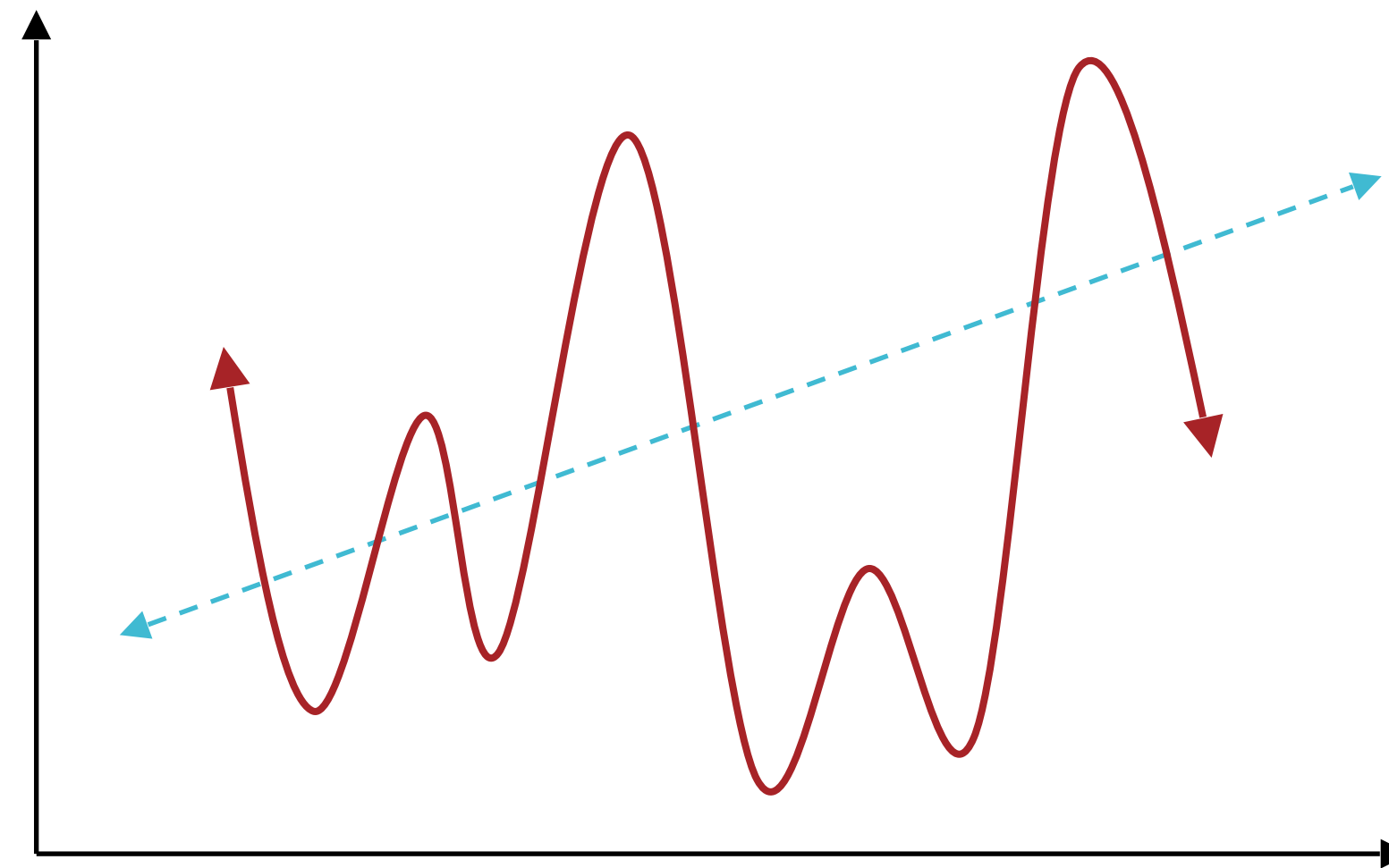
Non-convex functions

Convexity: local and global minima

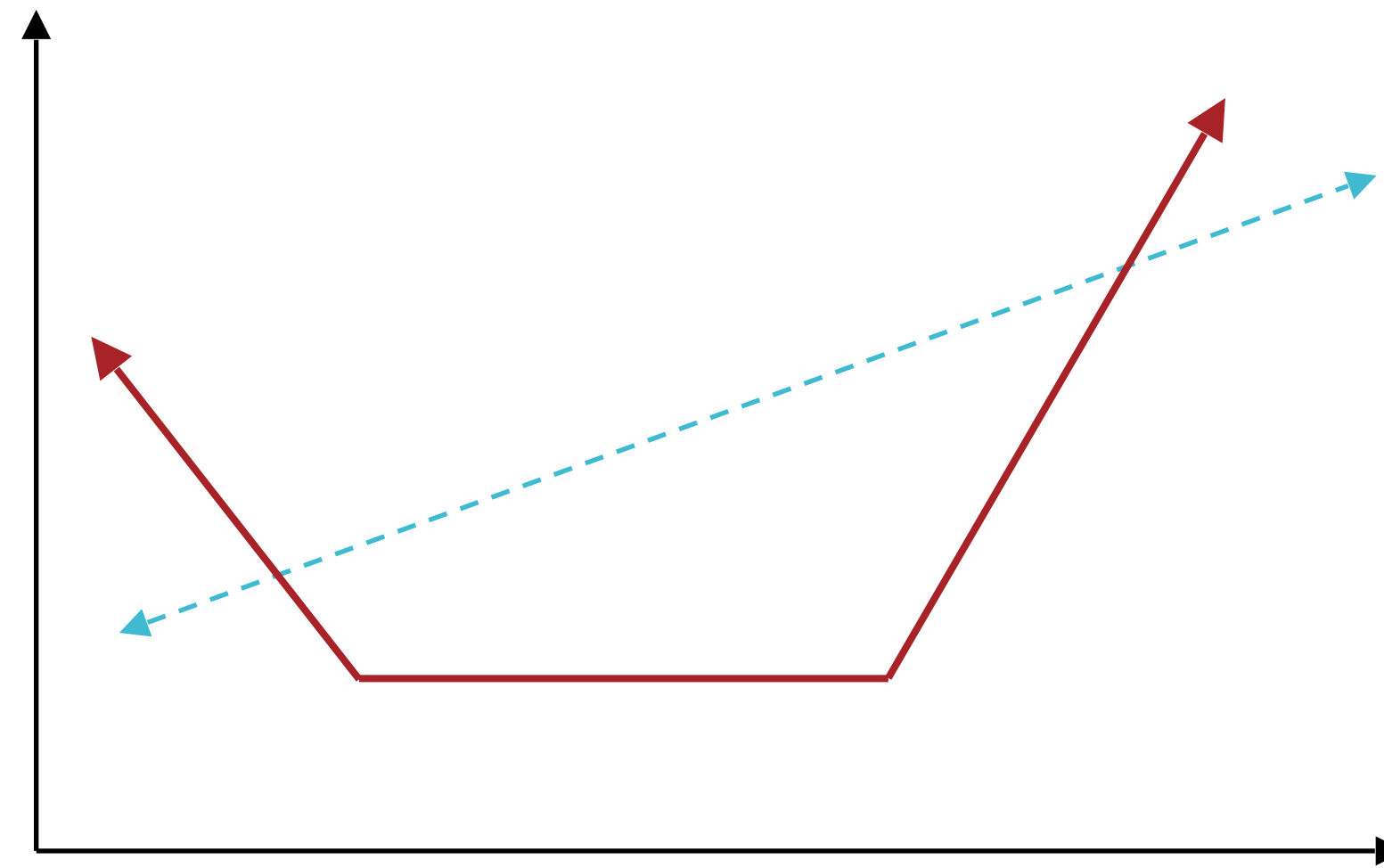


Given a function $f: \mathbb{R}^D \rightarrow \mathbb{R}$

- $\mathbf{x}^*$ is a global minimum iff
$$f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \forall \mathbf{x} \in \mathbb{R}^D$$

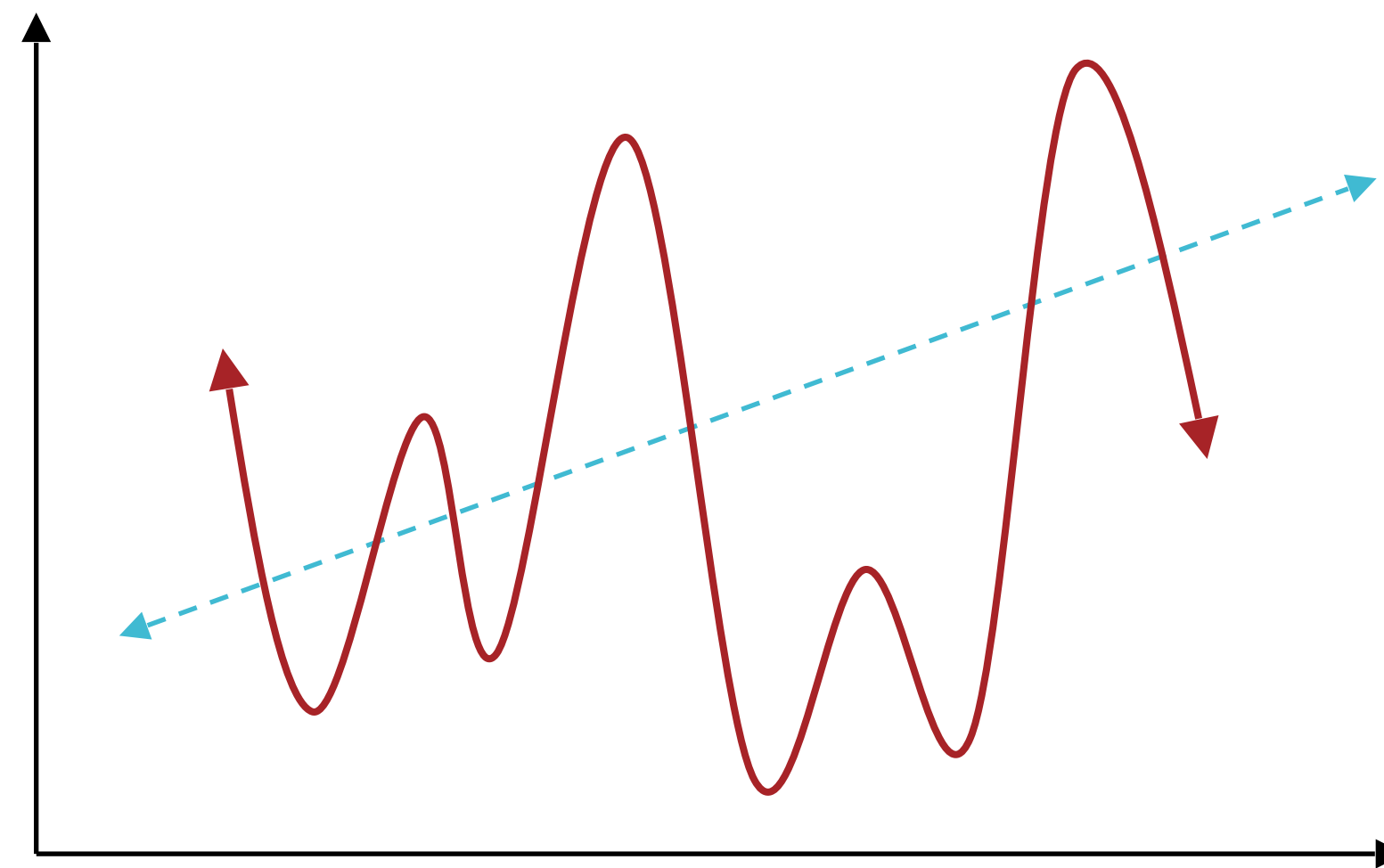


Convexity: local and global minima



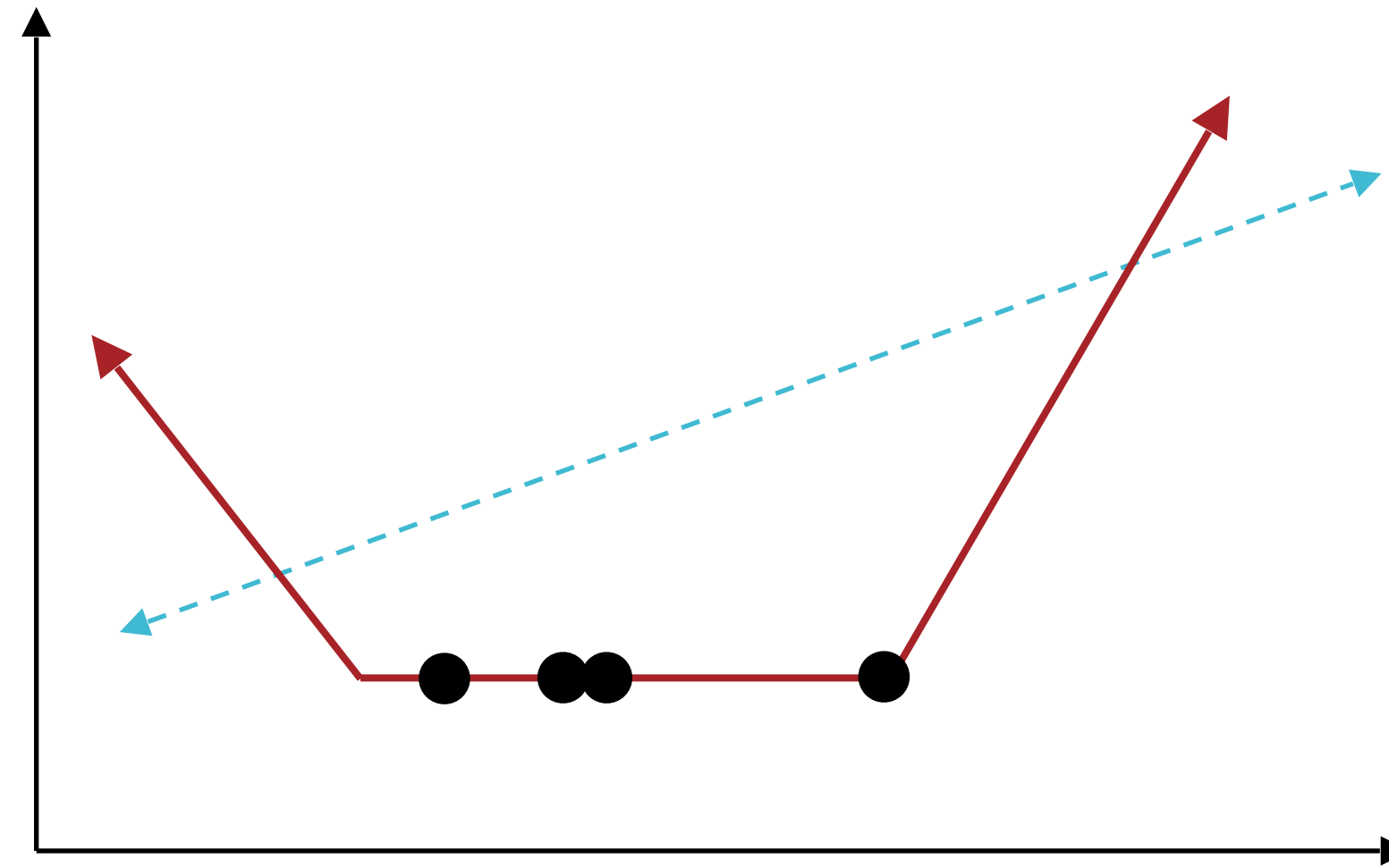
Given a function $f: \mathbb{R}^D \rightarrow \mathbb{R}$

- $\mathbf{x}^*$ is a global minimum iff
$$f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \forall \mathbf{x} \in \mathbb{R}^D$$

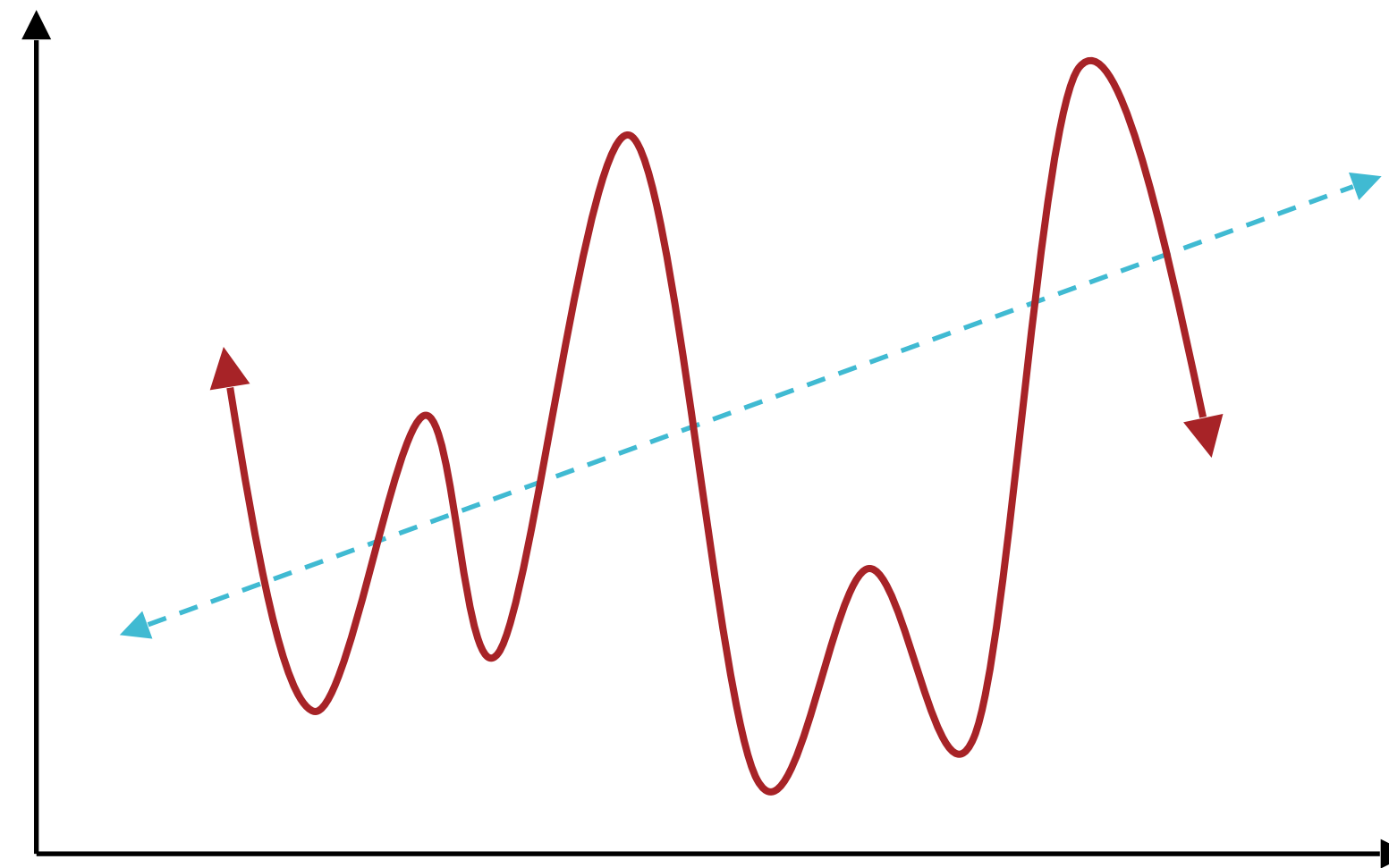


- $\mathbf{x}^*$ is a local minimum iff
$$\exists \epsilon \text{ s.t. } f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \forall \mathbf{x} \text{ s.t. } \|\mathbf{x} - \mathbf{x}^*\|_2 < \epsilon$$

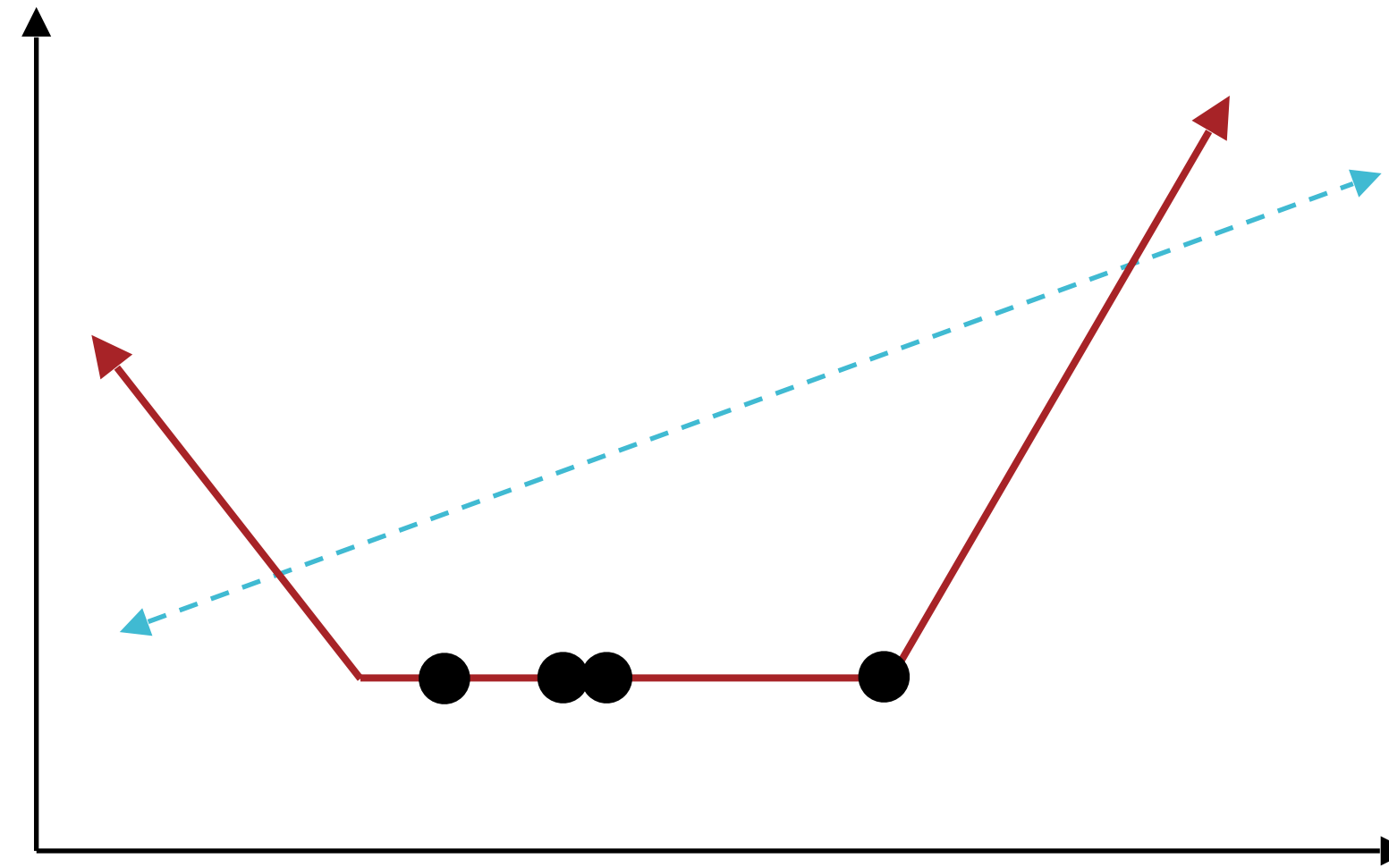
Convexity: local and global minima



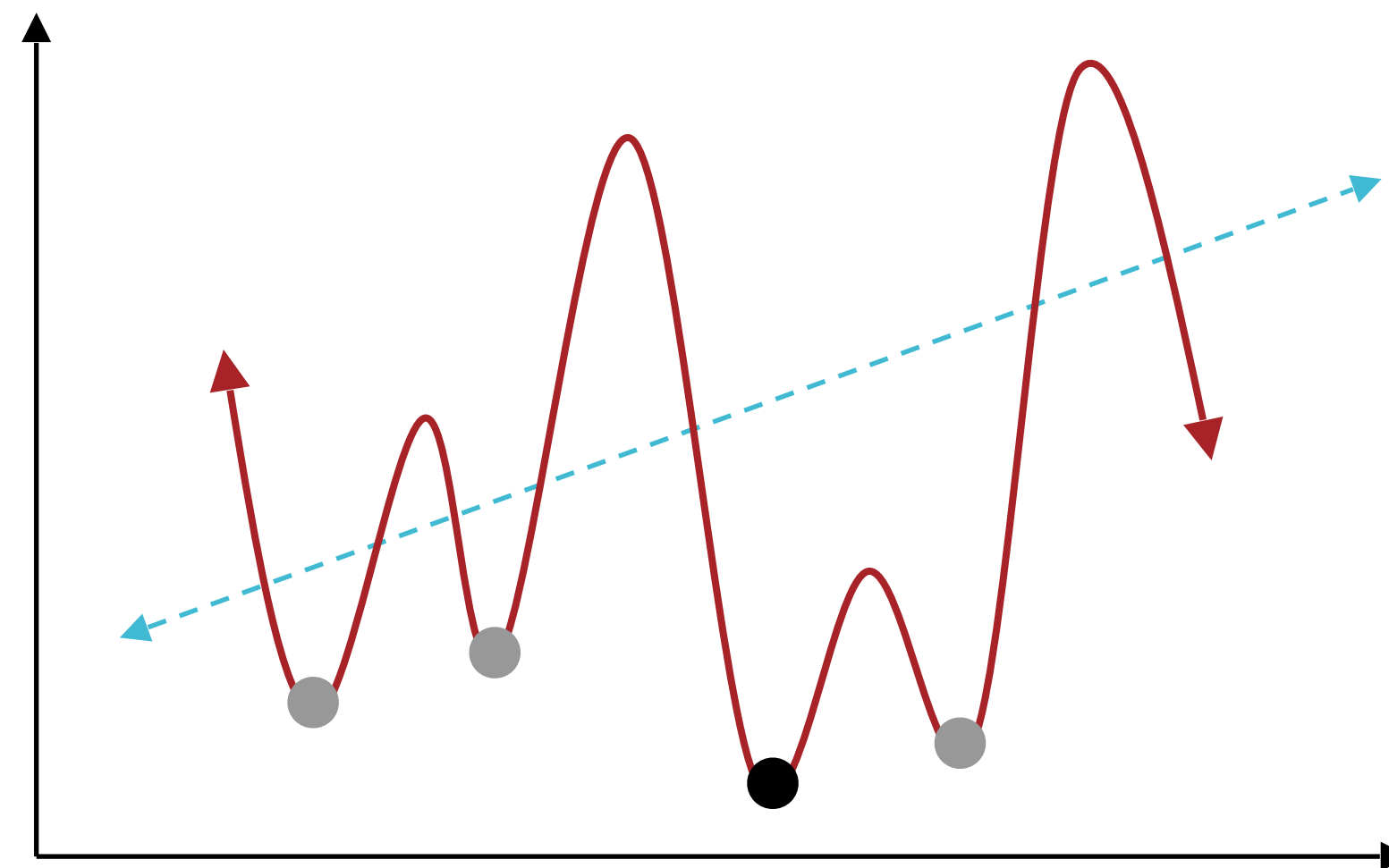
Convex functions:
Each local minimum is a
global minimum!



Convexity: local and global minima

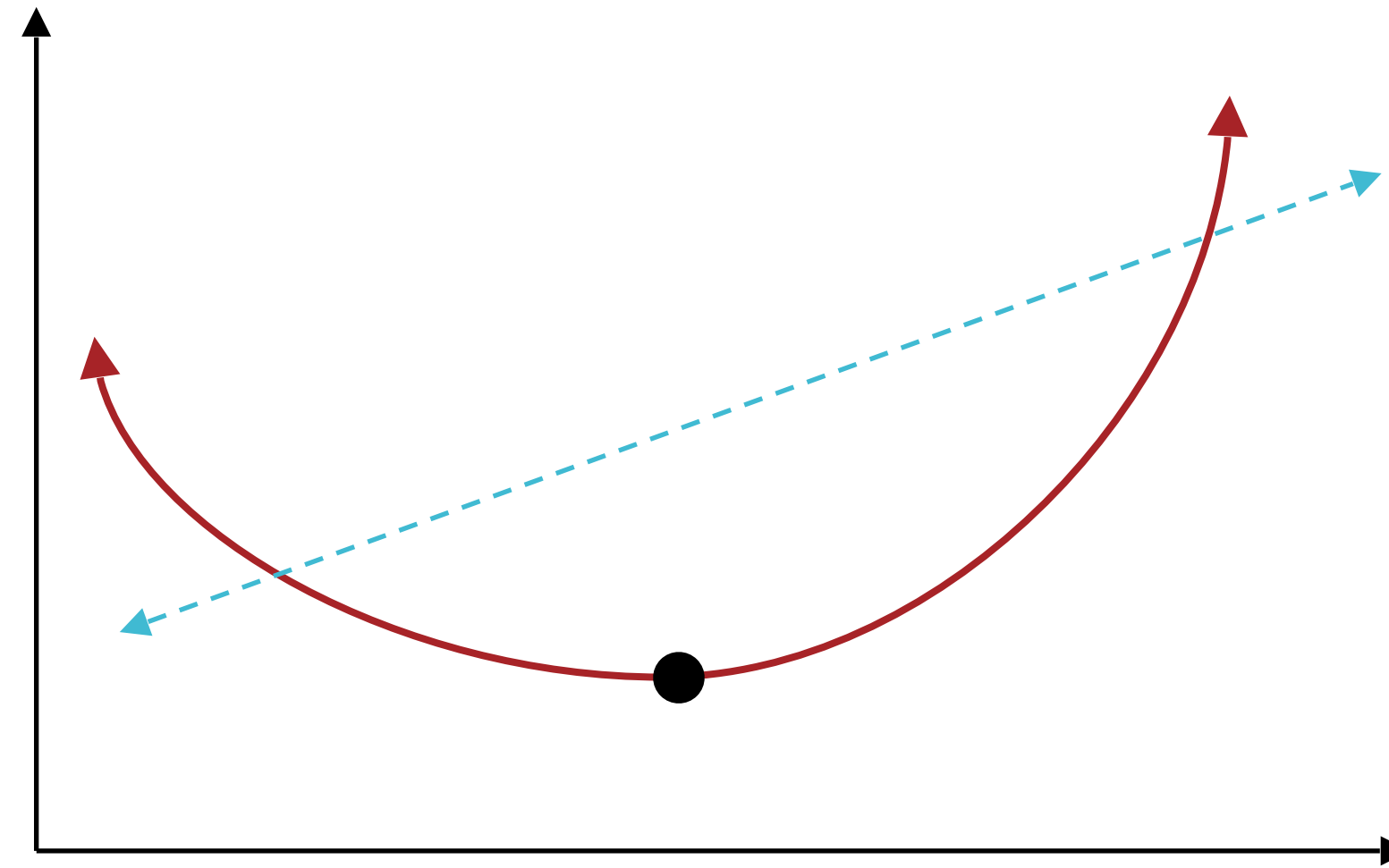


Convex functions:
Each local minimum is a
global minimum!

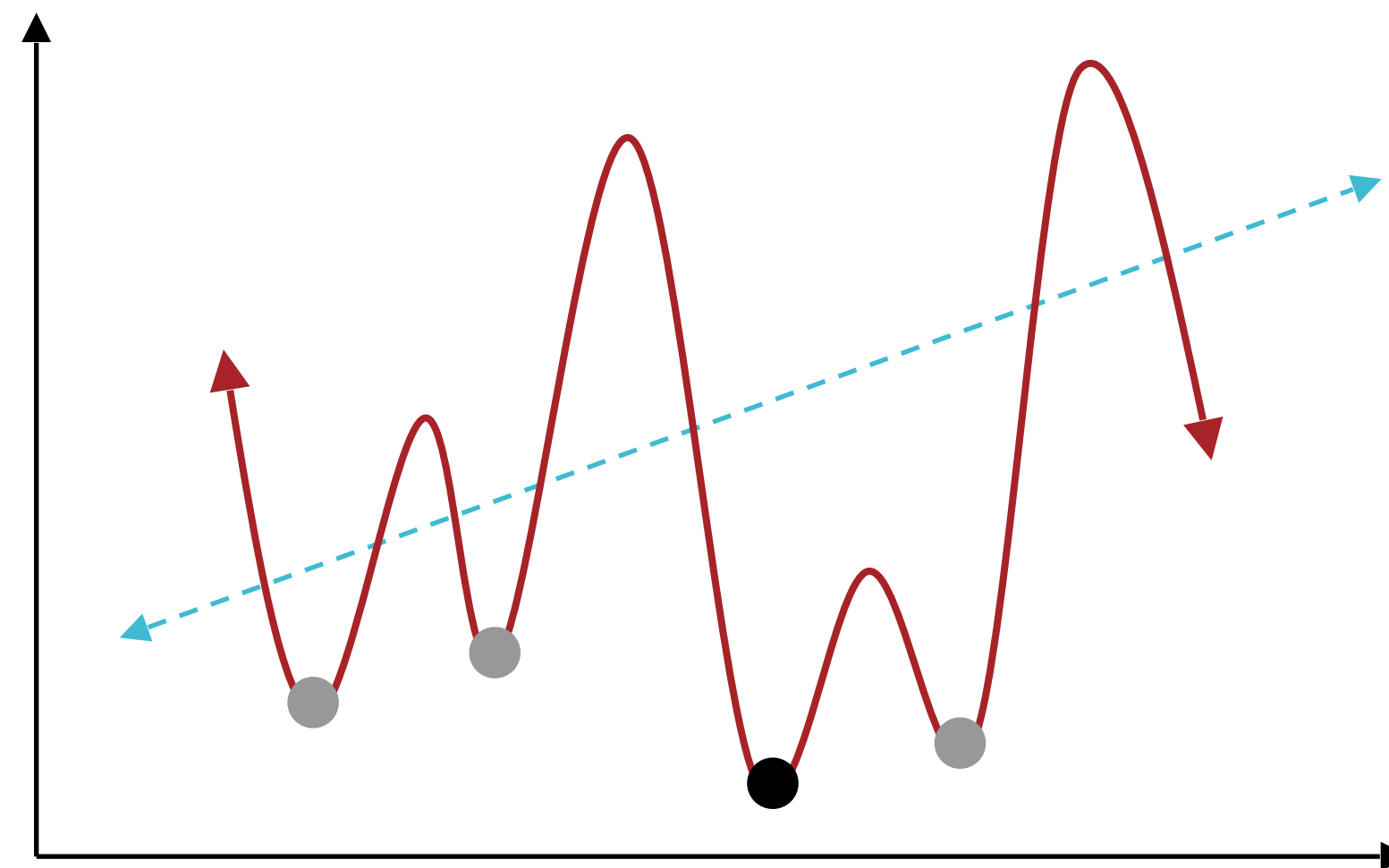


Non-convex functions:
A local minimum may or may
not be a global minimum...

Convexity: local and global minima



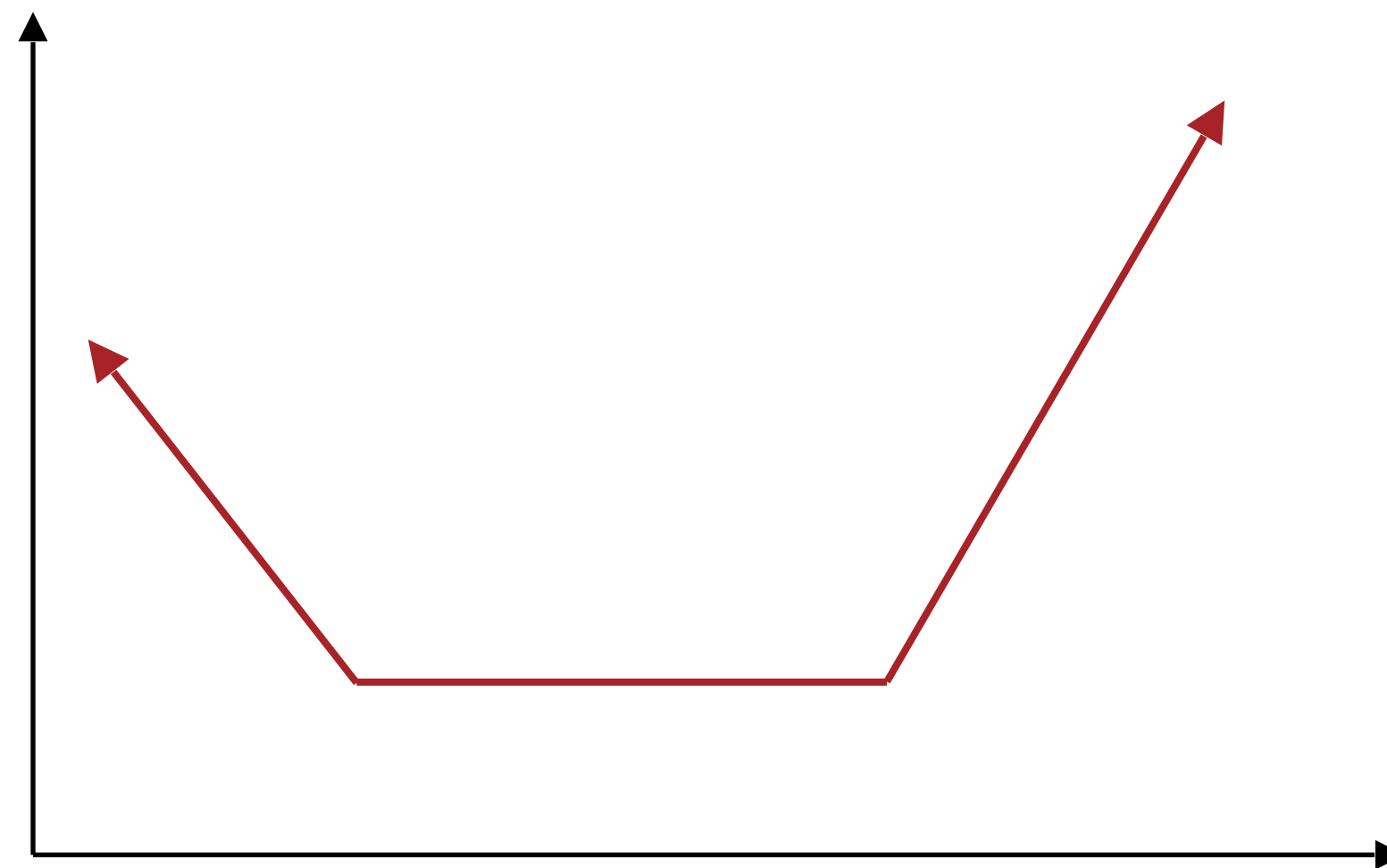
Strictly convex functions:
There exists a unique global
minimum!



Non-convex functions:
A local minimum may or may
not be a global minimum...

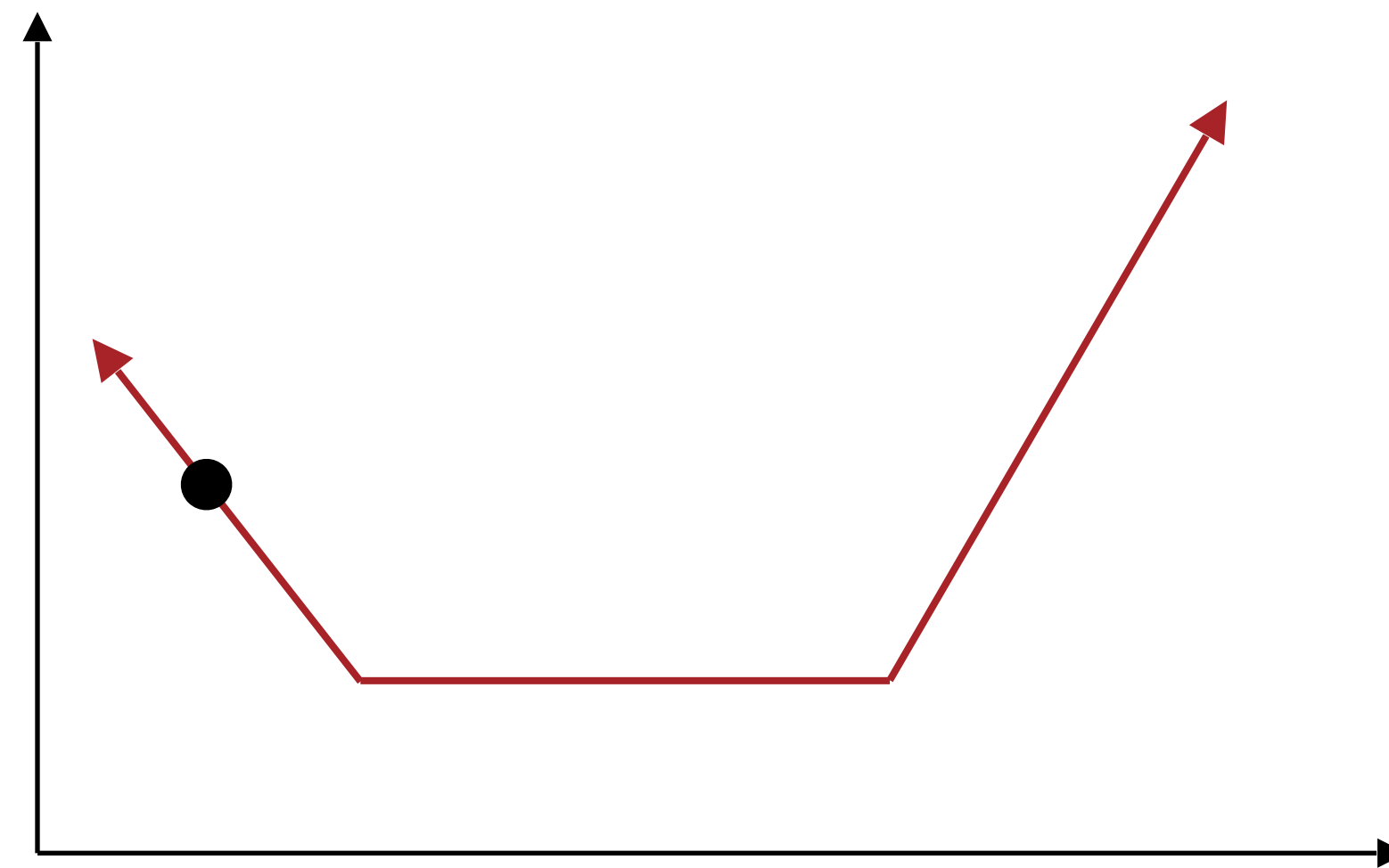
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Works great if the objective function is convex!



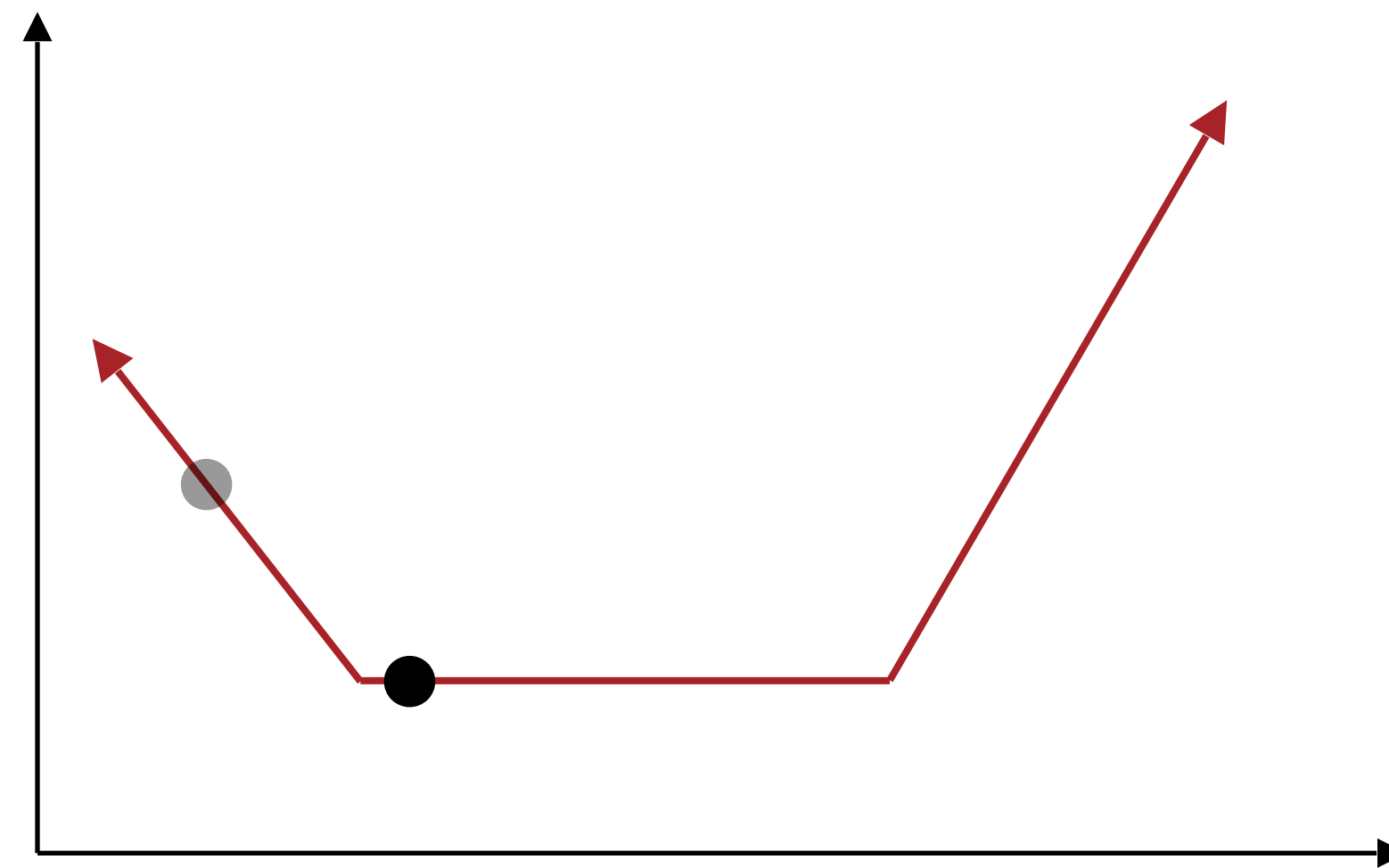
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Works great if the objective function is convex!



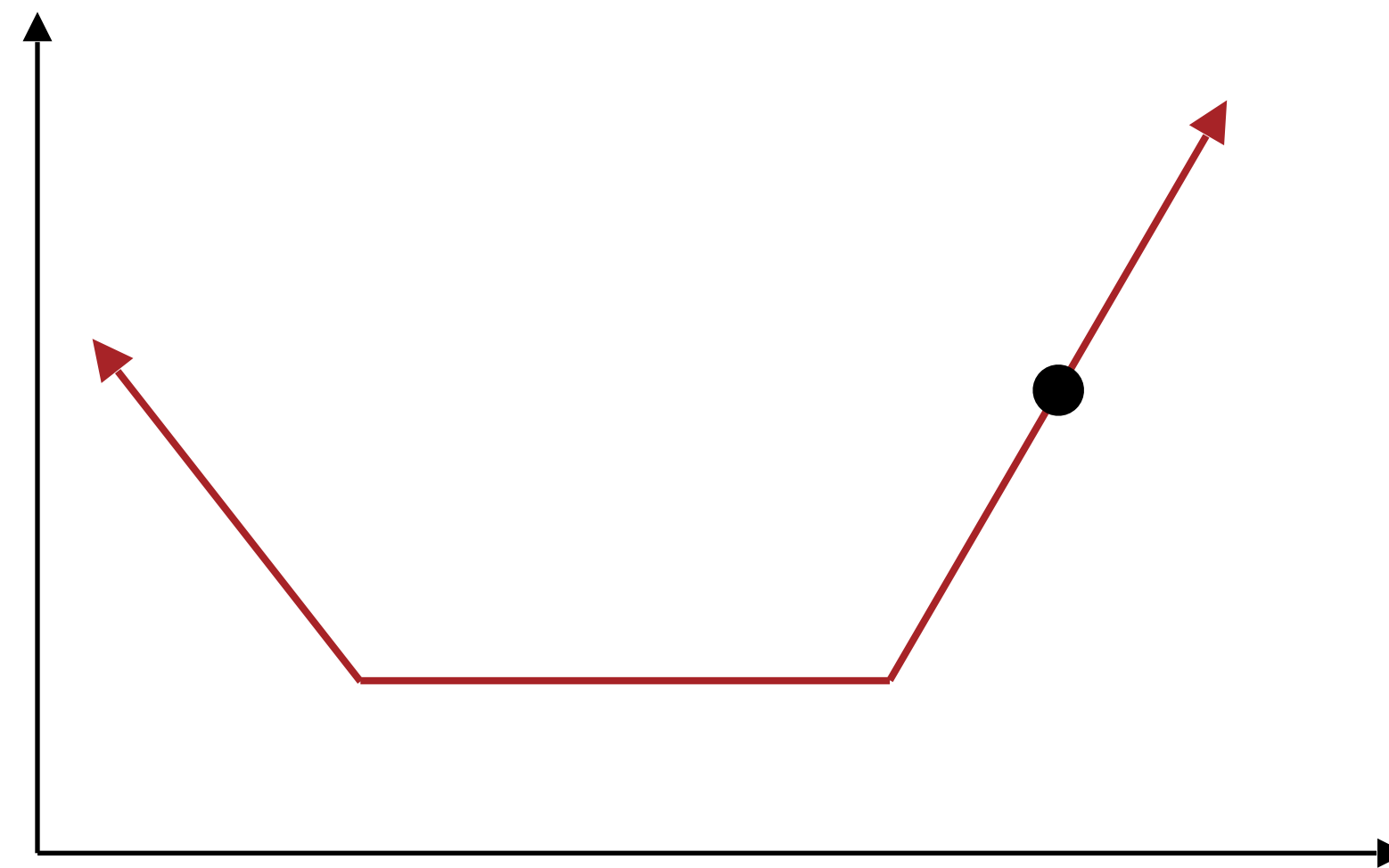
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Works great if the objective function is convex!



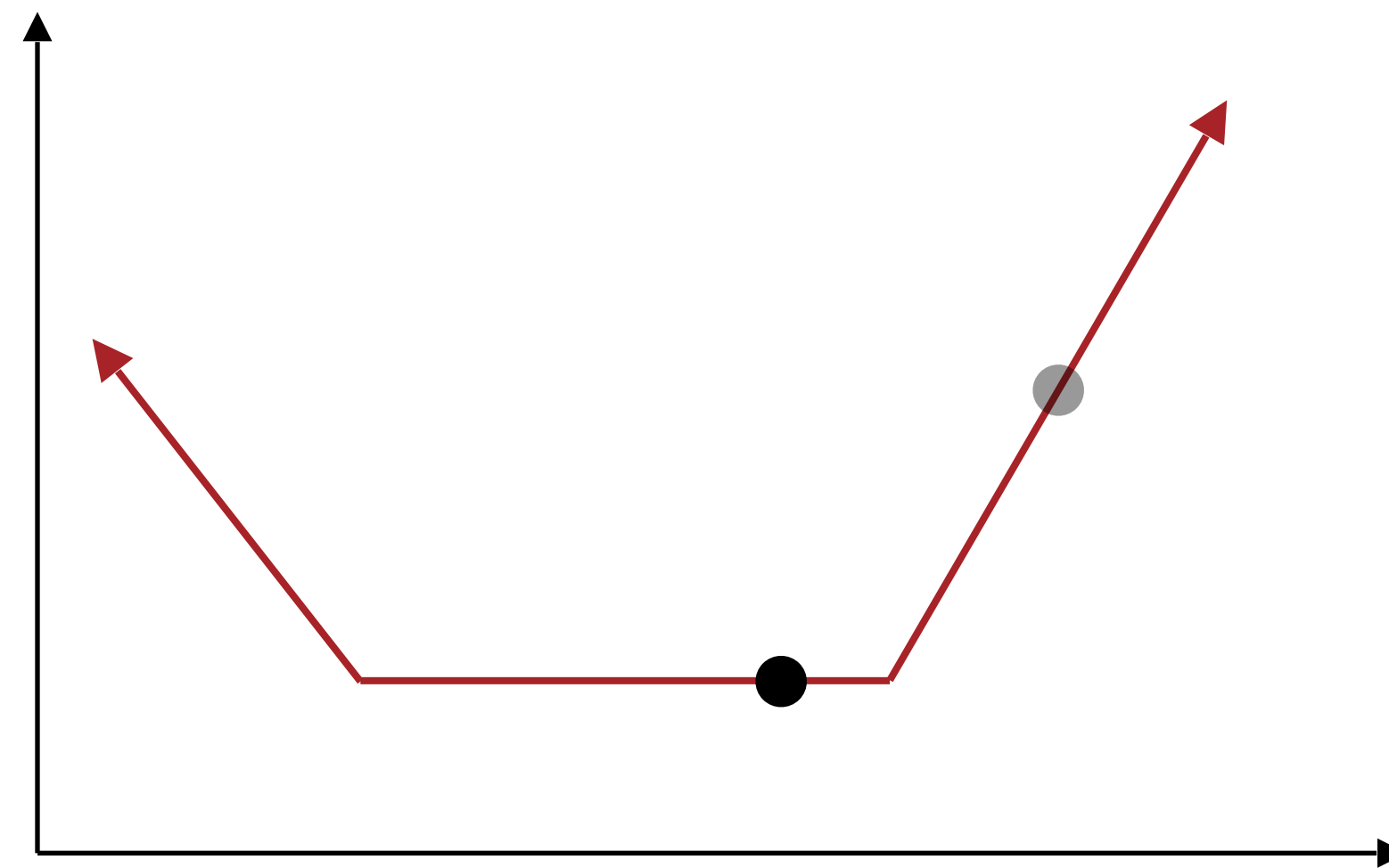
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Works great if the objective function is convex!



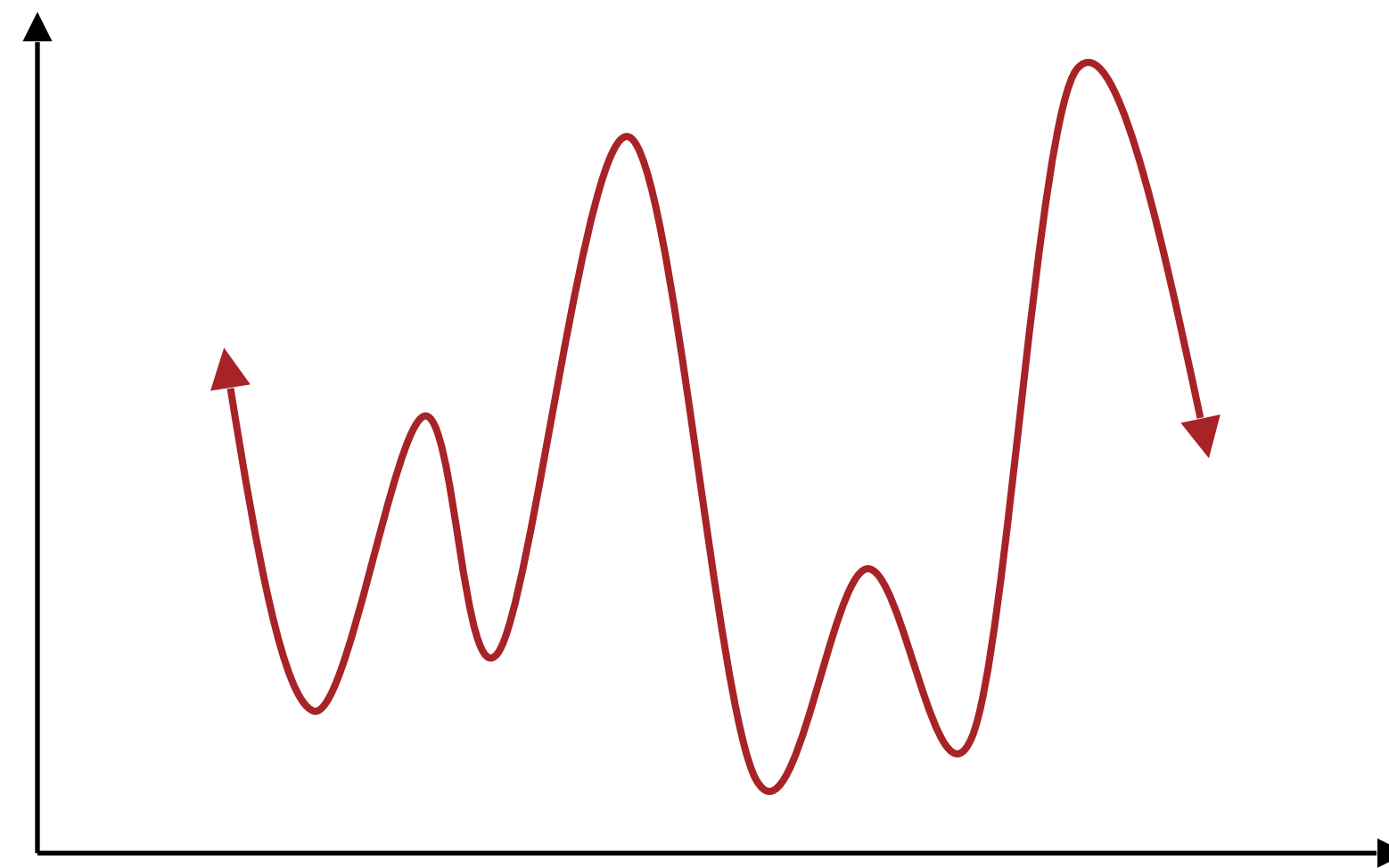
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Works great if the objective function is convex!



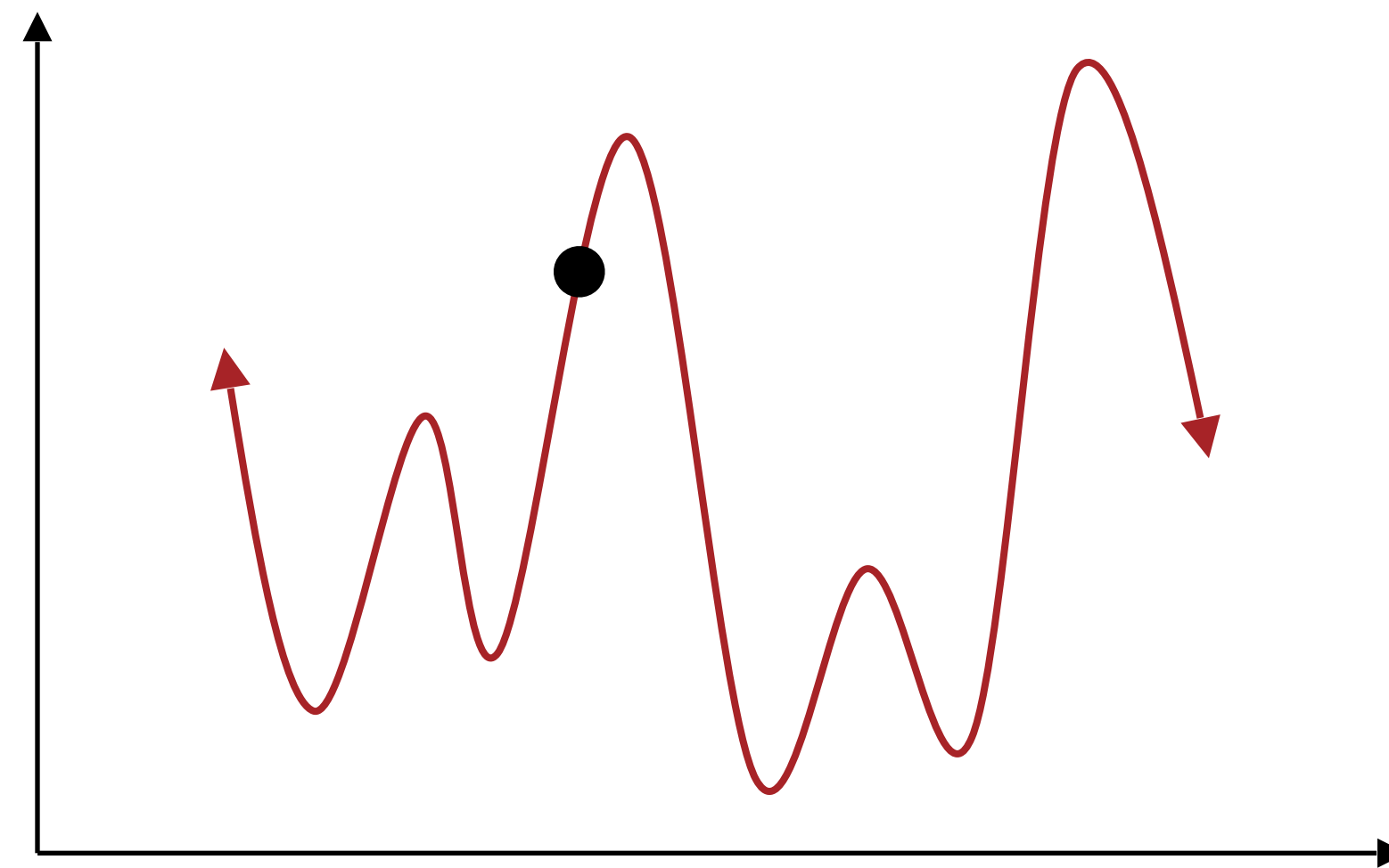
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Not ideal if the objective function is non-convex...



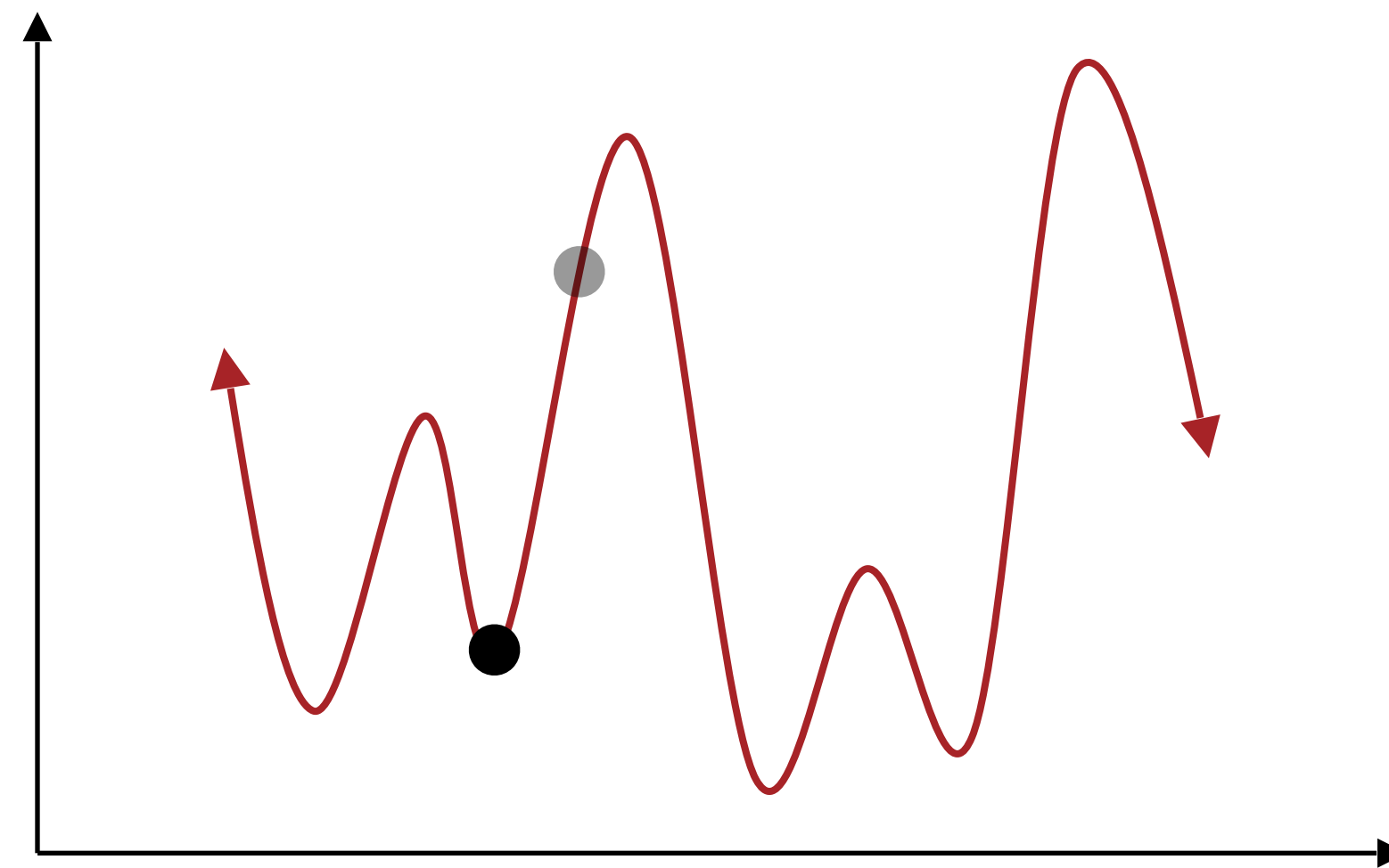
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Not ideal if the objective function is non-convex...



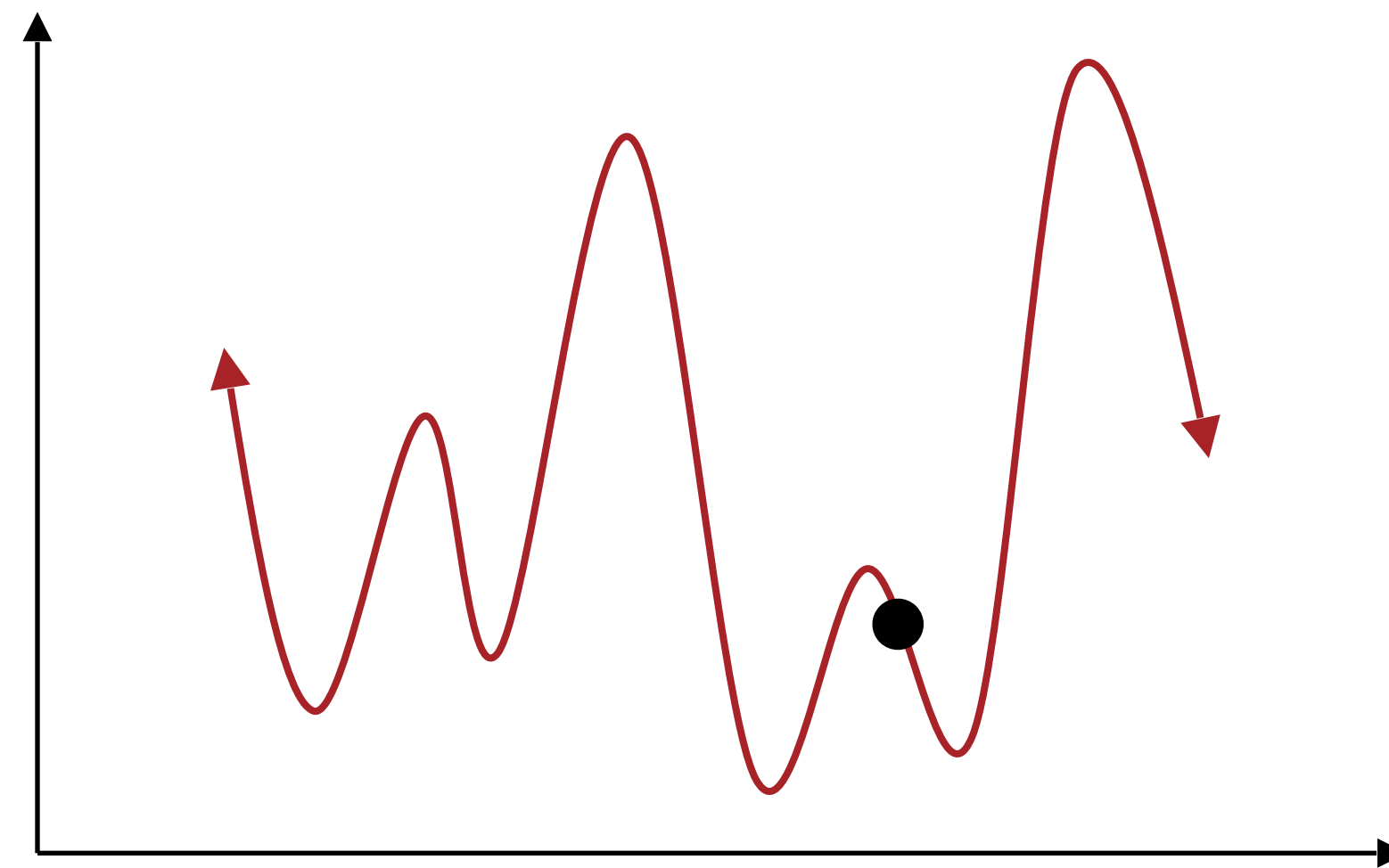
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Not ideal if the objective function is non-convex...



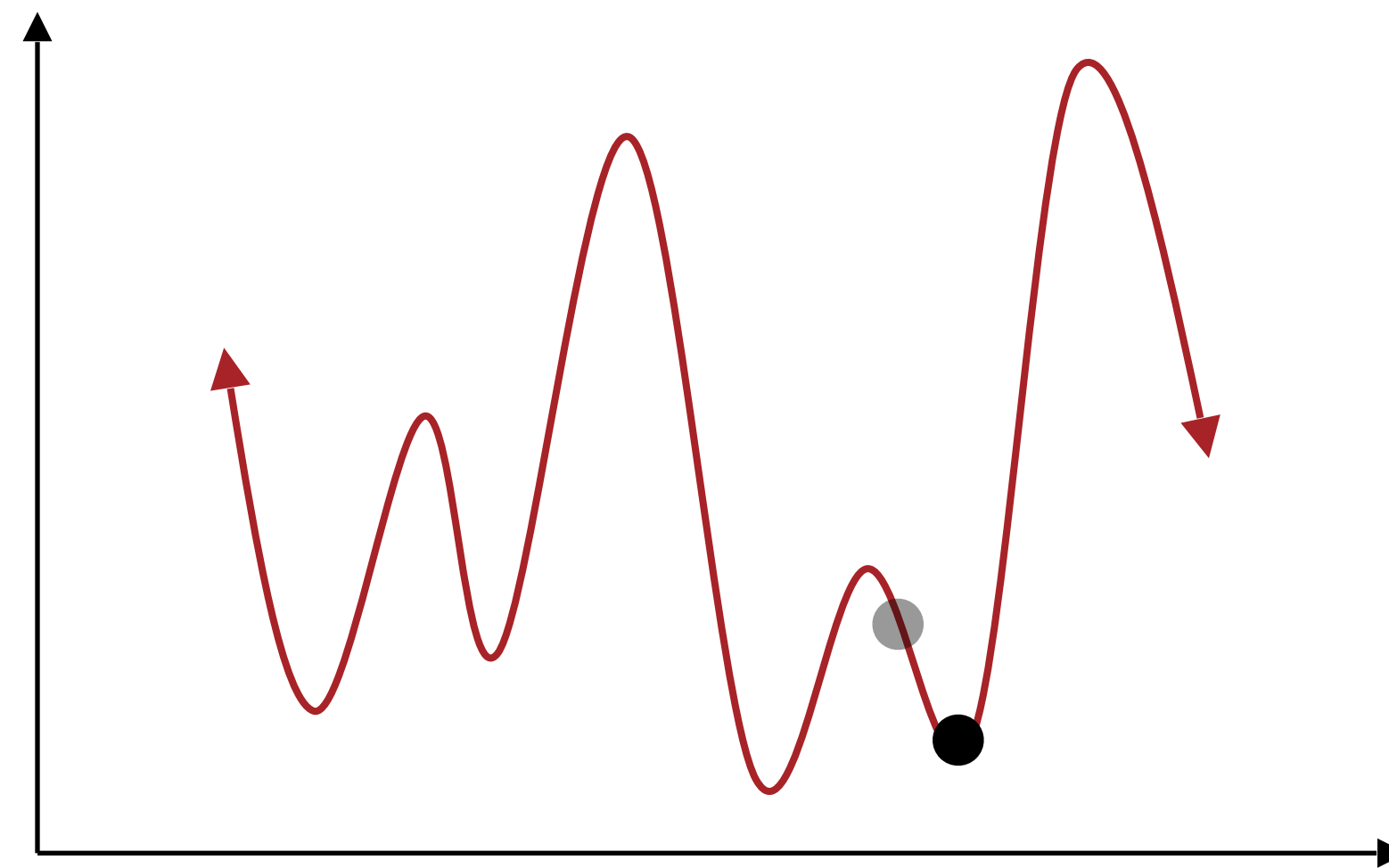
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Not ideal if the objective function is non-convex...



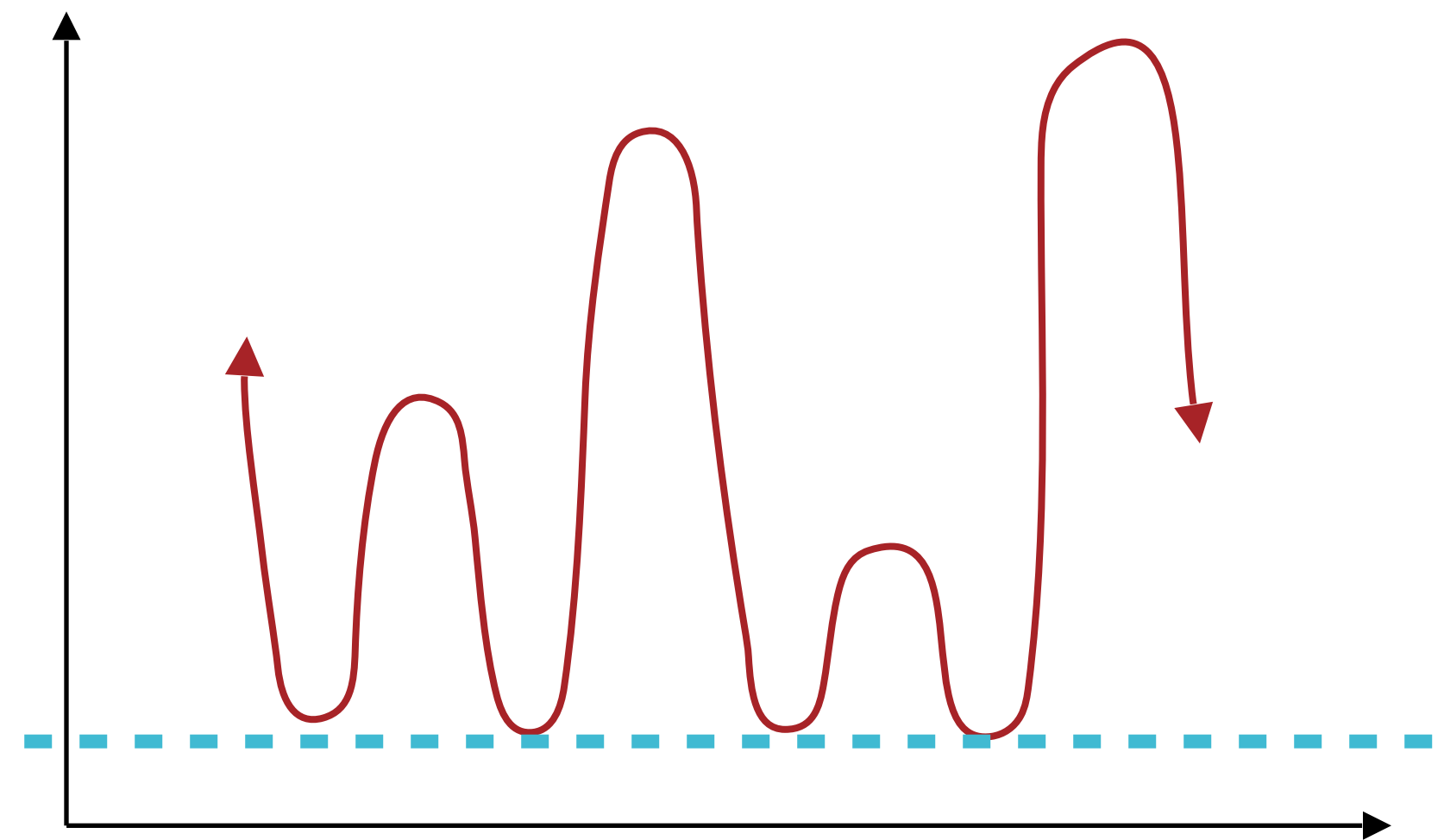
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Not ideal if the objective function is non-convex...



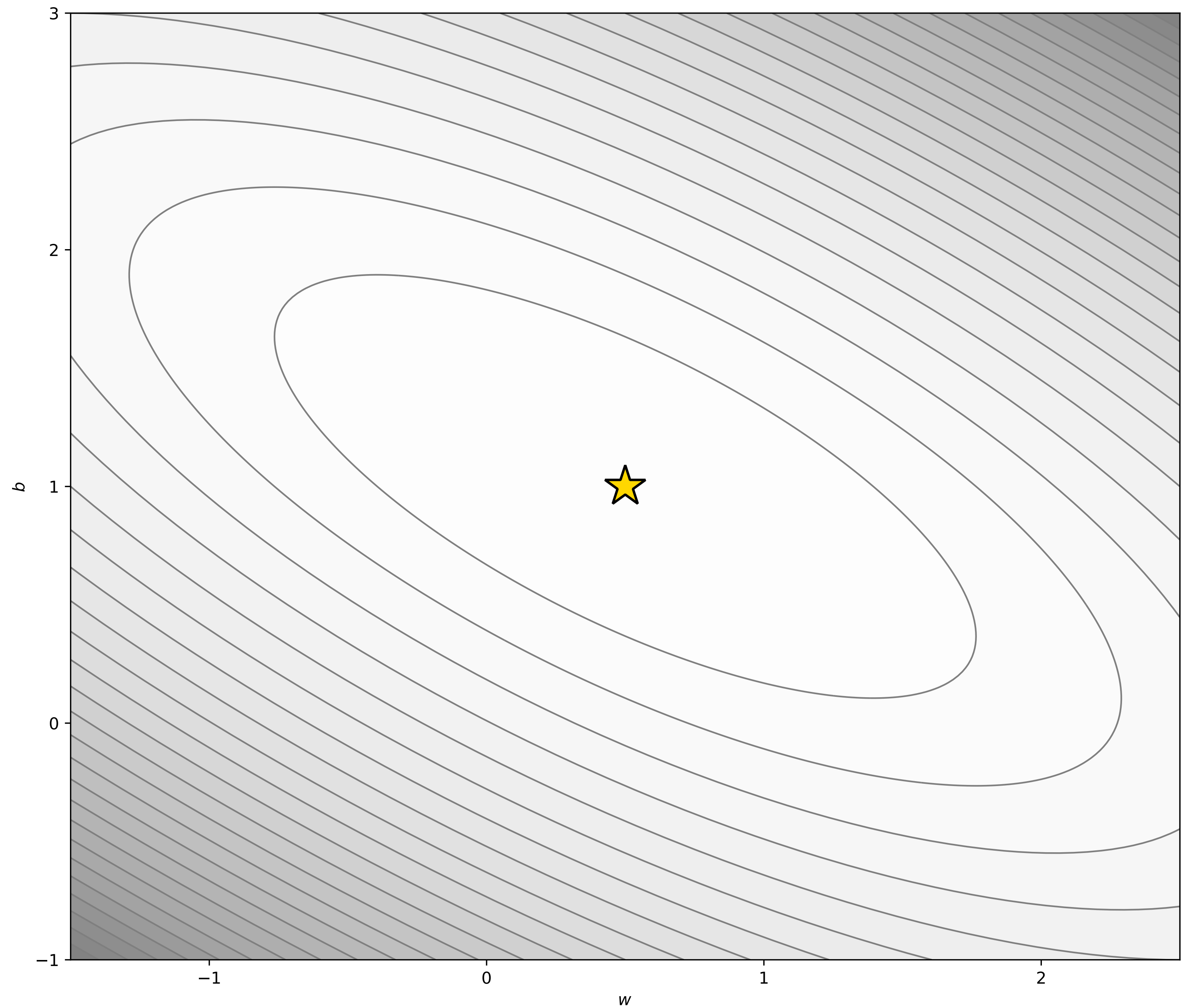
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ But can be OK if $\exists$ lots of pretty good local optima



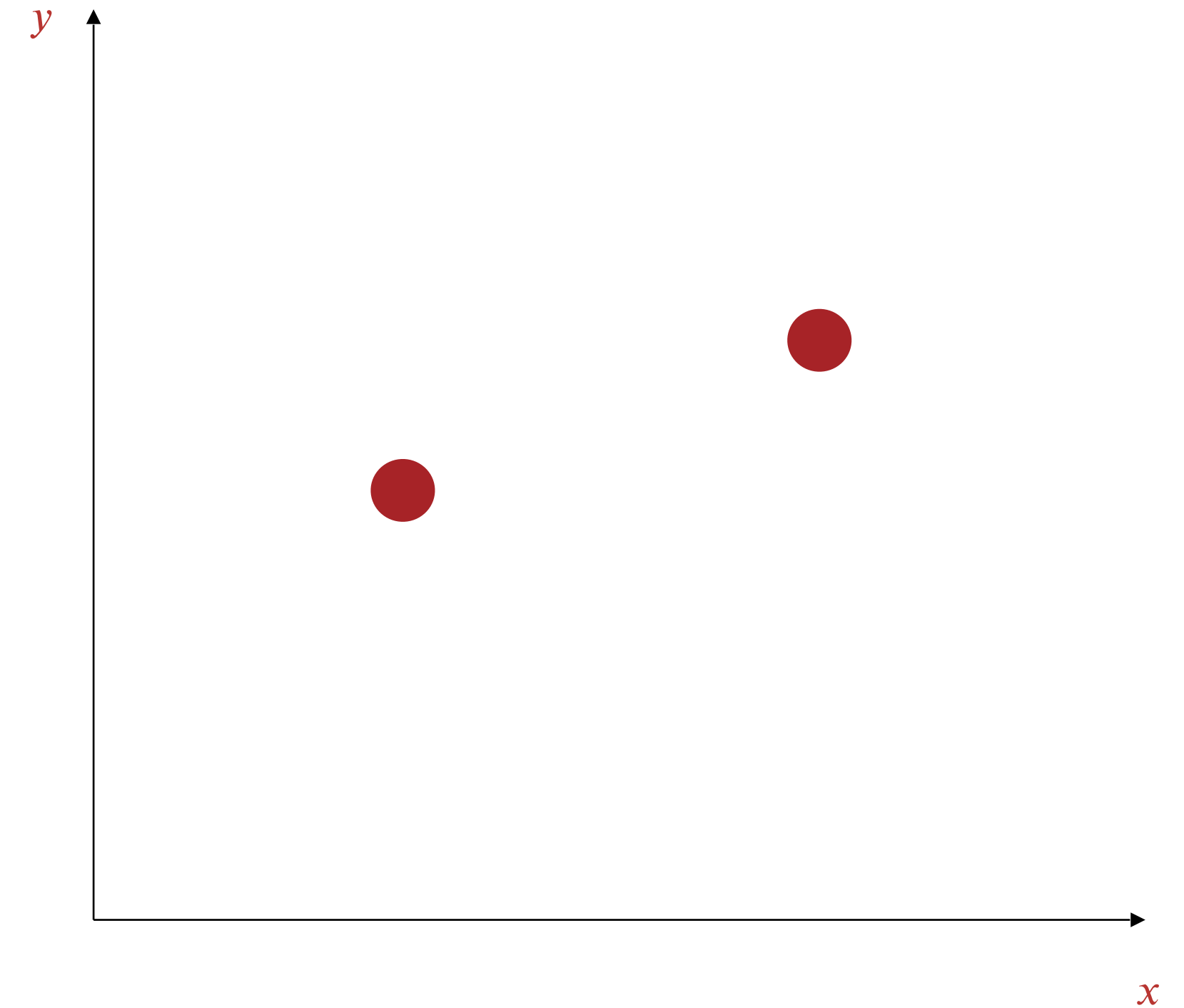
Gradient descent for linear regression: SSE is convex!

*In our simple problem,
strictly convex*



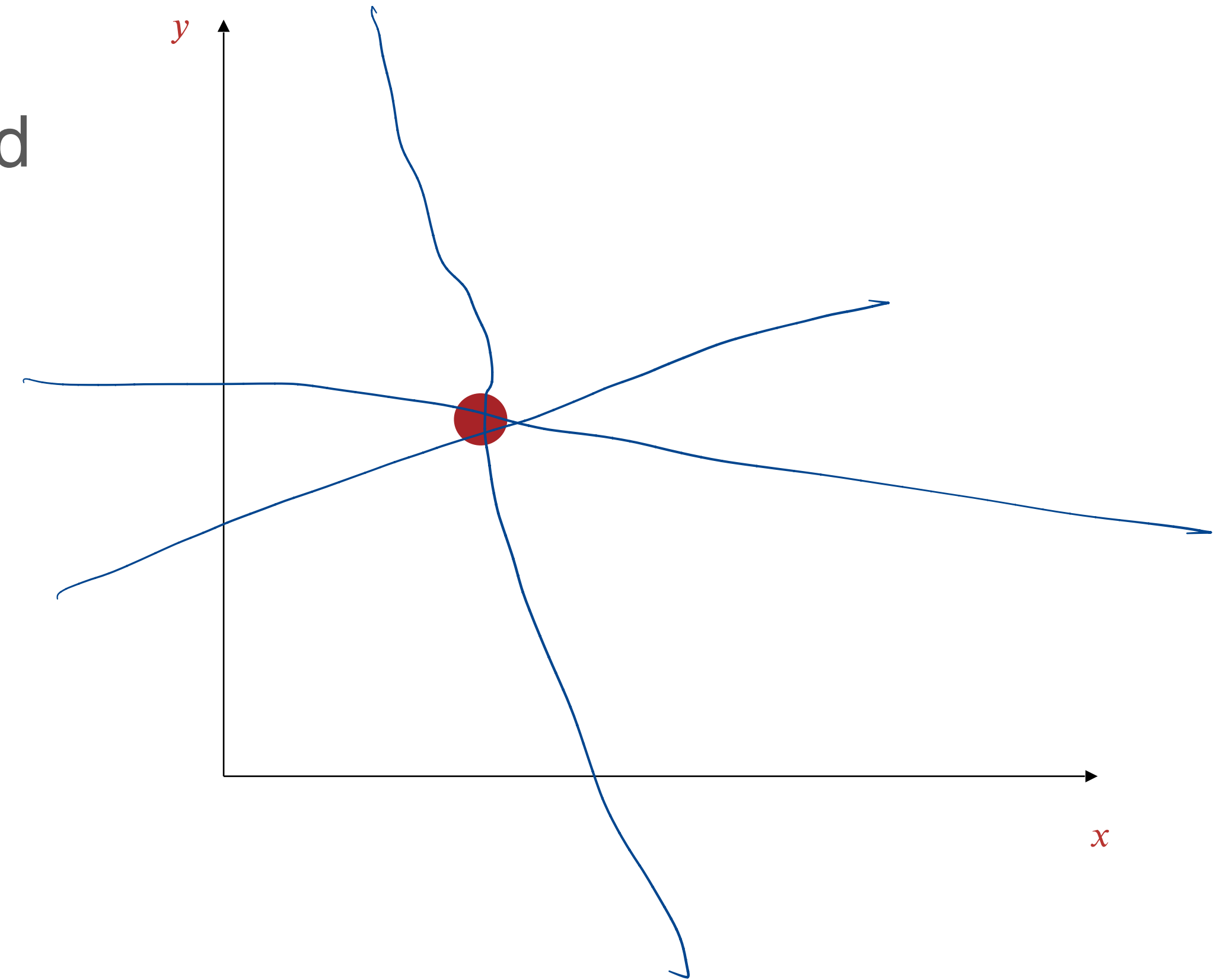
Linear Regression: Uniqueness

- Consider a 1D linear regression model trained to minimize the sum of squared errors: how many optimal solutions (i.e., sets of parameters w, b) are there for the given dataset?



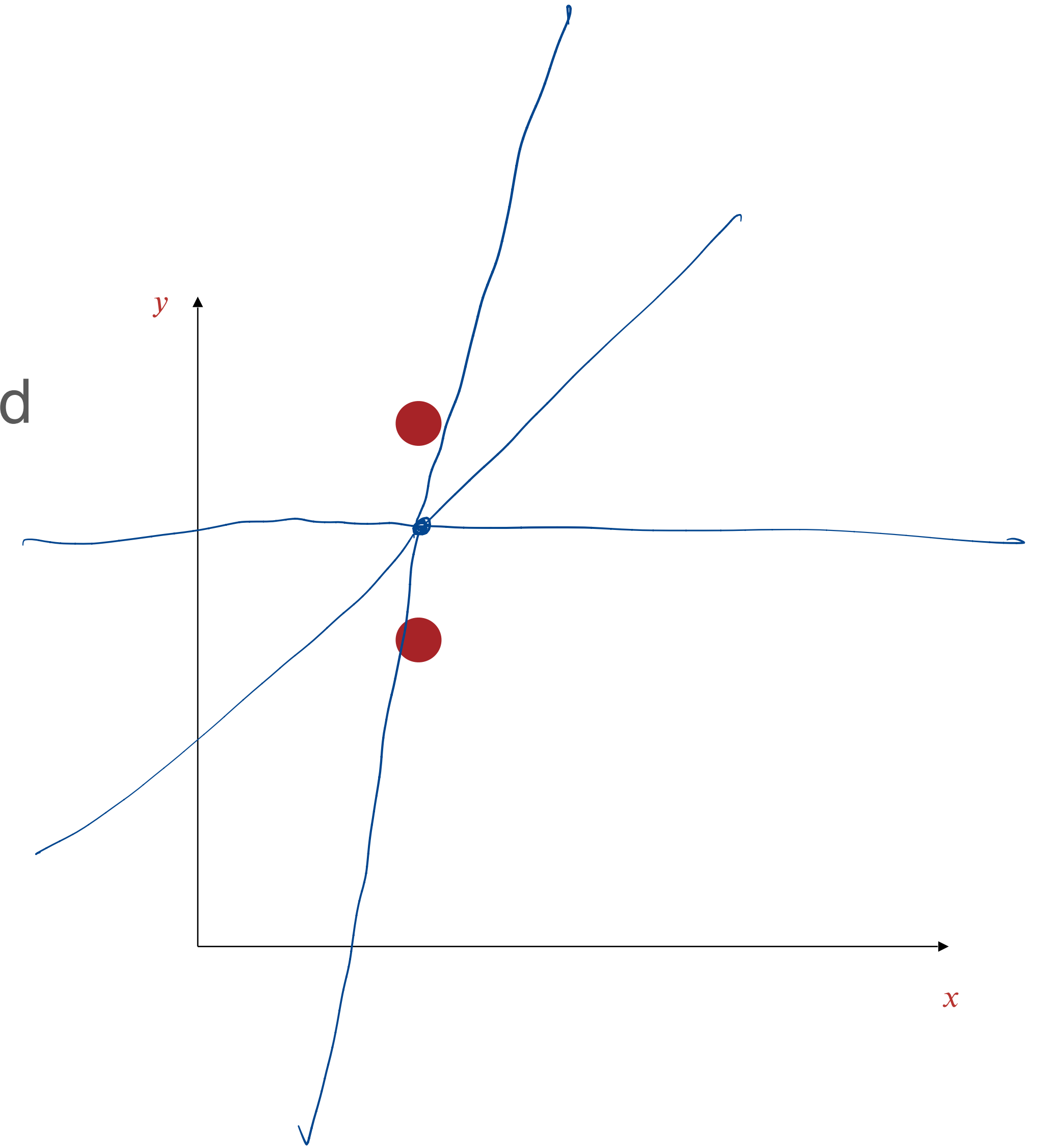
Linear Regression: Uniqueness

- Consider a 1D linear regression model trained to minimize the sum of squared errors: how many optimal solutions (i.e., sets of parameters w, b) are there for the given dataset?



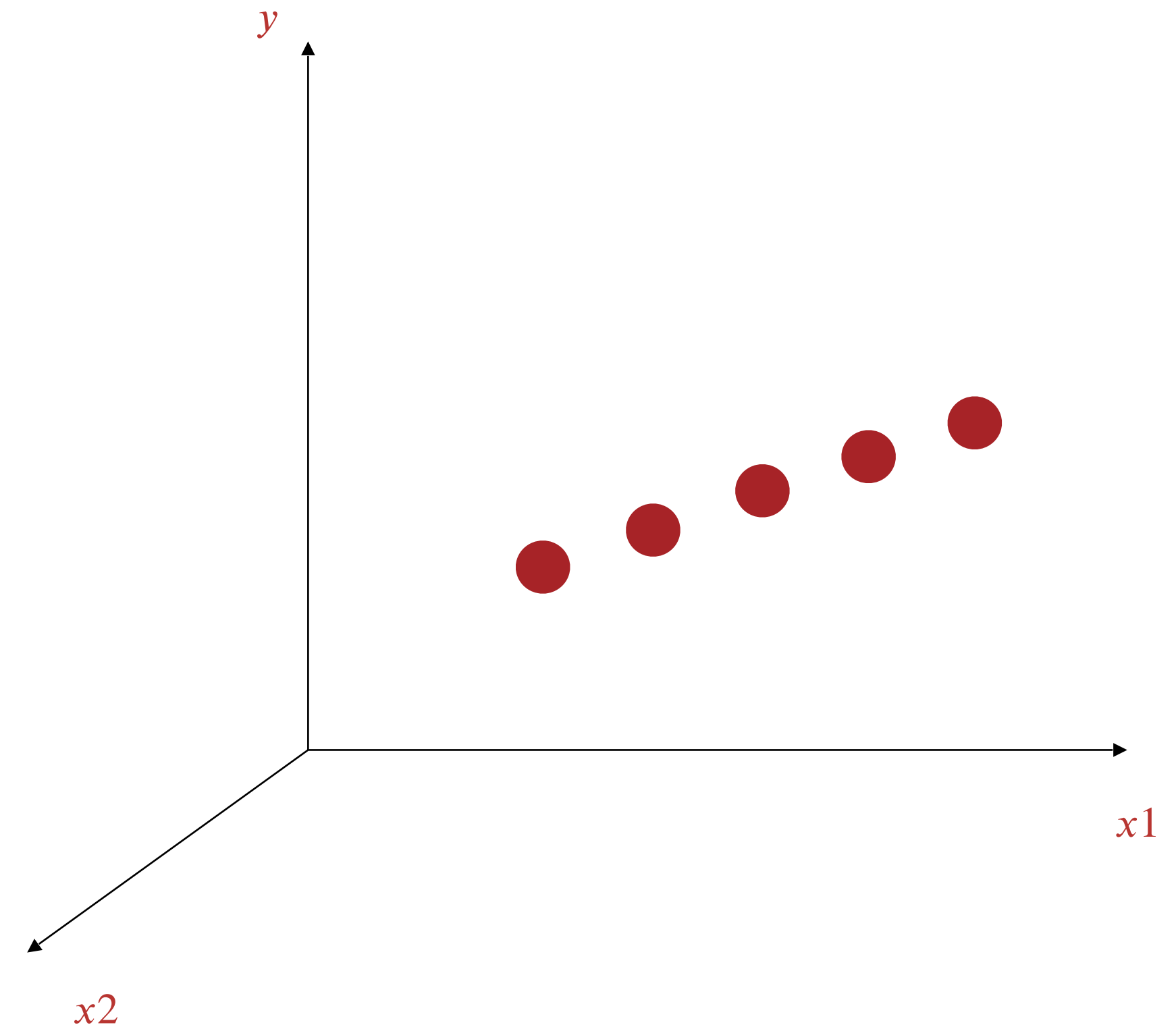
Linear Regression: Uniqueness

- Consider a 1D linear regression model trained to minimize the sum of squared errors: how many optimal solutions (i.e., sets of parameters w, b) are there for the given dataset?



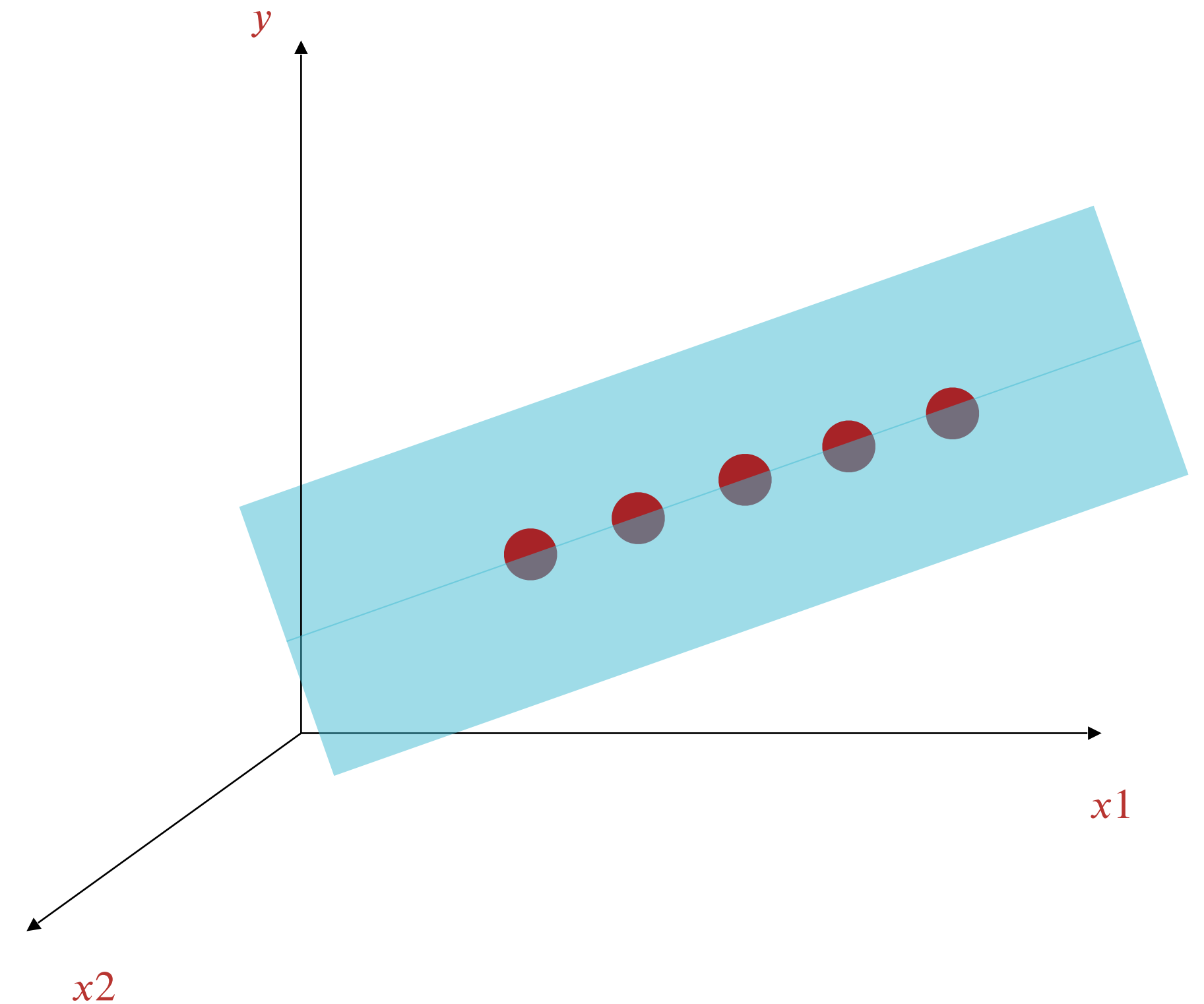
Linear Regression: Uniqueness

- Consider a 2D linear regression model trained to minimize the sum of squared errors: how many optimal solutions (i.e., sets of parameters w, b) are there for the given dataset?



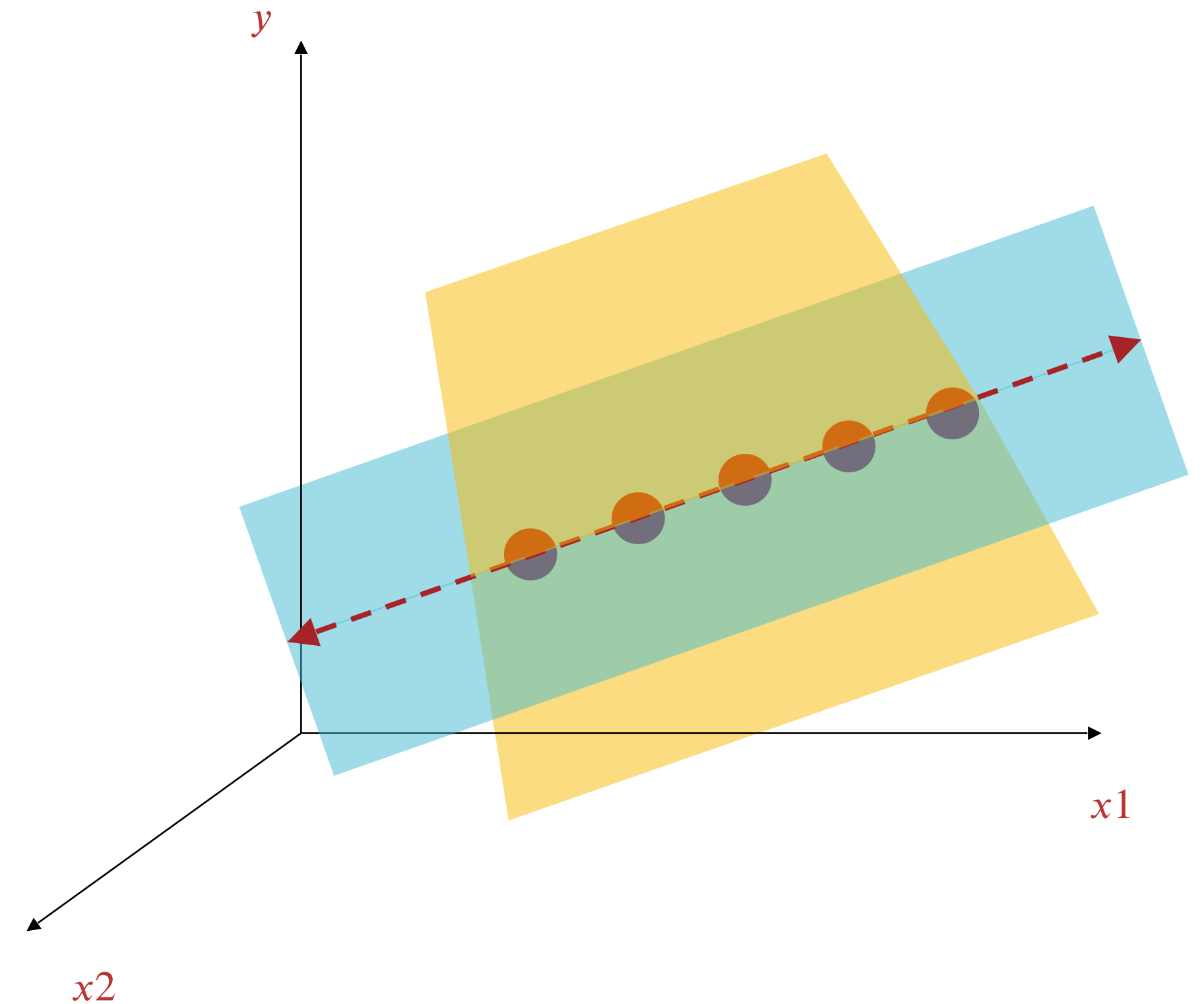
Linear Regression: Uniqueness

- Consider a 2D linear regression model trained to minimize the sum of squared errors: how many optimal solutions (i.e., sets of parameters w, b) are there for the given dataset?

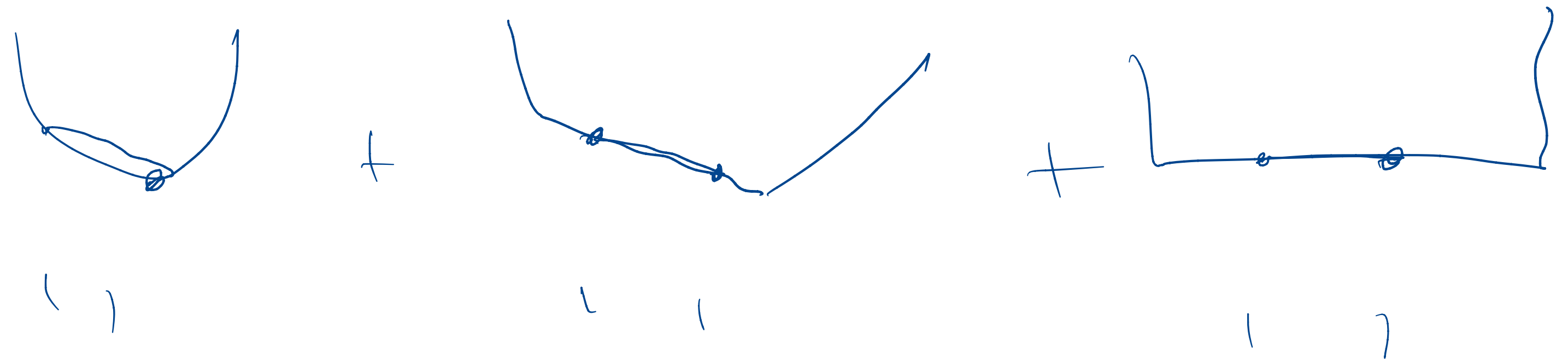


Linear Regression: Uniqueness

- Consider a 2D linear regression model trained to minimize the sum of squared errors: how many optimal solutions (i.e., sets of parameters w, b) are there for the given dataset?

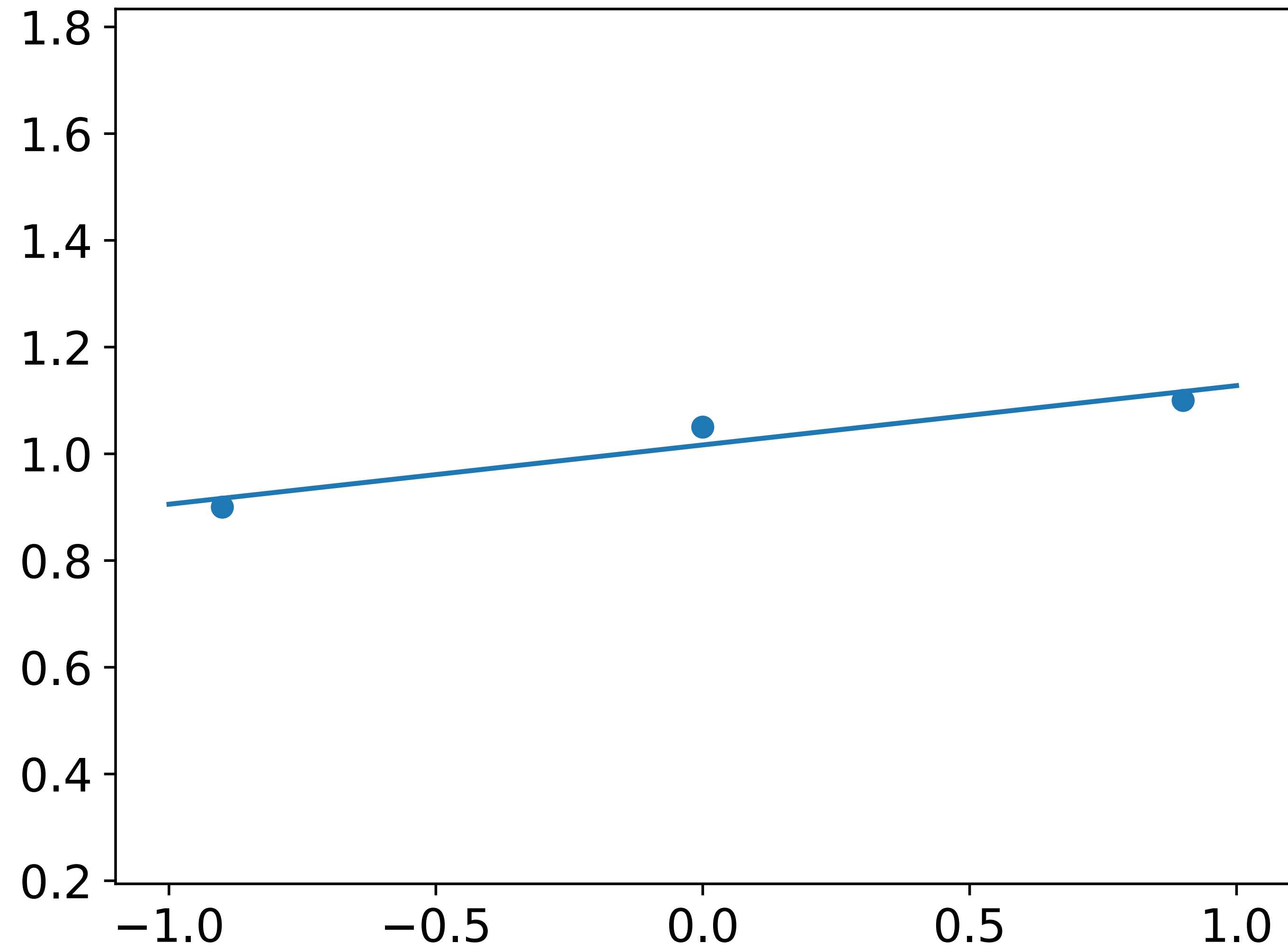


Uniqueness conditions

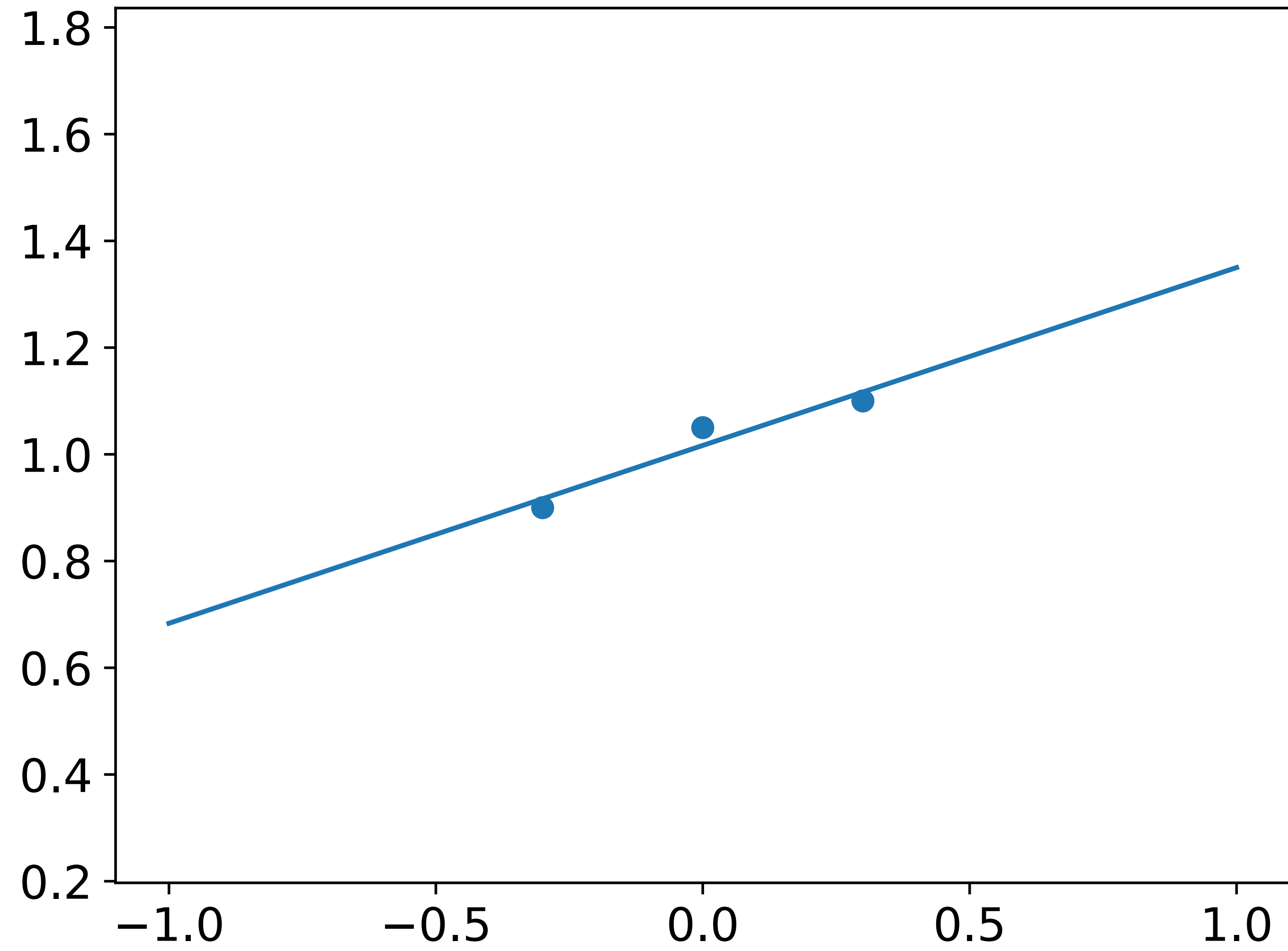


- Solution is unique if # examples N is *at least as large as* number of features k and all features are *linearly independent* (rank of X is k)
 - ▶ or if ridge parameter λ is strictly positive
- Can drop features to get independence, or add ridge, or resolve ambiguity by (e.g.) minimum norm solution

A tale of two datasets

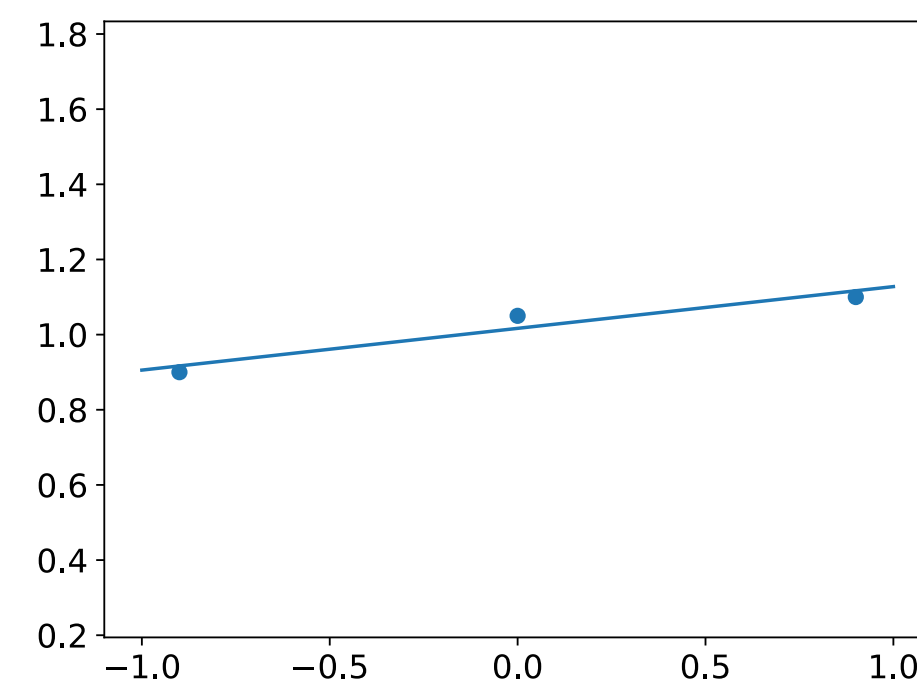
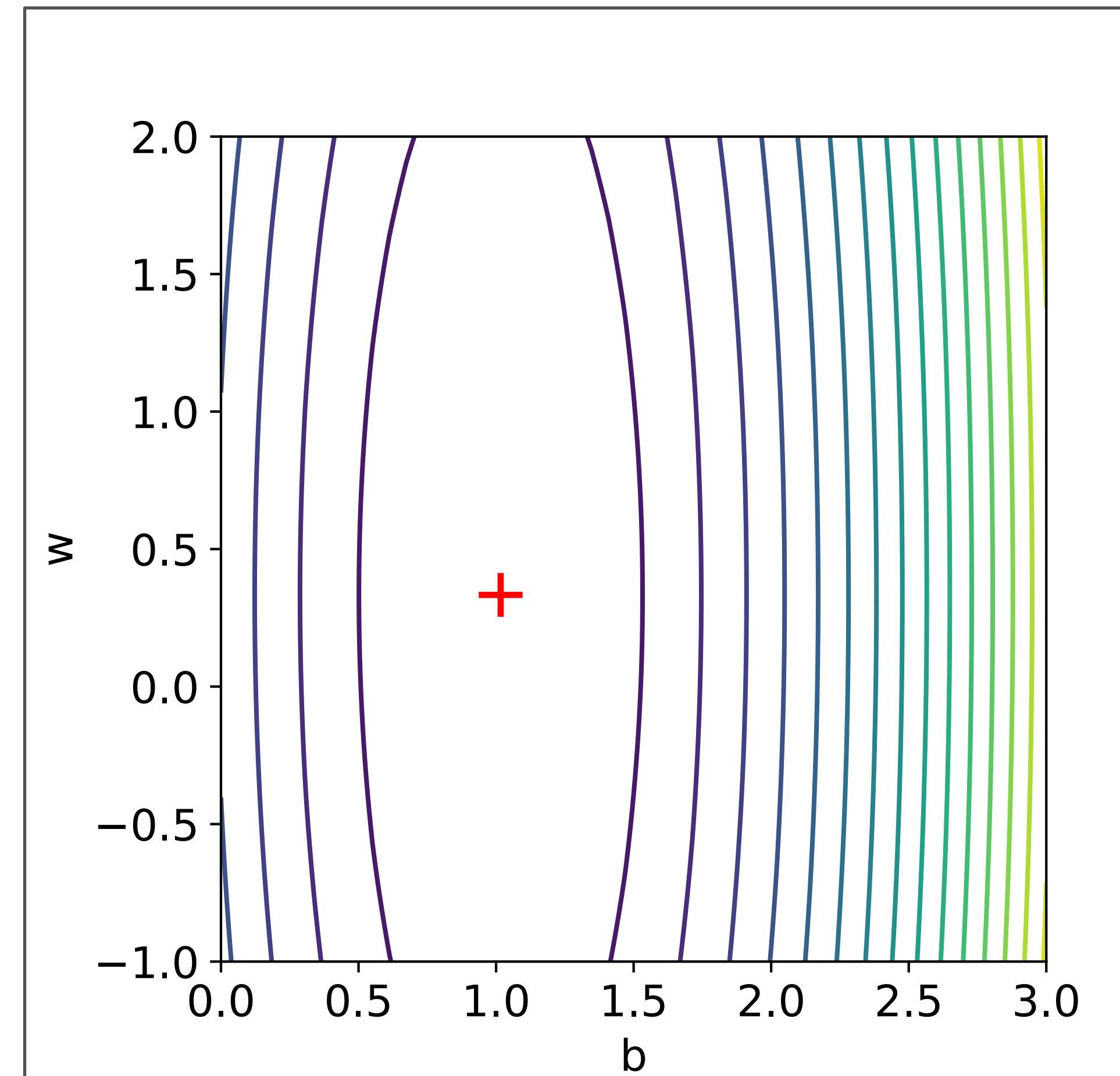
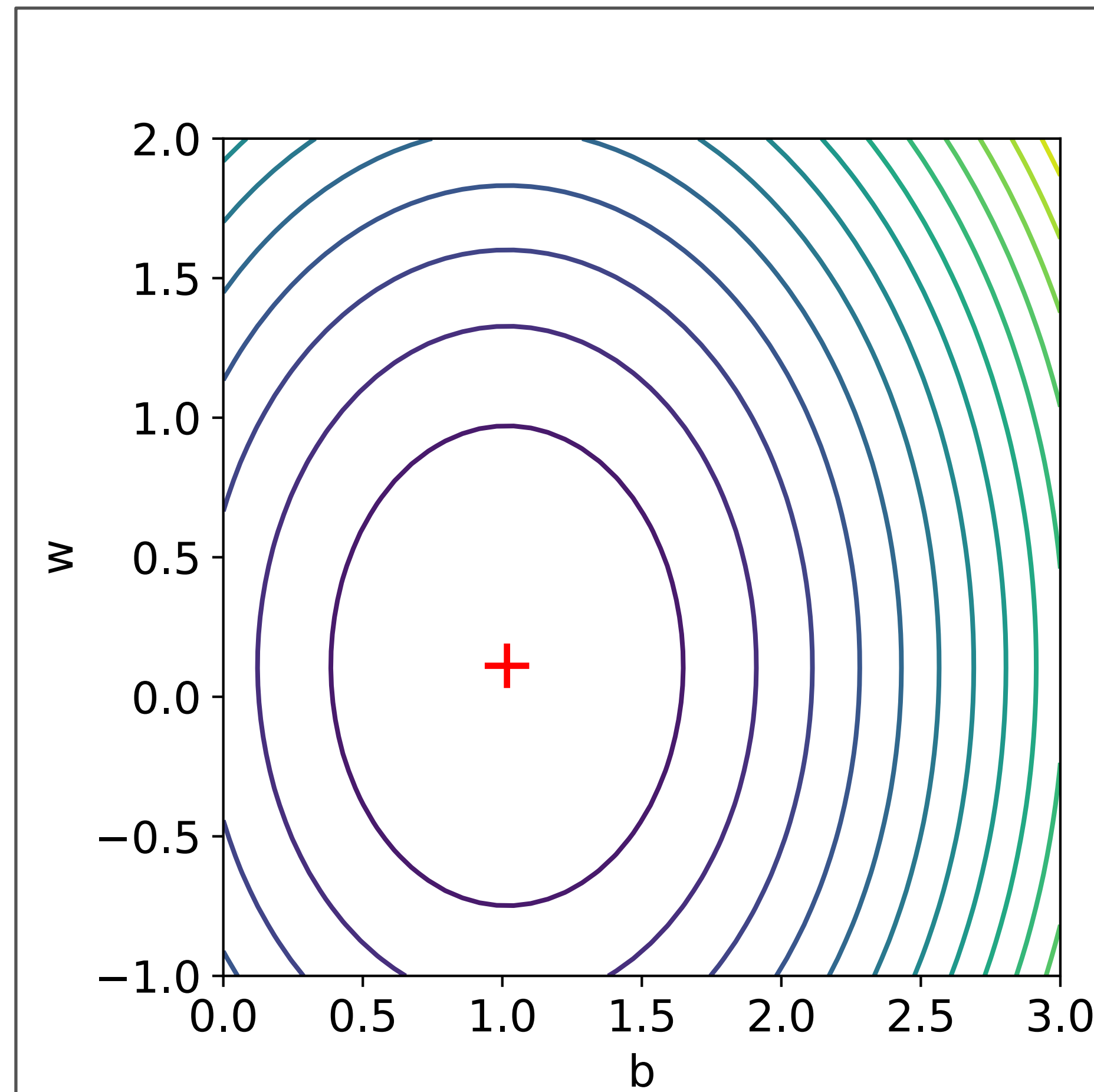


A tale of two datasets

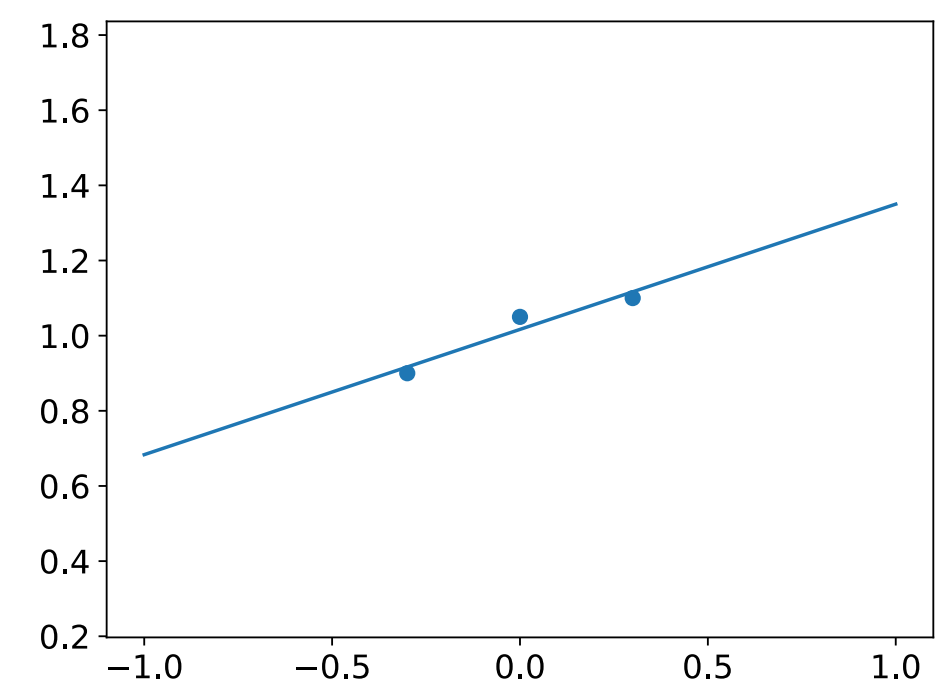


MSE loss

*non-unique solution is
the extreme example
of poor conditioning*

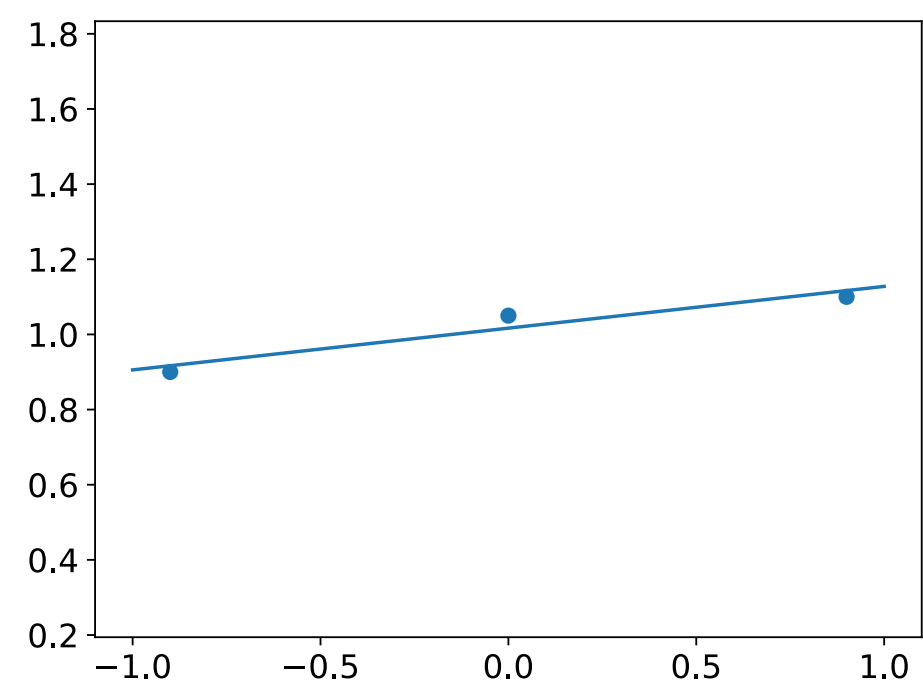


well conditioned

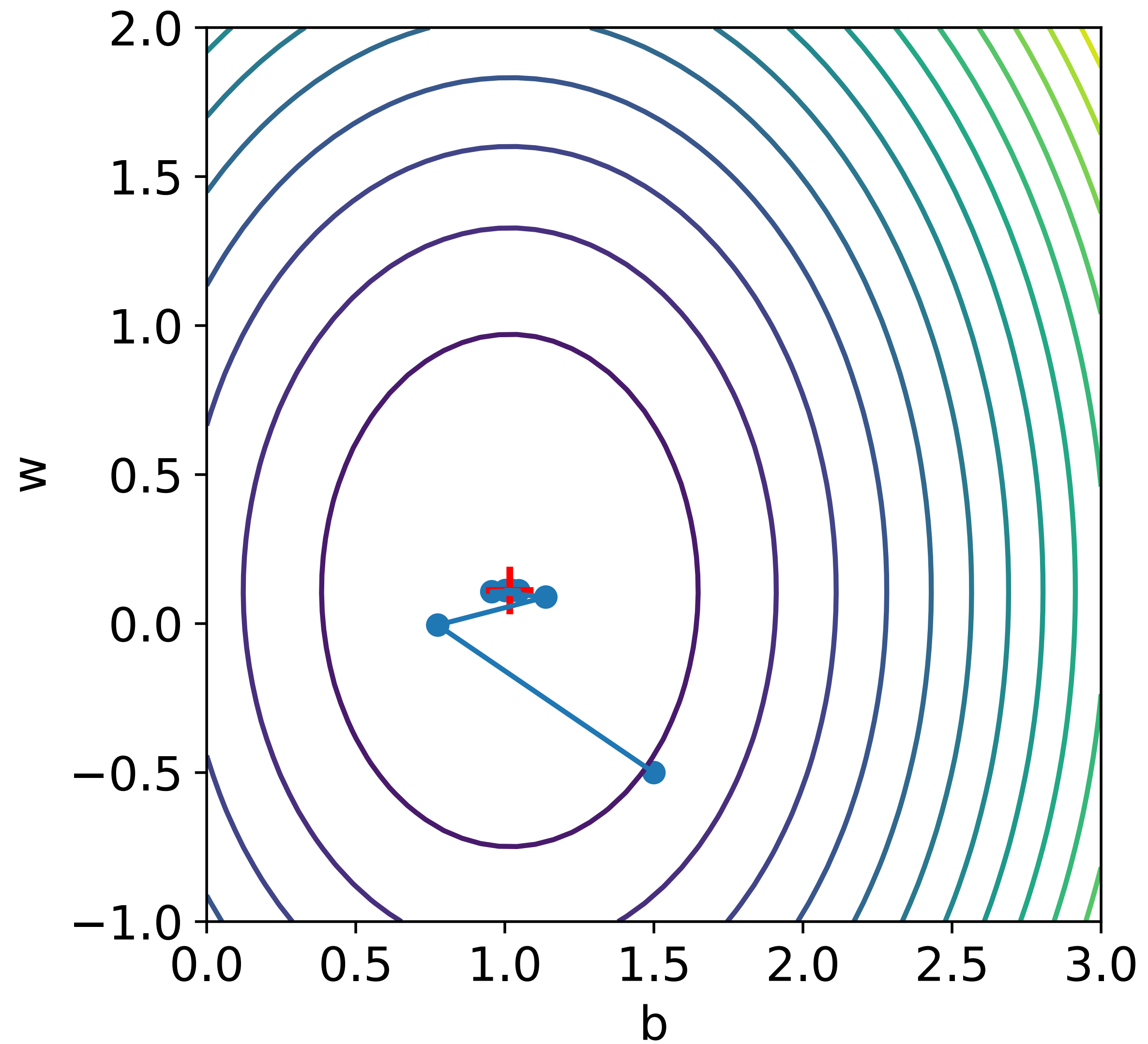


poorly conditioned

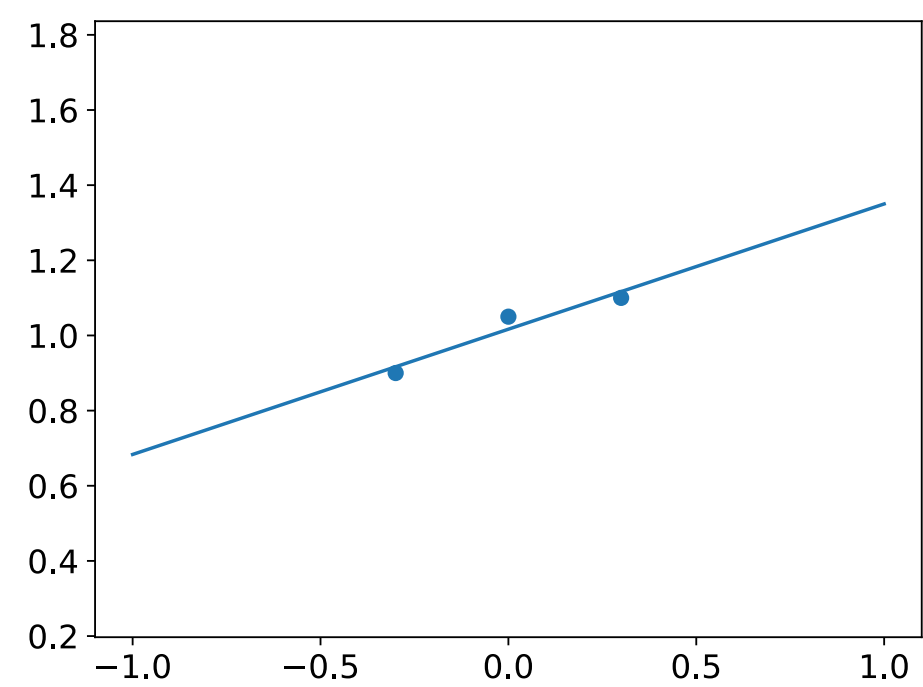
Gradient descent (good conditioning)



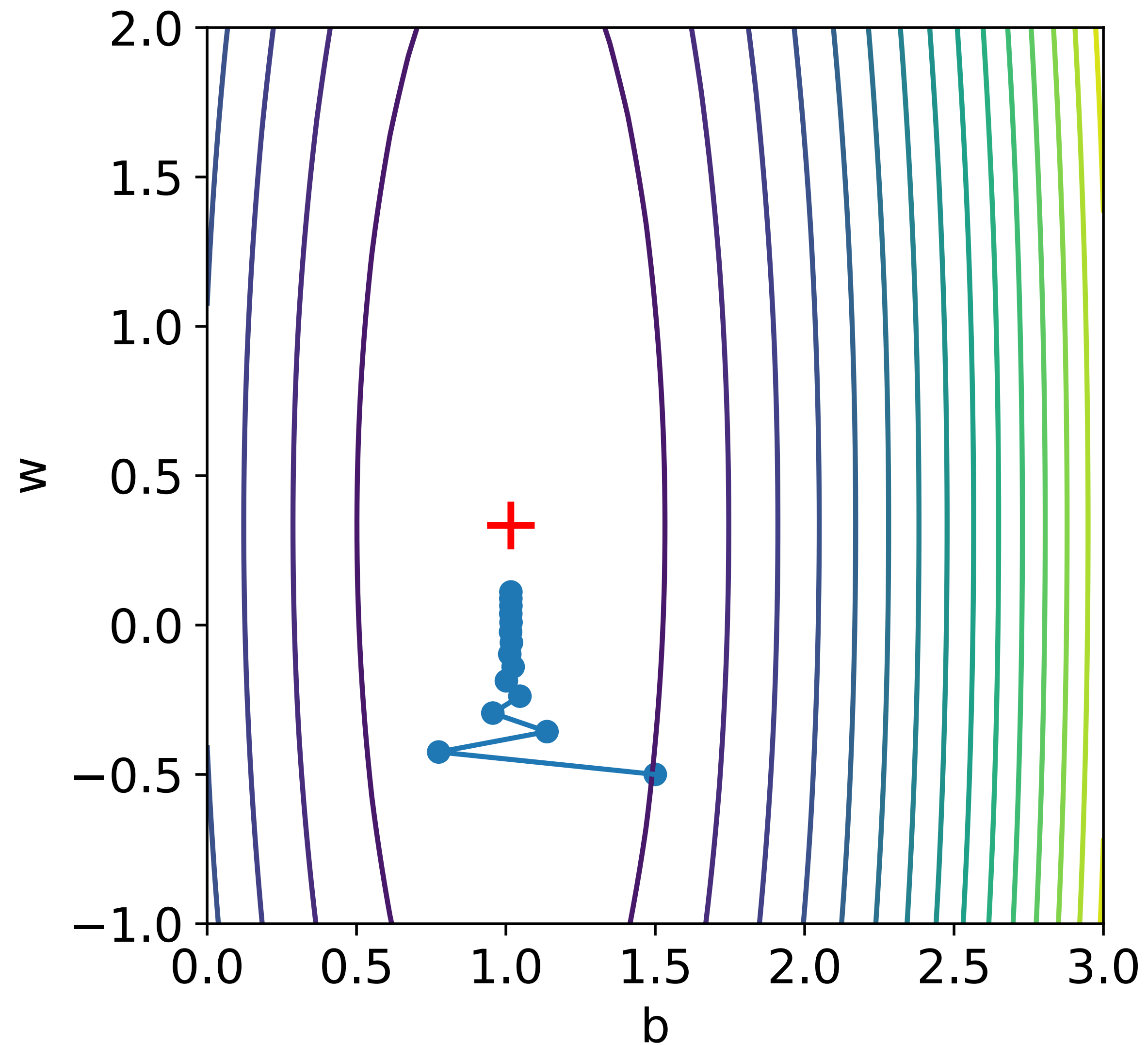
learning rate 0.5



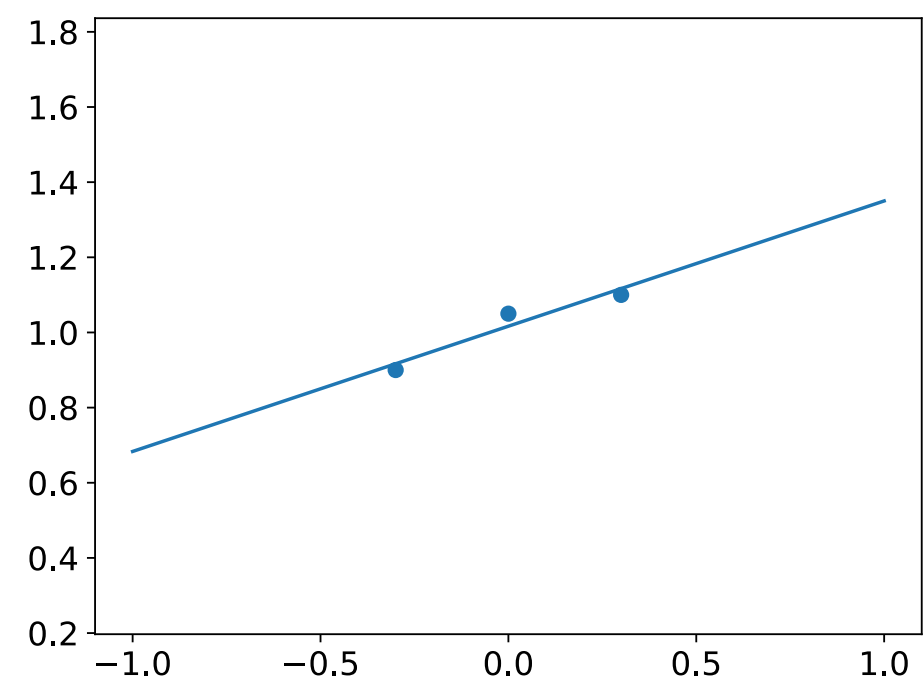
Gradient descent (poor conditioning)



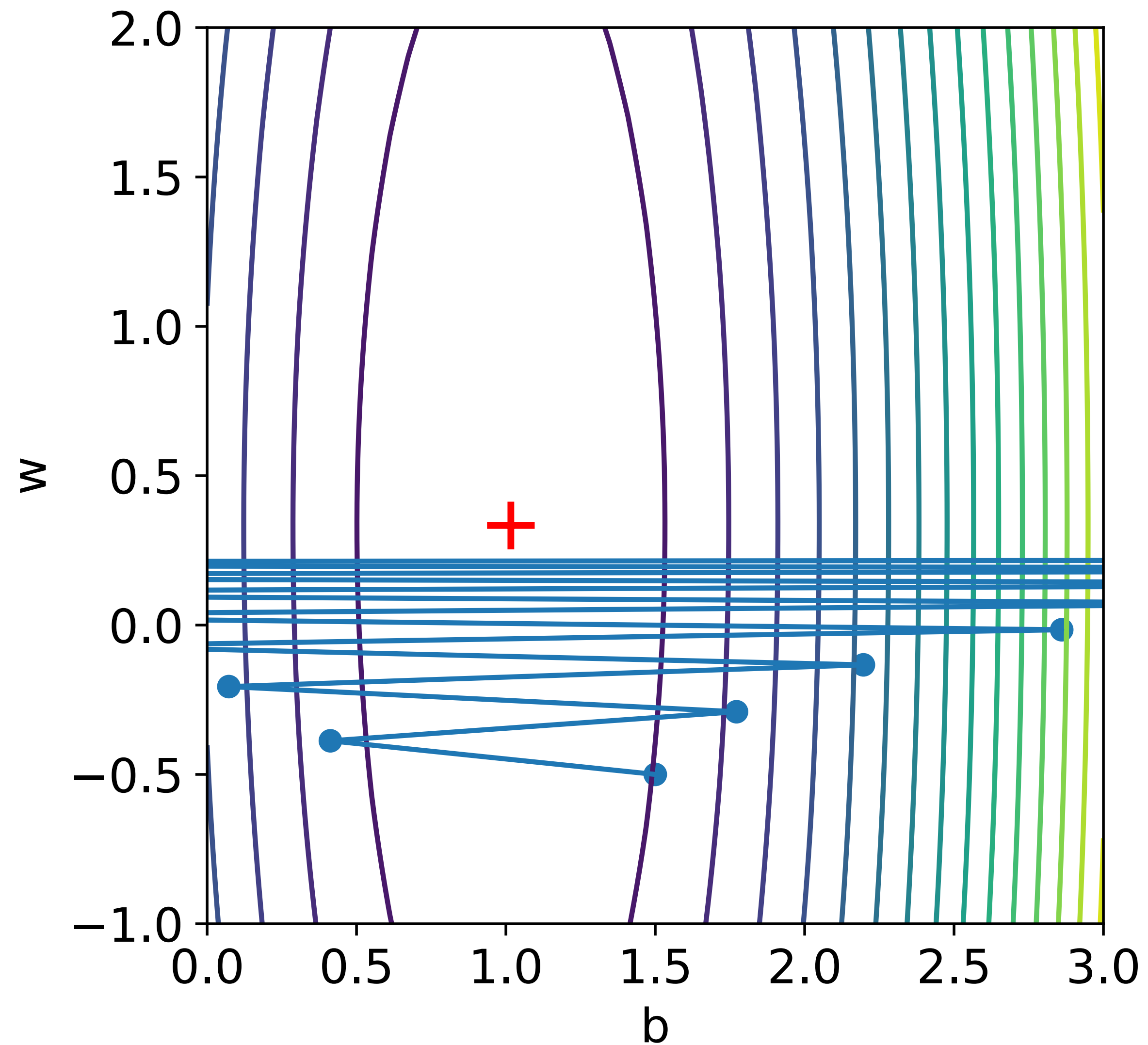
learning rate 0.5



Gradient descent (poor conditioning)



learning rate 0.75



What could we do?

- For horizontal direction (b), want a small learning rate to prevent oscillation
- For vertical direction (w), want a large learning rate to make fast enough progress
- Wouldn't it be great if we could automatically scale down the learning rate in directions where we start to oscillate?
 - ▶ then we'd be free to set learning rate high enough to make progress in other dimensions

EMA

Time scale: $\frac{1}{1-\beta}$
e.g., $\beta = 0.95 \rightarrow$ about 20 steps

- Exponential moving average

$$\bar{g} \leftarrow \beta \bar{g} + (1-\beta) g^{(t)}$$

if gradients always point $\uparrow$

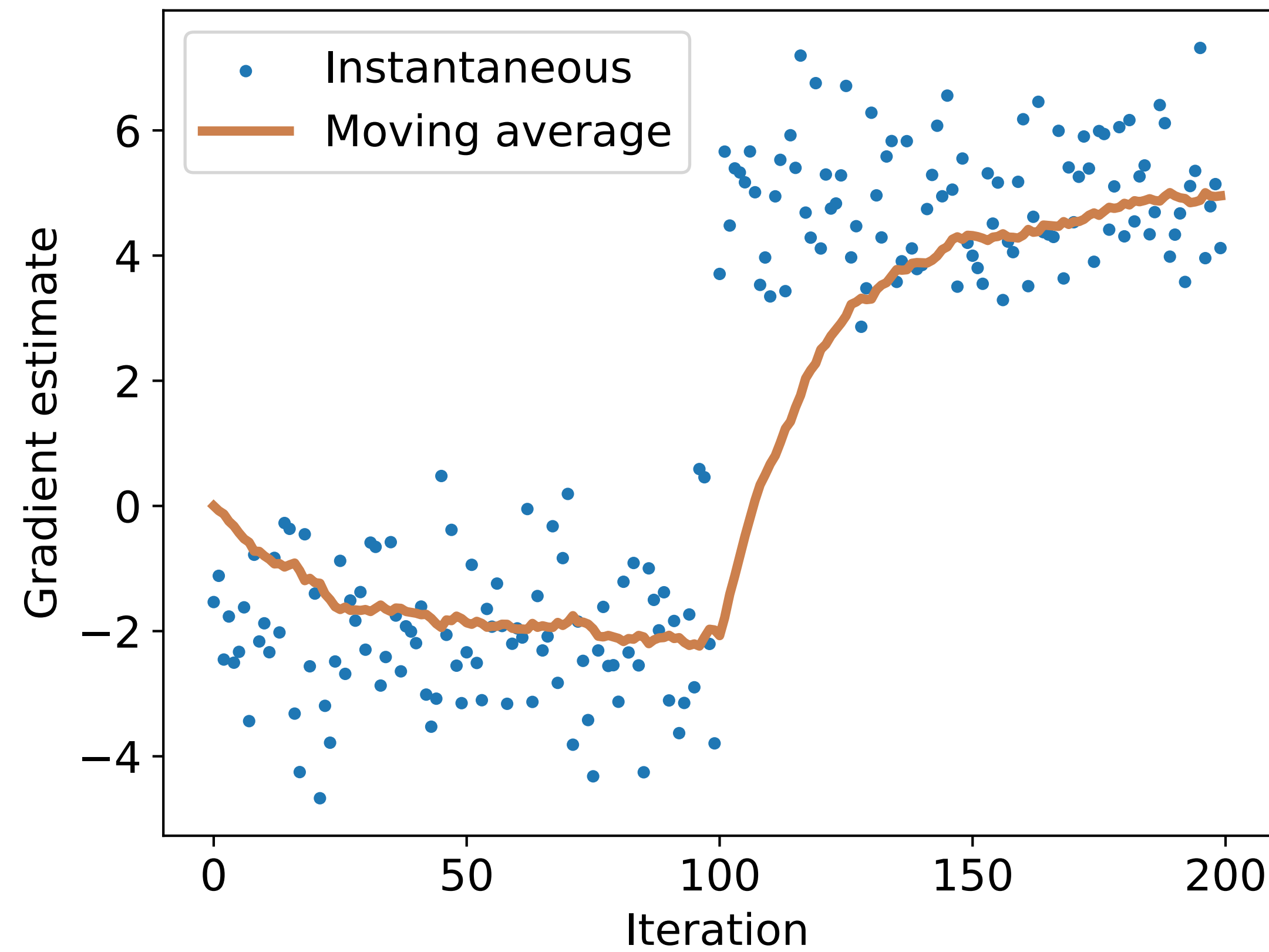


if gradients alternate $\uparrow \downarrow$



- Use EMA instead of gradient in GD: called **momentum**

EMA transient



$$\beta = 0.95$$

- EMA parameter β trades off *noise* vs. *transient*

$$\left(1 - \frac{1}{t}\right)^t \rightarrow \frac{1}{e}$$

as $t \rightarrow \infty$

Gradient descent with momentum

Called momentum because we tend to keep moving in same direction (even if gradient changes)

This is Polyak or heavy-ball momentum; others exist, notation may vary, documentation not always clear

Algorithm: gradient descent w/ momentum

Inputs: loss function $L(\theta)$, initial value $\theta^{(1)}$, step size η , momentum β

$\bar{g} \leftarrow 0$

For $i \leftarrow 1, 2, \dots$:

if **converged** break

$g \leftarrow \nabla L(\theta^{(i)})$

$\leftarrow$ for linear/ridge regression, $\lambda w + X^T(Xw - Y)$

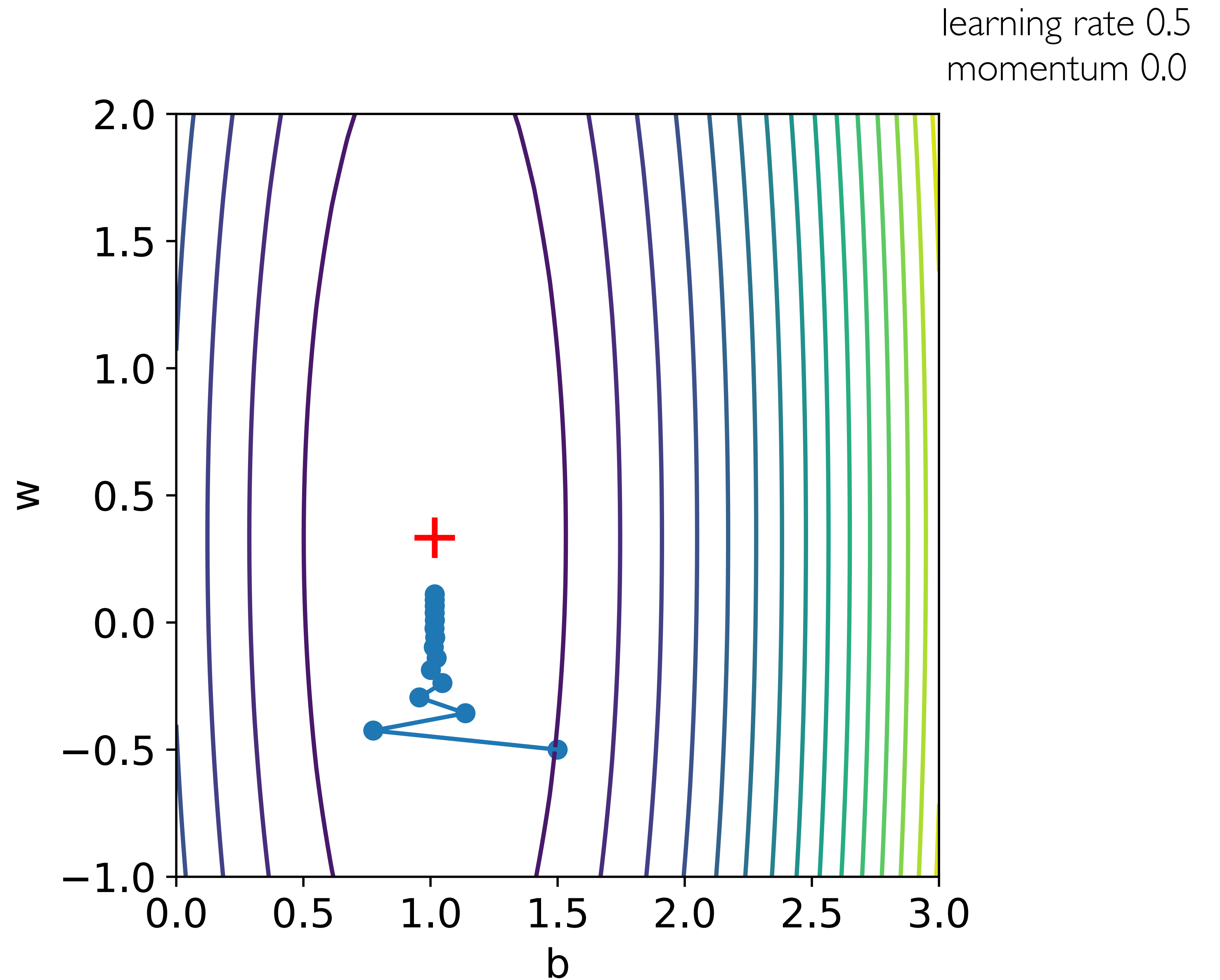
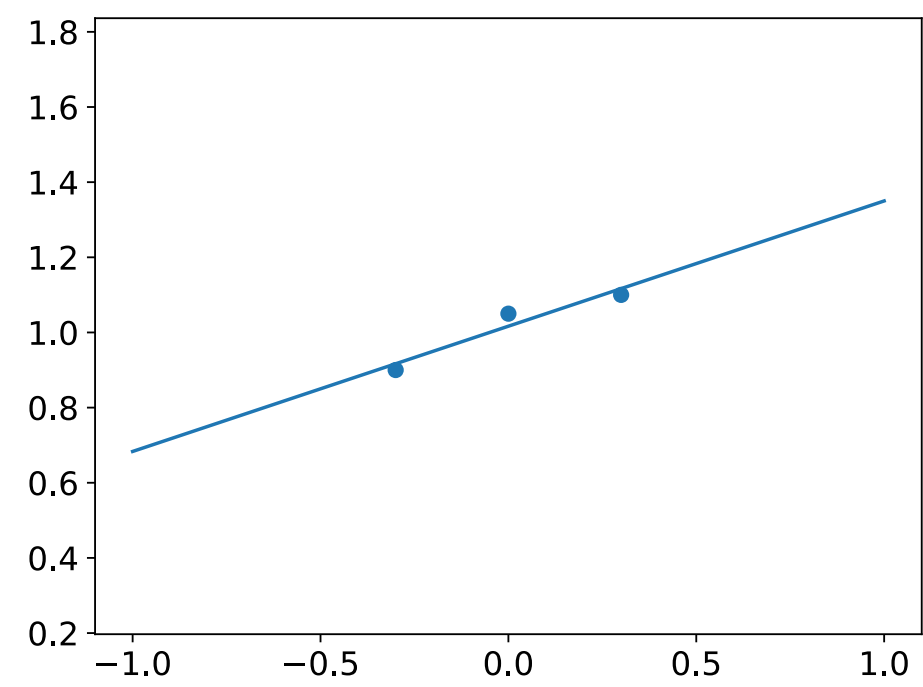
$\bar{g} \leftarrow \beta \bar{g} + (1 - \beta)g$

$\theta^{(i+1)} \leftarrow \theta^{(i)} - \eta \bar{g}$

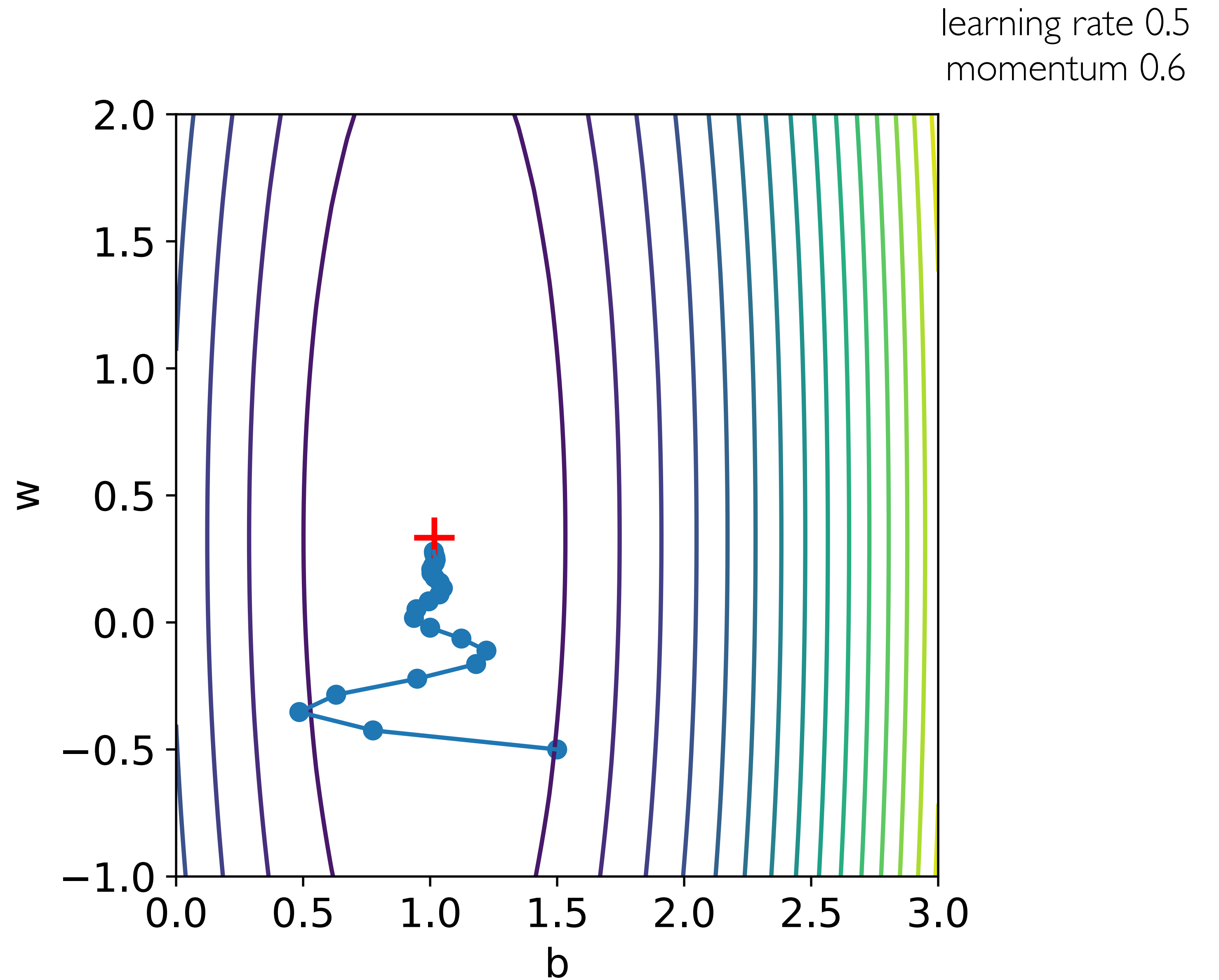
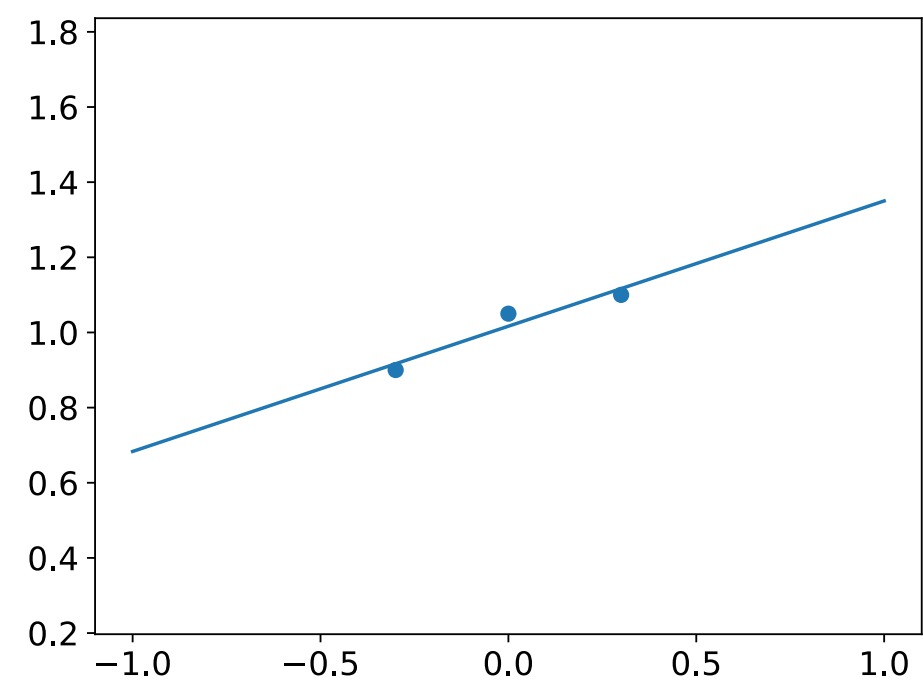
return $\theta^{(i)}$

- Similar convergence checks as above
- $\beta = 0$ is plain GD
- Variant: $\eta^{(i)}$ and $\beta^{(i)}$ (e.g., $\beta^{(i)} = 0, \beta, \beta, \beta, \dots$)
- Variant: scale $\bar{g}$ by $\frac{1}{1 - \beta^i}$ on step i

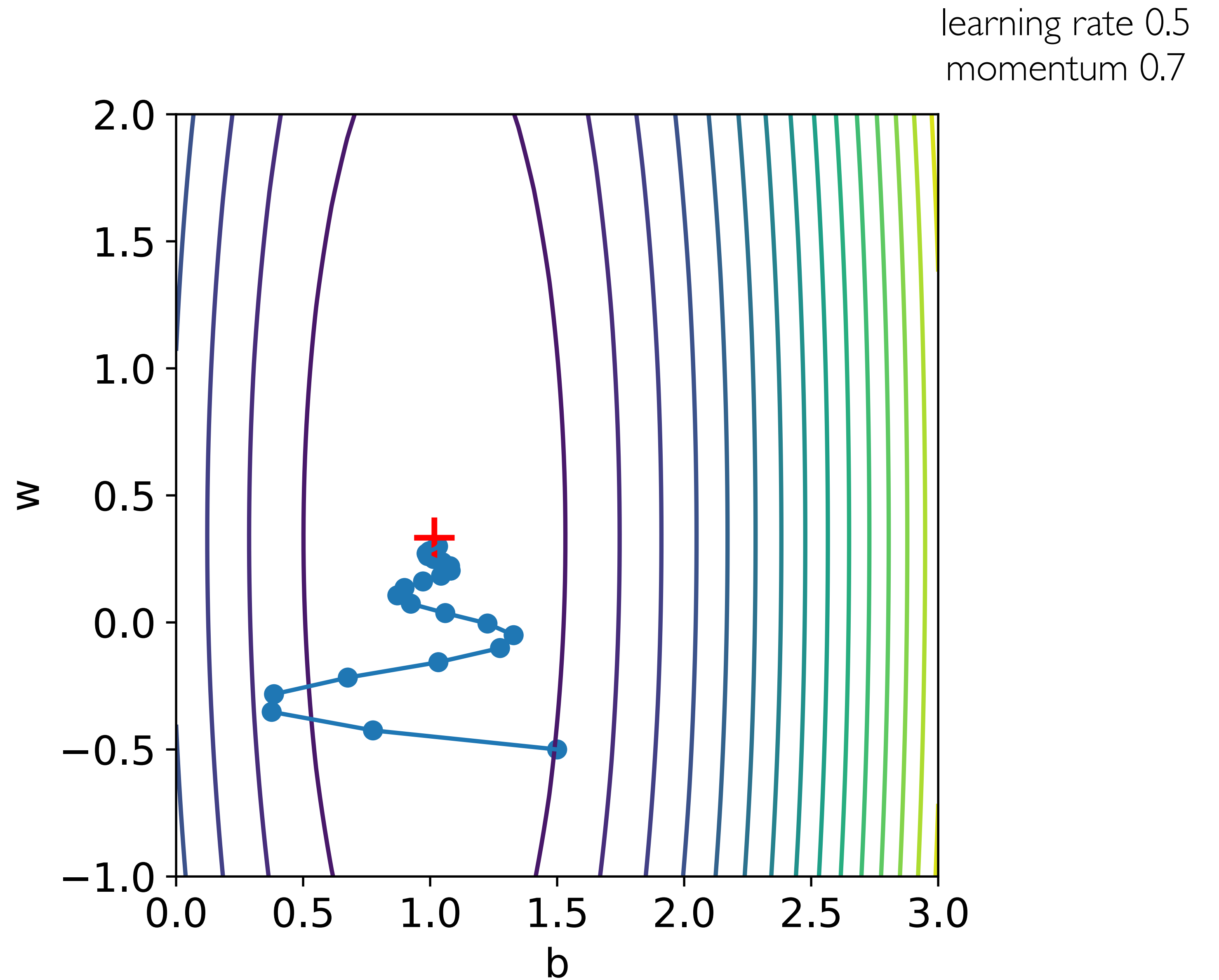
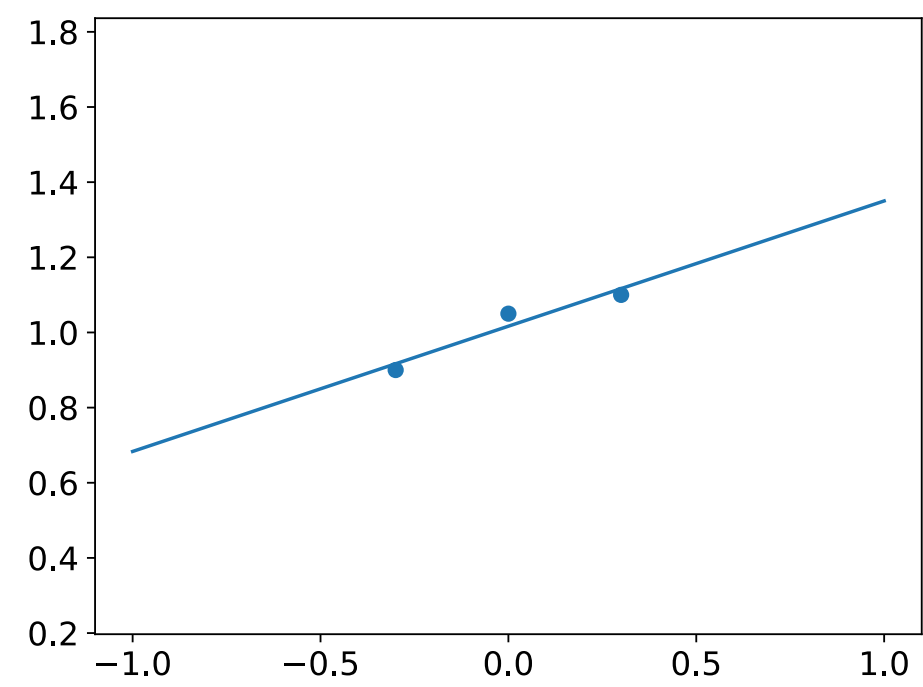
Gradient descent + momentum (poor conditioning)



Gradient descent + momentum (poor conditioning)



Gradient descent + momentum (poor conditioning)



Gradient descent + momentum (poor conditioning)

